

SQL SERVER & AZURE DBA

INTERVIEW HANDBOOK

2026 Edition

Professional DBA Interview Preparation

Always On Availability Groups • Distributed AG & Clustering • High Availability & Disaster Recovery
Security & Compliance • Performance Tuning • Azure SQL Database & Managed Instance
DevOps & Infrastructure as Code • AI for Database Administration • Microsoft Fabric & Analytics

23 Comprehensive Chapters

Professional Cheat Sheets, 100+ Interview Questions, Production Scenarios

Prepared by Ali Shaik

June 2026

Table of Contents

PART I — Core SQL Server Architecture & Administration 5

- Chapter 1 — SQL Server Architecture, Builds & System Databases 5
- Chapter 2 — Backup, Restore & Recovery Strategy 13
- Chapter 3 — Locking, Blocking, Deadlocks & Concurrency 17

PART II — High Availability & Disaster Recovery 20

- Chapter 4 — Always On Availability Groups (AG) 20
- Chapter 5 — Distributed Availability Groups (DAG) & Modern HA 30
- Chapter 6 — Failover Clustering, Quorum & WSFC 34
- Chapter 7 — Log Shipping, Database Mirroring & Replication 36

PART III — Performance Engineering 42

- Chapter 8 — Indexing, Statistics & Query Optimization 42
- Chapter 9 — Execution Plans, Optimizer Internals & Wait Statistics .. 47
- Chapter 10 — TempDB Internals & Contention 50

PART IV — Security & Compliance 55

- Chapter 11 — Encryption, Auditing & Compliance 55
- Chapter 12 — Identity, Active Directory & Microsoft Entra ID 58

PART V — Azure & Cloud Architecture 64

- Chapter 13 — Azure SQL Database (PaaS) 64
- Chapter 14 — Azure SQL Managed Instance (MI) 67
- Chapter 15 — SQL Server on Azure Virtual Machines (IaaS) 71
- Chapter 16 — Azure Networking for Data Platforms 74

PART VI — Migration & Modernization 77

- Chapter 17 — On-Premises ↔ Azure Migration Strategies 77
- Chapter 18 — SQL Server 2025/2026 & Microsoft Fabric for DBAs 82

PART VII — Modern DBA Operating Model 84

- Chapter 19 — DevOps, Automation & Infrastructure as Code for DBAs ... 84
- Chapter 20 — AI for Database Administration 88

Chapter 21 — Monitoring, Observability & Alerting	92
---	----

PART VIII — Real-World Practice 96

Chapter 22 — Production Incident Case Studies & Root Cause Analysis .	96
---	----

Chapter 23 — Patching, Upgrades & Change Management	101
---	-----

Appendix A — Quick Revision Cheat Sheets	105
--	-----

Appendix B — Top 100 Interview Questions - Rapid Fire	109
---	-----

Appendix C — Top 50 Real-Time Production Scenarios	114
--	-----

Part I — Core SQL Server Architecture & Administration

This part rebuilds the fundamentals a Senior or Lead DBA is expected to recite without hesitation: build and servicing terminology, system databases, startup and memory configuration, integrity checking, and the locking/blocking/deadlock model that underlies every concurrency question in an interview. Treat this part as the foundation that every later chapter — Always On, performance tuning, Azure — assumes you already know cold.

Chapter 1 · SQL Server Architecture, Builds & System Databases

1.1 Concept Overview

Every SQL Server instance is a layered stack: the SQLOS scheduling layer, the relational/storage engine, and a small set of system databases that the engine relies on to function. A Senior DBA needs to be fluent in how the engine identifies its own version and patch level, what each system database is for, how the instance starts and recovers, and how to diagnose problems using DBCC and DMVs before reaching for a restart.

1.2 Versioning & Servicing Terminology

“Build” refers to the specific internal version number of the SQL Server binaries (e.g. 16.0.4165.4 for a SQL Server 2022 CU), returned by `SELECT @@VERSION` or `SERVERPROPERTY('ProductVersion')`. Microsoft ships fixes through three vehicles, and confusing them is a common interview trip-wire:

- Hotfix — addresses one specific issue or a narrow set of issues for the customers who hit it; documented per-fix; not broadly regression-tested for the whole product.
- Cumulative Update (CU) — a tested, cumulative rollup of all prior hotfixes plus new fixes that meet the hotfix-acceptance bar (workaround availability, customer impact, reproducibility, complexity of the change). This is Microsoft's primary servicing model since SQL Server 2016.
- Service Pack (SP) — a fully tested, cumulative bundle of all hotfixes/updates plus a limited set of customer-requested design changes. Service Packs effectively stopped after SQL Server 2017; from 2017 onward, CUs are the only vehicle.

1.3 System Databases

Database	Purpose	DBA notes
master	Tracks all logins, instance-level configuration, linked servers, and the existence/location of every other database.	If master is lost, the instance cannot start; always back it up after any login/config/database-create change.
model	Template applied to every newly created database (default file sizes, recovery model, options).	Changing model affects only databases created afterward.
msdb	SQL Agent jobs, schedules, alerts, operators, backup/restore history, Database Mail, SSIS catalog, Service Broker.	Can normally be backed up/restored like a user database; cannot be detached by default.
tempdb	Temp tables, table variables, work tables for sorts/spools/hash joins, row versioning (snapshot isolation, triggers, MARS, online index rebuilds).	Recreated from model on every restart; the only system DB that should always have multiple, equal-sized data files.
Resource DB (mssqlsystemresource)	Hidden, read-only database holding all system objects (sys.* views, stored procedures) shipped with the engine.	Not visible in SSMS object explorer by default; upgraded automatically with each build, enabling fast side-

	by-side rollback of system-object definitions.
--	--

1.4 Startup, Memory & Configuration

Startup parameters

Startup parameters are configured in SQL Server Configuration Manager (or sqlservr.exe command line) and control how the engine boots:

- -d — path to the master data file (master.mdf).
- -l — path to the master log file (mastlog.ldf).
- -e — path to the error log file.
- -c — start as a console application, bypassing the Service Control Manager (faster diagnostic restarts).
- -f — start in minimal configuration mode (single user, minimal tempdb size, ignores some configuration options) — used to recover from corrupted system configuration or to detach a damaged msdb.
- -m or -m"SQLCMD" — start in single-user mode, optionally restricting the single connection to a named client application.
- -T — enable a global trace flag at startup (e.g. -T3608 starts the engine without auto-recovering any database except master, used in disaster recovery and msdb detach scenarios).

Memory configuration

On a dedicated, standalone box, a widely used starting rule of thumb is to reserve roughly 30% of physical RAM for the OS/other processes and grant the remainder to SQL Server's max server memory; on a server hosting multiple instances, divide the SQL Server share across instances proportionally to workload, never letting the sum exceed the safe ceiling for the OS. Two key counters to compare:

- Total Server Memory — what the engine is currently using.
- Target Server Memory — the ceiling the engine is trying to reach (driven by max server memory and what the OS will grant).

If Total is persistently lower than Target shortly after a restart, that is expected — the buffer pool is still ramping up. If Total stays well below Target for a long-running instance, max server memory may be configured too high relative to actual demand. If Total is ever greater than Target, treat it as a signal of memory pressure.

1.5 DBCC, Integrity Checks & Corruption Recovery

Routine DBCC commands a production DBA reaches for:

- DBCC CHECKDB — full allocation, catalog, and table-structure integrity check (internally runs CHECKALLOC, CHECKCATALOG, and CHECKTABLE).
- DBCC CHECKDB WITH PHYSICAL_ONLY — checks physical page/record-header integrity and allocation consistency only; much cheaper, suitable for daily runs on very large databases.
- DBCC CHECKDB WITH EXTENDED_LOGICAL_CHECKS — adds logical checks for indexed views, XML, spatial indexes; run weekly/at lower-traffic windows.
- DBCC CHECKDB WITH ESTIMATEONLY — reports the tempdb space CHECKDB would need, without actually running the checks.

- DBCC DBINFO — exposes metadata including dbi_dbccLastKnownGood, used to find when CHECKDB last completed successfully.
- DBCC SHOWCONTIG / sys.dm_db_index_physical_stats — fragmentation analysis.
- DBCC SHOW_STATISTICS — inspect a statistics object's header, histogram, and density vector.

PRODUCTION PLAYBOOK — Running CHECKDB on a 30-40 TB database

Divide tables roughly evenly across 7 logical buckets.

Run DBCC CHECKALLOC twice a week (cheap, catches allocation corruption fast).

Run DBCC CHECKCATALOG once a week (metadata consistency).

Run DBCC CHECKTABLE against one bucket per day, so the full database is covered weekly without ever running a single multi-day CHECKDB.

Tune MAXDOP for CHECKDB — more worker threads read pages faster on multi-core hardware.

Watch tempdb I/O — CHECKDB stages fact/aggregation data in tempdb, so slow tempdb storage throttles the whole check.

If zero downtime is mandated, run checks against a readable secondary in an Always On AG instead — the storage differs from primary, but it still validates data-page integrity.

Use PHYSICAL_ONLY on weekdays and EXTENDED_LOGICAL_CHECKS on weekends as a lighter-weight compromise.

Suspect & corrupted databases

- A database flips to SUSPECT when SQL Server cannot complete crash recovery (e.g. unreadable log or data pages).
- Detaching a SUSPECT database was possible through SQL Server 2005, but is blocked from SQL Server 2008 onward.
- Recovery path: place the database in EMERGENCY mode (read-only, single-user, bypasses normal recovery), run DBCC CHECKDB, and only as a last resort use REPAIR_ALLOW_DATA_LOSS, which can delete pages/rows to restore structural consistency.
- After REPAIR_ALLOW_DATA_LOSS, always reconcile row counts per table against a pre-incident baseline to identify exactly what was lost — never assume zero data loss just because the repair completed.
- msdb cannot be detached under normal conditions; if msdb is corrupted and there is no backup, start the instance in minimal configuration mode with trace flag 3608 (which skips auto-recovery of all databases except master) and then detach msdb manually.

```
net start MSSQLSERVER /T3608 /f /m"SQLCMD"
SQLCMD -E -S . -Q "EXEC sp_detach_db msdb"
```

1.6 Heaps, Filegroups & Schema Notes

- A heap is a table with no clustered index; rows have no enforced physical order. Heaps are common on staging/landing tables where bulk-load speed matters more than read pattern.
- Transaction log files never belong to a filegroup — filegroups apply only to data files.
- A table can be created on a single filegroup explicitly with CREATE TABLE ... ON [FilegroupName]; without an ON clause it lands on the default filegroup.
- Adding a new column to a large table is a metadata-only operation in modern SQL Server as long as the column is nullable or has no default value requiring a table rewrite; adding a NOT NULL column with a

default on a pre-2012 engine (or certain edge cases) can trigger a full table rewrite and a large log spike — always check whether the operation will be an online metadata change before running it against a multi-billion-row table.

1.7 Interview Questions — Beginner

Q1. What is a SQL Server build, and how do you check it?

A: The build is the specific internal version/patch number of the engine binaries. Check it with `SELECT @@VERSION`, or more reliably `SELECT SERVERPROPERTY('ProductVersion')`, `SERVERPROPERTY('ProductLevel')`, `SERVERPROPERTY('Edition')`.

Q2. What are the system databases in SQL Server, and what is each one for?

A: master (instance/login/database catalog), model (template for new databases), msdb (Agent jobs, backup history, SSIS, Service Broker), tempdb (temp objects, sorts, row versioning), and the hidden Resource database (read-only system objects). See the table above for detail.

Q3. What are startup parameters?

A: Flags passed to `sqlservr.exe` (configured via Configuration Manager) that control how the instance boots — for example `-d/-l` for the master database file paths, `-e` for the error log, `-m` for single-user mode, and `-T` for a startup trace flag.

Q4. How do you start SQL Server in single-user mode?

A: Add the `-m` startup parameter (optionally `-m"AppName"` to restrict the single connection to a named application) via Configuration Manager, or start `sqlservr.exe -m` from the command line, then connect with a single admin connection (SQLCMD or SSMS using the DAC).

Q5. What is a heap table?

A: A table with no clustered index. Rows are stored with no guaranteed physical order, identified internally by a Row ID (file:page:slot) rather than a clustering key.

Q6. How do you check the logical CPU/processor count visible to SQL Server?

A: `SELECT cpu_count FROM sys.dm_os_sys_info`; this reports logical processors, which matters when comparing licensed/visible cores against MAXDOP and affinity settings.

Q7. How do you find out when SQL Server was last restarted?

A: `SELECT sqlserver_start_time FROM sys.dm_os_sys_info`; alternatively, the timestamp of tempdb's creation date (`SELECT create_date FROM sys.databases WHERE name='tempdb'`) approximates the last restart, since tempdb is rebuilt every startup.

Q8. Which filegroup does the transaction log live in?

A: None — filegroups are a data-file concept only. The transaction log is always a separate, non-filegrouped structure.

1.8 Interview Questions — Intermediate

Q9. Differentiate a hotfix, a cumulative update, and a service pack.

A: A hotfix targets one specific issue for the customers who hit it; a Cumulative Update (CU) is a tested rollup of all prior hotfixes plus new fixes meeting the hotfix bar, and is Microsoft's primary servicing vehicle from SQL Server 2016 onward; a Service Pack was a larger tested bundle of hotfixes plus limited feature/design changes, discontinued after SQL Server 2017.

Q10. What is the use of the Resource database?

A: It is a hidden, read-only system database holding every system object (system views, stored procedures, functions) shipped with the engine, decoupled from user databases so Microsoft can upgrade system objects atomically with each build and roll them back quickly if needed.

Q11. Can we detach a SUSPECT database?

A: Not from SQL Server 2008 onward (it was permitted through 2005). The supported path is EMERGENCY mode plus DBCC CHECKDB, with REPAIR_ALLOW_DATA_LOSS only as a last resort.

Q12. Can we detach msdb?

A: Not under normal circumstances. If msdb itself is corrupted with no usable backup, start the instance with trace flag 3608 in minimal/single-user mode (which skips auto-recovery of every database except master) and detach msdb manually with sp_detach_db.

Q13. How will you recover a SUSPECT database?

A: Put it into EMERGENCY mode, run DBCC CHECKDB to assess the damage, and attempt repair with the least destructive option first; REPAIR_ALLOW_DATA_LOSS is the last resort and must be followed by a row-count reconciliation against a known-good baseline to quantify exactly what was lost.

Q14. What does DBCC CHECKDB WITH ESTIMATEONLY return?

A: Only the amount of tempdb space CHECKDB would need to check the specified user database — it does not perform the actual integrity check.

Q15. When do you need to place a database in EMERGENCY mode, and is it required for every corruption?

A: Only when the database is otherwise inaccessible (for example, stuck SUSPECT). Not every corruption needs EMERGENCY mode — many corrupt-page or corrupt-index issues can be resolved online with targeted DBCC CHECKTABLE/REPAIR or by restoring the affected pages from a clean backup, while the database stays online.

Q16. How will you troubleshoot a log file growth issue that is causing disk space problems?

A: Identify the cause first with sys.databases.log_reuse_wait_desc — common causes are ACTIVE_TRANSACTION, LOG_BACKUP (no log backups being taken on a FULL/BULK_LOGGED database), REPLICATION, ACTIVE_BACKUP_OR_RESTORE, or AVAILABILITY_REPLICA (slow secondary holding the log open).

If the cause is missing log backups, take one immediately — that is almost always the fix, not a manual shrink.

Identify and break up any single oversized transaction (large deletes, bulk loads) into batches with explicit smaller transactions.

Only shrink the log file (DBCC SHRINKFILE) after the root cause is resolved and the log has actually emptied — shrinking without fixing the cause just causes the file to grow right back.

Q17. Due to a memory issue, you cannot even open Management Studio and the server is unresponsive. How do you troubleshoot?

A: Connect using the Dedicated Administrator Connection (DAC) via SQLCMD (sqlcmd -A) rather than SSMS, since the DAC reserves a connection even under severe memory pressure.

Once connected, query `sys.dm_os_memory_clerks`, `sys.dm_os_process_memory` and Total/Target Server Memory to confirm whether SQL Server itself, or another process on the box, is consuming memory.

Use `sys.dm_exec_requests` / `sys.dm_exec_sessions` to find long-running or memory-intensive queries (large sorts/hashes/spills) and Resource Governor or kill the offending session if necessary.

Check the SQL Server error log for out-of-memory or RESOURCE_MONITOR warnings written just before the slowdown.

1.9 Interview Questions — Advanced & Scenario-Based

Q18. What new features should a 2026-era DBA know across recent SQL Server releases (2019 -> 2022 -> 2025)?

A: 2019: Accelerated Database Recovery (ADR) and the persistent version store, Intelligent Query Processing (table variable deferred compilation, approximate COUNT DISTINCT, scalar UDF inlining), Always On secondary-to-secondary readable replica scale-out, and clustered columnstore enhancements.

2022: Contained Availability Groups (config and logins move with the AG), Query Store on secondary replicas, Always Encrypted with secure enclaves, an enhanced Last-Known-Good DBCC CHECKDB tracking, and built-in support for Azure Synapse Link / Object storage integration (S3-compatible, in later updates).

2025 (GA Nov 2025): native AI (vector data type/vector index, ready for embeddings/RAG inside T-SQL), native JSON type with JSON indexes, built-in REST endpoints and Change Event Streaming to Event Hubs, RegEx in T-SQL, the Optimized Locking database option (removes most lock escalation without code changes), a tempdb resource-governor sandboxing option, mirroring straight into Microsoft Fabric OneLake, and credential-free (managed identity) connections to Azure Storage/Key Vault/REST endpoints.

Q19. A developer needs to widen a 1 TB table's identity column from int to bigint without hours of downtime or hundreds of GB of log. How do you do it in seconds?

A: Enable data compression (row or page) on the heap/clustered table and on every non-clustered index on the table.

Once every index is compressed, the `ALTER TABLE ... ALTER COLUMN ... BIGINT NOT NULL` becomes a metadata-only operation rather than a full data rewrite, completing in roughly a second regardless of table size, instead of generating hundreds of GB of transaction log over many hours.

After the change, compression can be left in place or rolled back with `ALTER INDEX ... REBUILD ... WITH (DATA_COMPRESSION = NONE)` if it is not otherwise wanted.

Q20. Total Server Memory is consistently greater than Target Server Memory — what does that tell you?

A: Total exceeding Target is a sign of external memory pressure (the OS or another process is forcing SQL Server below its configured ceiling) and should be investigated immediately; under normal healthy operation Total approaches but does not exceed Target.

Q21. How do you adjust memory when several SQL Server instances share one box (standalone and clustered)?

A: Start from roughly 30% of RAM reserved for the OS, then divide the remaining 70% across instances by workload weight, validating with `perfmon`/DMV memory counters over a representative load window and adjusting max server memory per instance rather than leaving it at the (dangerous) unlimited default.

Q22. How will you change the SQL Server port number, and does it require a restart?

A: Change the TCP port in SQL Server Configuration Manager under the instance's Protocols > TCP/IP > IP Addresses properties (or via the registry/PowerShell for automation). Yes — a restart of the SQL Server service is required for the new port to take effect.

Q23. What considerations apply when adding a new column to a very large existing table?

A: Check whether the addition is a metadata-only operation (nullable column, or NOT NULL with no default needing back-fill on modern engine versions) versus a full table rewrite; consider the impact on existing query plans/statistics, replication/CDC schema synchronization, downstream ETL contracts, and — for AG environments — the log volume that will be shipped to every secondary.

1.10 Quick Revision Notes

- Build = exact patch level; CU is the modern servicing vehicle (SPs stopped after 2017).
- 5 system DBs: master, model, msdb, tempdb, Resource DB (hidden, read-only).
- -f = minimal config; -m = single user; -T3608 = skip auto-recovery except master (msdb rescue).
- SUSPECT databases cannot be detached (blocked since 2008) — use EMERGENCY mode + CHECKDB instead.
- CHECKDB = CHECKALLOC + CHECKCATALOG + CHECKTABLE; use PHYSICAL_ONLY/bucketing for huge databases.
- Log file never has a filegroup. Heap = table without a clustered index.
- Compress every index before widening a huge int -> bigint column to make it metadata-only.

Chapter 2 · Backup, Restore & Recovery Strategy

2.1 Concept Overview

Backup and restore strategy is the single most interview-tested topic in DBA hiring because it is the one area where a mistake is unrecoverable. A Senior DBA must know recovery models cold, understand exactly how full/differential/log backups relate to the LSN chain, and be able to reason about tail-log backups, copy-only backups, and the constraints that High Availability technologies (Always On, Managed Instance) place on the backup chain.

2.2 Recovery Models

Recovery model	Log behavior	Point-in-time restore	Typical use
SIMPLE	Log is truncated automatically on checkpoint; no log backups possible.	No — only to the last full/differential.	Dev/test, ETL staging databases.
FULL	Log retains every operation until a log backup is taken.	Yes, to any point in time within the retention.	Production OLTP, anything needing Always On / log shipping / mirroring.
BULK_LOGGED	Minimally logs certain bulk operations (BULK INSERT, index rebuilds, SELECT INTO) to reduce log volume.	Limited — point-in-time restore is not guaranteed across a log backup containing a minimally logged operation.	Temporary switch during a known-large bulk load, then switch back to FULL.

LSNs and the backup chain

Every record in the transaction log carries a Log Sequence Number (LSN), structured as three hexadecimal parts: VLF number : offset within the VLF : slot number within the log block. A higher LSN always represents a later change. Full, differential, and log backups are all anchored to LSNs — a differential backup captures every extent changed since the last full backup (tracked via the Differential Changed Map, DCM, per data file), and a log backup captures every log record since the last log (or full) backup.

2.3 Backup Types & Special Cases

- Full backup — every used page in the database, plus enough of the active log to bring the backup to a consistent point. Issuing a full backup does NOT break the differential base or the log chain (this is a common misconception); a regular full backup is fully compatible with subsequent differential and log backups continuing normally.
- Differential backup — only extents changed since the last full backup. Identified internally via the DCM page per data file; SQL Server can tell which backup is the differential base from the backup history / DCM bitmap.
- Log (transaction log) backup — every log record since the previous log or full backup; required on FULL/BULK_LOGGED databases to control log growth and enable point-in-time restore.
- Copy-only backup — a full or log backup that does NOT affect the normal backup chain (does not reset the differential base, does not truncate the log in a way that breaks the chain). This is the only backup

type permitted on an Always On secondary replica or on Azure SQL Managed Instance dev/test copy scenarios.

- Tail-log backup — captured when a database is damaged or offline and you need to capture the very last, not-yet-backed-up log records before restoring; use `BACKUP LOG ... WITH NO_TRUNCATE`, or `WITH CONTINUE_AFTER_ERROR` if the database itself is damaged.

RULE TO MEMORIZE

You cannot restore a differential backup on top of a copy-only full backup — only the true (non-copy-only) full backup that actually served as the differential base, followed by its differentials and then logs, forms a valid restore chain. On an Always On secondary, only copy-only full and log backups are possible — never a true differential.

2.4 Backup Priority & Always On

Always On lets you set a Backup Preference (Prefer Secondary, Secondary Only, Primary, or Any Replica) plus a numeric backup priority per replica (0-100) used to break ties when multiple replicas are eligible. The point of backup preference is workload offloading — moving the I/O and CPU cost of backups off the primary, which is exactly why interviewers want to hear the word “offload” rather than just “reduce load.” Use `sys.fn_hadr_is_primary_replica('DBName')` to branch logic in a maintenance job: when it returns 1, run a full/differential; when it returns 0, run only copy-only full or log backups.

2.5 Restoring a Database That Is Part of an AG

1. Remove the database from the Availability Group (or restore to a different name and re-add later), since a database that is a member of an AG cannot be directly overwritten by `RESTORE` on the primary in place.
2. Restore the full backup chain (full -> differential if any -> logs) `WITH NORECOVERY` until the final log, then `WITH RECOVERY` (or in one statement for the final restore).
3. Re-join the database to the AG: on the primary it is already a member if you restored in place; for a new/rebuilt secondary, the database must be restored `WITH NORECOVERY` using the same backup chain (or via automatic seeding / direct seeding) before it can be joined to the AG as a secondary replica.
4. Validate synchronization health afterward via `sys.dm_hadr_database_replica_states`.

2.6 Interview Questions — Beginner

Q1. What is a tail-log backup, and when is it useful?

A: A backup of the log records not yet captured, taken when a database is offline/damaged and about to be restored. It guarantees no committed work is lost in the restore. Syntax: `BACKUP LOG dbname TO ... WITH NO_TRUNCATE` (healthy but offline) or `WITH CONTINUE_AFTER_ERROR` (damaged database).

Q2. How does SQL Server identify a differential backup's base?

A: Through the Differential Changed Map (DCM) page tracked per data file, which flags every extent modified since the last full backup; the backup history records which full backup is the differential base.

Q3. Can we perform a backup/restore operation on tempdb?

A: No. tempdb cannot be backed up or restored; it is recreated from model on every instance restart.

Q4. How can you take a full backup without interrupting the LSN chain (i.e. without breaking subsequent differential/log backups)?

A: A regular (non-copy-only) full backup never breaks the log or differential chain by itself — that is the default, expected behavior. The chain is only broken by actions like switching to SIMPLE recovery, detaching the database, or by deliberately taking a COPY_ONLY backup for an out-of-band copy.

Q5. What is the specific limitation on backups taken from an Always On secondary replica?

A: Only copy-only full backups and log backups are supported on a secondary; true (non-copy-only) full and differential backups are not possible there.

2.7 Interview Questions — Intermediate & Scenario-Based

Q6. What is the use of backup preference in Always On, and what keyword should you use when explaining it?

A: It controls which replica's backup job actually executes (Prefer Secondary / Secondary Only / Primary / Any Replica), and the priority value breaks ties among eligible replicas of equal preference. The key word interviewers listen for is offloading — moving backup I/O/CPU cost off the primary onto a secondary.

Q7. Is it possible to run full backups on the primary replica and log backups on the secondary in the same AG?

A: Yes — branch your maintenance script using `sys.fn_hadr_is_primary_replica('DBName')`; when it returns 1 run the full/differential job, when it returns 0 (secondary) run only the log-backup job.

Q8. We know differentials are not possible on a secondary. Can we restore copy-only full, then a differential, then logs?

A: No. You cannot restore a differential on top of a copy-only full backup. Only a true (non-copy-only) full backup can serve as the base for a differential restore chain; on a secondary you are limited to copy-only full plus log backups only.

Q9. Will `sp_change_users_login` report back orphaned users at the Windows-authentication level?

A: No — it only reports orphaned SQL logins. Use `sp_validatelogins` to find orphaned Windows logins/users.

Q10. If you restore a database from a source server to a target server, is there any database property that does not carry over?

A: Yes — the TRUSTWORTHY property is intentionally not preserved across a restore, for security reasons (it must be explicitly re-enabled on the target if genuinely required).

Q11. Your AG is configured for asynchronous commit with manual failover, and both servers restart at the same time (a graceful restart). Will the AG come back up healthy automatically?

A: After a graceful restart both the primary and secondary databases go through normal crash recovery and should come back online automatically; however, with manual failover mode the AG/databases may come up requiring you to manually resume data movement if they do not auto-resume, so always verify with `sys.dm_hadr_database_replica_states` rather than assuming.

Q12. A column holding sensitive data is protected with column-level encryption. Can you restore that database without exporting the encryption keys/certificates?

A: Yes, for the backup/restore mechanics themselves — TDE has the database tied to a certificate/key that must travel with it, but column-level encryption (via a Database Master Key / certificate inside the database) backs up and restores along with the database. The real risk is operational: if you do not also know the DMK password, or the encryption certificate is lost, the encrypted column data becomes unreadable even though the restore itself succeeded.

Q13. Can you create multiple passwords for a Database Master Key?

A: Yes — a DMK can be protected by multiple passwords (and optionally by the Service Master Key as well). This is different from the Service Master Key, which does not support multiple passwords.

2.8 Quick Revision Notes

- SIMPLE = no log backups possible. FULL = point-in-time restore. BULK_LOGGED = log volume relief for bulk ops, with restore caveats.
- Only copy-only full + log backups are allowed on an AG secondary and on MI dev-copy scenarios — never a true differential.
- Backup Preference offloads I/O from primary; remember the word “offload” in interviews.
- Tail-log backup before restoring a damaged/offline database: WITH NO_TRUNCATE or WITH CONTINUE_AFTER_ERROR.
- TRUSTWORTHY never survives a restore — re-enable explicitly if genuinely needed.
- tempdb cannot be backed up or restored, ever.

Chapter 3 · Locking, Blocking, Deadlocks & Concurrency

3.1 Concept Overview

Locking, blocking, and deadlocks are three different things that candidates routinely conflate — interviewers specifically probe for whether you can draw the line between them precisely. Locking is the mechanism; blocking is a normal (if sometimes excessive) consequence of locking; a deadlock is a specific unresolvable cycle of blocking that the engine detects and terminates automatically.

3.2 Locking vs. Blocking vs. Deadlocking

Term	Definition
Locking	The mechanism SQL Server uses to protect data consistency by granting sessions controlled access (shared, exclusive, update, intent, schema, etc.) to rows, pages, or tables.
Blocking	A normal consequence of locking: Session B must wait because Session A holds an incompatible lock on the resource Session B wants. Excessive or long-duration blocking is a performance problem, but the behavior itself is by design.
Deadlock	A circular wait: Session A waits on a resource held by Session B, while Session B waits on a resource held by Session A (directly or transitively). Neither can ever proceed. SQL Server's Lock Monitor detects the cycle and kills one session (the “deadlock victim”), automatically rolling back its transaction.

3.3 Lock Escalation

Lock escalation converts many fine-grained row/page locks into a single table lock once a threshold (roughly 5,000 locks on one statement, by default) is crossed, to reduce lock-management overhead. Escalation jumps directly from row or page granularity to table — there is no intermediate row-to-page escalation step. To prevent unwanted escalation:

- Break large batch DML into smaller chunks (e.g. DELETE TOP (1000) ... in a loop instead of one giant DELETE).
- Fix non-SARGable predicates (functions/computations wrapping a column on the left side of a WHERE clause prevent index seeks, forcing scans that touch — and lock — far more rows than necessary).
- Add or adjust indexes so the query can seek rather than scan, shrinking its lock footprint.

```
DECLARE @done bit = 0;
WHILE (@done = 0)
BEGIN
    DELETE TOP(1000) FROM LogMessages WHERE LogDate < '20020102';
    IF @@ROWCOUNT < 1000 SET @done = 1;
END;
```

3.4 Diagnosing Blocking

5. Identify the head of the blocking chain with `sys.dm_exec_requests` (`blocking_session_id`), `sp_who2`, or `sp_WhoIsActive`.
6. Inspect what the blocker is actually doing — `wait_type`, `wait_resource`, the executing statement (`sys.dm_exec_sql_text`), and how long it has held its transaction open.
7. Decide whether to wait it out, optimize/kill the blocker, or attack the root cause (missing index causing a long scan, an application leaving a transaction open, a runaway ad-hoc query).
8. For recurring blocking, capture an Extended Events session on `lock_acquired/blocked_process_report` rather than relying only on point-in-time snapshots.

PRODUCTION SCENARIO — Blocking on a readable secondary caused by reporting queries

Symptom: reporting queries against a readable Always On secondary intermittently block, even though the secondary is read-only.

Root cause: the redo thread applying changes from the primary needs to take locks; long-running reporting queries on the secondary can block the redo thread (or vice versa), and the secondary's default Snapshot Isolation can still see blocking on the redo thread itself in some versions/configurations.

Fix: keep reporting transactions short, ensure the secondary has adequate I/O/CPU headroom so redo keeps pace, and on SQL Server 2022+ consider contained AG and Query Store on secondary features to keep an eye on the specific query shapes causing the issue.

3.5 Trace Flags & Tooling for Deadlocks

- Trace Flag 1222 (and legacy 1204) writes deadlock graph details to the SQL Server error log whenever a deadlock occurs.
- `system_health` Extended Events session also captures deadlock graphs by default, with no trace flag required, and is always running.
- SSMS can render a deadlock graph visually from the XML captured by either mechanism.

3.6 Interview Questions — Beginner

Q1. What is the difference between locking, blocking, and a deadlock?

A: Locking is the consistency mechanism; blocking is a normal side effect when one session's lock conflicts with another's request; a deadlock is a circular, unresolvable blocking chain that the engine detects and breaks by killing one participant.

Q2. What trace flag captures deadlocks in the SQL Server error log?

A: Trace Flag 1222 (detailed XML-style deadlock graph); the older Trace Flag 1204 gives a less detailed format. The always-on `system_health` Extended Events session also captures deadlocks without needing any trace flag.

Q3. What is lock escalation?

A: The automatic conversion of many fine-grained row/page locks on one statement into a single table lock once an internal threshold is crossed, to reduce lock-management overhead — it escalates directly from row or page to table, never row to page.

3.7 Interview Questions — Intermediate & Advanced

Q4. How will you troubleshoot blocking in production?

A: Find the head blocker via `sys.dm_exec_requests.blocking_session_id` or `sp_WhoIsActive`, inspect its current statement and how long it has held the transaction, then either optimize the blocking query, add a

missing index removing an unnecessary scan, shorten the transaction boundaries, or — if it is a runaway/erroneous session — terminate it. For chronic issues, capture a `blocked_process_report` Extended Events session for trend analysis instead of one-off snapshots.

Q5. How do you prevent lock escalation on large batch operations?

A: Chunk the DML into small batches with an explicit loop (e.g. `DELETE TOP(1000)` in a `WHILE` loop), fix non-SARGable predicates so the optimizer can seek instead of scan, and add or refine indexes to shrink the row set actually touched by the statement.

Q6. How will you troubleshoot HADR_SYNC_COMMIT wait types?

A: `HADR_SYNC_COMMIT` measures the time between a transaction becoming ready to commit on the primary and every synchronous-commit secondary acknowledging that it has hardened (written to its own log) the corresponding LSN. High waits here point to slow I/O, high CPU, or network latency/bandwidth issues on the synchronous secondary, not on the primary itself.

Q7. In a failover-clustered instance (FCI), do you still need storage when configuring MSDTC?

A: Yes, if you configure MSDTC as its own clustered resource it needs its own disk and IP address; however, SQL Server can also leverage the local MSDTC on each node without a dedicated clustered MSDTC resource in many modern configurations.

Q8. Explain narrow plans vs. wide plans for an UPDATE against a clustered index column.

A: A narrow plan (per-row update) updates the clustered index and every non-clustered index for one row before moving to the next — simple, but can cause many random I/Os since non-clustered index order rarely matches clustered order. A wide plan (per-index update) instead sorts all the non-clustered index changes in memory first, then applies each index's changes in its own sorted order, trading some up-front sort cost for far less random I/O on large updates.

3.8 Quick Revision Notes

- Locking = mechanism. Blocking = normal wait. Deadlock = unresolvable cycle, auto-killed by the Lock Monitor.
- Escalation jumps row/page -> table directly, around the ~5,000-lock threshold per statement.
- TF 1222 + `system_health` XEvents = deadlock graphs without extra setup.
- `HADR_SYNC_COMMIT` high waits = slow I/O / CPU / network on the synchronous secondary.
- Chunk big DML; fix SARGability; index to seek, not scan — the three levers against escalation and blocking alike.

Part II — High Availability & Disaster Recovery

Always On Availability Groups are the single most heavily interviewed topic for a Senior/Lead SQL Server DBA role, and for good reason — they sit at the intersection of clustering, networking, security, backup strategy, and patching discipline. This part builds the full HA/DR stack: traditional Always On AG, Distributed Availability Groups (DAG) and their cloud/migration use cases, the Windows clustering and quorum mechanics underneath both, and the legacy log shipping / mirroring / replication technologies you will still be asked to compare against AG.

Chapter 4 · Always On Availability Groups (AG)

4.1 Concept Overview

An Availability Group protects a set of user databases as a single failover unit. One replica is PRIMARY (read/write); up to eight secondaries (five synchronous) replicate transaction log records to keep their copy of each database current. Unlike Failover Clustered Instances (FCI), AG works at the database level, needs no shared storage, and gives you a built-in offload path for backups and read-only reporting. Underneath a traditional AG sits a single Windows Server Failover Cluster (WSFC) and a single Active Directory domain — this single-cluster dependency is exactly what Distributed AG (Chapter 5) removes.

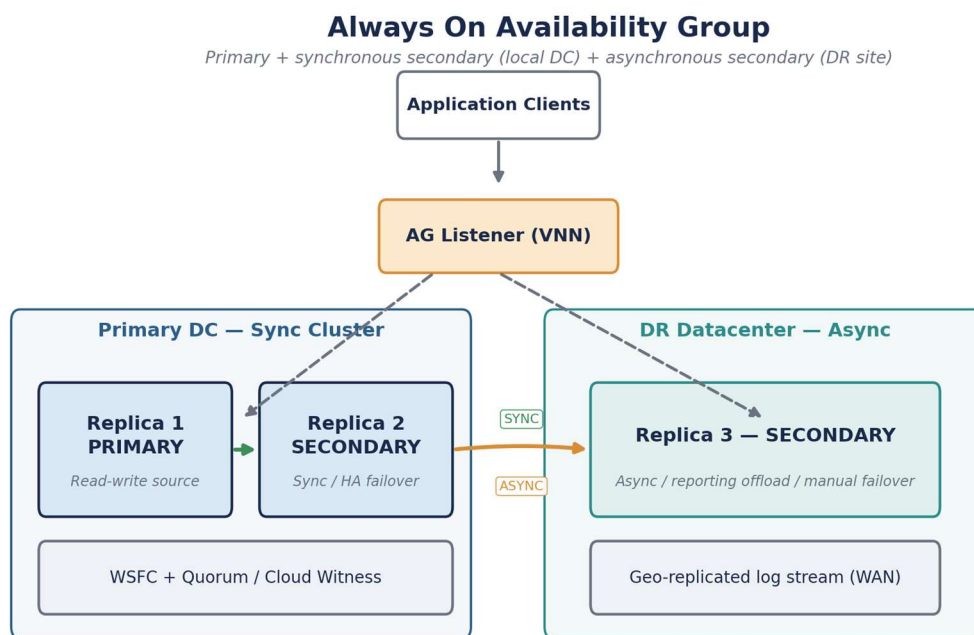


Figure 4.1 — Always On AG multi-subnet architecture: synchronous secondary in the primary datacenter, asynchronous secondary at the DR site, all behind one Listener.

4.2 Quick Facts

Setting	Value / Note
Max secondary replicas	8 total (Enterprise), of which up to 5 may be synchronous-commit
Default AG connection_timeout	10 seconds
Default isolation level on secondary	SNAPSHOT (forced, regardless of primary's isolation level)
Basic AG (Standard edition)	2 replicas, 1 database per AG, no read-only routing, no multiple listeners

Contained AG (2022+)

Replicates logins, permissions, SQL Agent jobs, and other instance-scoped metadata automatically — closes the historic “sync logins manually” gap

4.3 Prerequisites & Setup

- Windows Server Failover Cluster (WSFC) already built and healthy across all nodes that will host a replica, with a working quorum (see Chapter 6).
- All instances at the same edition tier needed for the AG feature set (Basic AG on Standard edition has reduced capability versus Enterprise).
- Always On Availability Groups feature enabled via SQL Server Configuration Manager on every instance (service restart required).
- Matching or compatible database compatibility level/collation is recommended; SQL Server build/edition can differ across replicas to support rolling upgrades.
- A domain account (or gMSA) with rights to create the Availability Group Listener's computer object in Active Directory.
- Full recovery model on every database that will join the AG, plus at least one full backup already taken (AG seeding needs a starting point unless you use automatic seeding).

PRE-STAGING — CNO and VCO permissions for AG creation

Interviewers often expect “prestaging” as the keyword, not just “sysadmin/Administrator rights.”

CNO (Cluster Name Object) — the Windows cluster's own computer account in AD. To have it created automatically, the account building the failover cluster needs Create Computer Objects permission on the OU/container where the cluster's member servers live.

VCO (Virtual Computer Object) — the AG Listener's computer account. The account needs List Contents, Read All Properties, Read Permissions, and Create Computer Objects on that OU — or the CNO must be pre-staged with Full Control over a disabled computer object matching the Listener name, if the AD team will not grant broad OU rights to the DBA team.

4.4 Listener, SPN, DNS & File Share

AG Listener and SPN

The Listener is a Virtual Network Name (VNN) plus one or more Virtual IPs that lets applications connect to “whichever node is primary right now” without knowing node names. A Service Principal Name (SPN) is registered in AD against the Listener's account so that clients can authenticate to it using Kerberos rather than falling back to NTLM — missing/duplicate SPNs are one of the most common AG connectivity defects found in production.

- Creating multiple Listeners on the same AG: not natively supported by the AG Listener object itself, but the common workaround is to add additional IP addresses/ports to the existing Listener for multi-subnet scenarios, or to use a Distributed Network Name (DNN, 2019+) which removes the VIP/Listener entirely and relies on the cluster's own DNS-based name resolution — each replica's own IP doubles as the “listener” endpoint, which is why DNN setups show no dedicated Listener IP in `sys.availability_group_listener_ip_addresses`.
- File Share Witness — a simple SMB share on a separate server used as a quorum vote when you don't have (or don't want) a third full node or disk witness; common in two-node AG/FCI clusters and now largely superseded by Cloud Witness (an Azure Storage account blob) for geographically dispersed clusters.

- DNS issues in AG typically show up as: stale DNS cache on the client after failover (fix: lower the Listener's DNS TTL, or use RegisterAllProvidersIP=1 with MultiSubnetFailover=True in the connection string so the client tries all registered IPs in parallel rather than waiting on DNS/TTL); duplicate or orphaned DNS A-records for a renamed/rebuilt Listener; and missing reverse-lookup zones breaking Kerberos SPN validation.

4.5 Operational Sync Tasks (Logins, Users, Files)

Task	Guidance
Sync logins	Create the SQL/Windows login on the primary; if a Windows login, it is visible everywhere automatically via AD. For SQL logins, recreate on every secondary and reconcile the SID with ALTER LOGIN ... WITH PASSWORD=... or sp_help_revlogin so the SID matches the orphaned user once databases fail over. From SQL Server 2022, Contained Availability Groups remove this manual step almost entirely.
Sync users (secondary)	Database users replicate automatically as part of the database itself (they live inside the user database). What does NOT replicate automatically is the server-level login the user maps to on a non-contained AG — that is the actual gap people mean when they say “sync users.”
Add database files	ALTER DATABASE ... ADD FILE on the primary; the file is created automatically on every secondary in the same physical path. If paths differ across replicas, the secondary will fail to apply the file-add operation until the directory structure matches — a frequent “secondary not healthy after maintenance” root cause.
Permissions / users on both nodes?	Creating the login/user and granting permission only on the primary is sufficient for a non-contained AG too, since the database-level user and its permissions travel with the database; only the server login itself needs the manual sync step above.

4.6 Backups, Backup Priority & Restore

- Backup priority is configured per replica (0-100) in the AG properties; SQL Server picks the eligible replica with the highest priority (ties broken alphabetically) when a maintenance plan or backup job uses BACKUP DATABASE ... and checks sys.fn_hadr_backup_is_preferred_replica.
- Why prefer backups on a secondary at all? The headline reason is offloading I/O from the primary, but the sharper interview answer is that it reduces the primary's resource contention during peak transactional load — backups become a non-event on the primary's storage/CPU budget.
- Only COPY_ONLY full backups are supported on a secondary replica — you cannot take a differential backup on a secondary, and a differential backup cannot be restored on top of a copy-only full backup; you must use a normal (non copy-only) full backup as the differential base.

- Programmatically decide where to run a backup job with `sys.fn_hadr_is_primary_replica('DBName')` — returns 1 on the primary (run full/normal backups there) and 0 on secondaries (run log/copy-only backups there).
- Log shipping secondaries and mirrored databases cannot be backed up at all — this is the classic comparison point examiners want you to volunteer when discussing AG's backup-offload advantage.

4.6.1 Restoring a Database That Is Part of an AG

9. Remove the database from the Availability Group (it must leave the AG before it can be restored over).
10. Restore the database on the (former) primary instance as you would any standalone database (full + diff + log chain, or STANDBY/NORECOVERY chain as required).
11. Re-add the database to the AG and let it re-seed (automatic seeding) or restore the same backup chain WITH NORECOVERY on each secondary before joining it back, matching LSNs exactly.

Gotcha — the TRUSTWORTHY property does not travel with a restore

Every other database property is preserved end-to-end on RESTORE, but TRUSTWORTHY is deliberately reset to OFF on the target for security reasons and must be re-enabled explicitly if the database relies on it (e.g. CLR modules signed by a certificate in master, or cross-database ownership chaining).

4.7 Patching & In-Place Upgrades Inside an AG

Patch the least-impactful replica first, validate, fail over, then patch the node that used to be primary — never patch the active primary first.

PRODUCTION RUNBOOK — Patching a multi-DC AG (2 sync nodes + 1 async DR node)

1. On the secondary you are about to patch, change AG failover mode from Automatic to Manual so the patch/restart can't trigger an unplanned failover.
2. Suspend data movement for that secondary's databases (run the SUSPEND from the secondary you are patching — suspending from the primary pauses data movement to ALL secondaries, which is rarely what you want).
3. Apply the CU/patch and restart the secondary.
4. Resume data movement for that secondary's databases and wait for it to report SYNCHRONIZED/HEALTHY.
5. Perform a manual, planned failover once the patched secondary is healthy.
6. Patch the former primary (now a secondary) using the same suspend/patch/resume sequence.
7. Fail back to the original primary if required by change policy.
8. For the asynchronous DR replica: pause data movement → patch → validate SQL/DB health → resume data movement. Async replicas tolerate a longer pause window since they are not on the RPO-critical sync path.

- An in-place upgrade (e.g. 2019 → 2022) inside an AG follows the same secondary-first, one-node-at-a-time discipline — upgrade every secondary, fail over, then upgrade the former primary. AG supports replicas at different builds/editions specifically to make this rolling pattern possible.
- Plan for N-2 patch levels rather than always chasing the very latest CU on day one, and pilot every CU in a lower environment for at least a week — several recent CUs on SQL Server 2019/2022 broke patching specifically when SSISDB was part of an AG, which is exactly the kind of regression a staged rollout catches before production.

4.8 Encryption (TDE) Inside an AG

- TDE is enabled per-database, but the certificate/key protecting the Database Encryption Key (DEK) must be created on every replica's master database — export the certificate (and private key) from the primary, then CREATE CERTIFICATE ... FROM FILE on each secondary before the encrypted database can join or replicate into that secondary.
- Adding an already-TDE-enabled database to an AG follows the normal add-database wizard, but seeding will fail silently or report an error until the matching certificate exists on the target secondary — always pre-stage the certificate first.
- Column-level / Always Encrypted is a different mechanism from TDE: it has no certificate-export dependency for AG membership, so a database using only column-level encryption can be backed up/restored or AG-seeded freely; the only failure mode is application connectivity to decrypt the column if the client doesn't hold the right key/Key Vault access.

4.9 Monitoring & Health

DMV / Tool	Use
sys.dm_hadr_availability_replica_states	Replica role, connected/synchronization state, operational state
sys.dm_hadr_database_replica_states	Per-database sync state, log_send_queue_size, redo_queue_size — the two columns to watch for latency
sys.dm_hadr_availability_group_states	Overall AG health (HEALTHY / PARTIALLY_HEALTHY / NOT_HEALTHY) and primary_recovery_health
sys.dm_hadr_cluster_*	Underlying WSFC node/quorum visibility from inside SQL Server
sp_server_diagnostics	Lightweight rolling health snapshot used internally by the Resource Health subsystem
AlwaysOn_health Extended Events session	Always running by default; captures failover, lease-timeout, and diagnostic events without extra setup

Command-line latency check (no GUI required):

```
SELECT ag.name AS ag_name, ar.replica_server_name,
       drs.synchronization_state_desc, drs.log_send_queue_size,
       drs.redo_queue_size, drs.last_commit_time
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas ar ON drs.replica_id = ar.replica_id
JOIN sys.availability_groups ag ON ar.group_id = ag.group_id;
```

- Health check at a glance: query sys.dm_hadr_availability_group_states.synchronization_health and primary_recovery_health; a green AG dashboard in SSMS is reading the same DMVs.
- AG Failover history is recorded in the AlwaysOn_health Extended Events session — filter for event id 1480 (availability group failover events) to build a timeline without trawling raw error logs.
- Default statistics created for queries run against a read-only secondary live in tempdb (the secondary's own database is read-only, so the engine cannot persist new statistics objects into it).

4.10 Troubleshooting Playbook

“I can connect to primary but not to the secondary (read-only)”

12. Confirm the application's connection string includes `ApplicationIntent=ReadOnly` — without it, the Listener will always route to the primary regardless of read-only routing configuration.
13. Verify read-only routing lists are configured on every replica via `ALTER AVAILABILITY GROUP ... MODIFY REPLICA ... WITH (READ_ONLY_ROUTING_URL=...)` and that `READ_ONLY_ROUTING_LIST` is populated on the current primary (routing is driven from whichever replica is primary at the time).
14. Check the secondary's own database-level read-only-routing connection URL and that the secondary's role allows connections (`ALLOW_CONNECTIONS = READ_ONLY` or `ALL`).
15. Confirm the secondary database itself is in a synchronized/healthy, online, readable state — a database that is still recovering or has fallen out of sync will simply not accept new connections.

“Secondary is not in sync”

16. Check `redo_queue_size` and `log_send_queue_size` in `sys.dm_hadr_database_replica_states` to see whether the bottleneck is network send (log queue) or apply/redolog on the secondary (redo queue).
17. If redo is the bottleneck, look for I/O latency on the secondary's storage, an undersized secondary relative to the primary, or a single-threaded redo limitation on pre-2016 builds (parallel redo improved significantly from SQL Server 2016 onward).
18. If the log-send queue is the bottleneck, check network bandwidth/latency between sites, and confirm the secondary isn't suspended (`RESUME` the database if `SUSPENDED`).
19. Examine the cluster log, SQL Server error log, `AlwaysOn_health` and `system_health` Extended Events sessions, and the SQL DIAG XEvent files together — AG sync issues frequently have a root cause visible in only one of these five sources.

PRODUCTION SCENARIO — Reporting queries blocking on a readable secondary

This is the redo-thread contention pattern detailed in Chapter 3.4 — long-running reporting transactions on the secondary contend with the REDO thread applying changes from the primary, even though the secondary is technically read-only.

Mitigation specific to AG: shorten reporting transactions, ensure the secondary has I/O/CPU headroom to keep redo caught up, and consider routing the heaviest reports to a dedicated, lower-priority secondary rather than the one closest to the primary in the routing list.

Other recurring causes of AG connection timeouts in the field: VM snapshot operations on the host (freeze I/O for seconds at a time), antivirus real-time scanning on data/log volumes, and blocking on the secondary caused by the redo thread itself when reporting transactions overlap with heavy incoming change volume.

If both replicas of an asynchronous-commit, manual-failover AG are restarted gracefully, both primary and secondary databases simply go through normal crash recovery and come back online; if a database does not automatically resume on the secondary afterward, manually `RESUME` it.

4.11 Interview Questions — Beginner

Q1. What are the prerequisites for setting up Always On?

A: A healthy WSFC across the candidate nodes, the Always On feature enabled on every instance (with a service restart), full recovery model on every database to be protected, an account with rights to pre-stage the Listener's computer object in AD, and at least one full backup of each database for seeding.

Q2. What is the default connection_timeout for an Availability Group?

A: 10 seconds.

Q3. What is the default isolation level on an AG secondary database?

A: Snapshot isolation — enforced regardless of the isolation level configured/requested on the primary, because the secondary database is read-only.

Q4. Do you need to create logins on every node of an AG, or just the primary?

A: Create on the primary; Windows logins are visible everywhere via AD automatically, and database users travel with the database itself. The one manual step on a non-contained AG is making sure a SQL login's SID matches on every secondary so the database user maps correctly after failover — Contained AG (2022+) removes this step.

4.12 Interview Questions — Intermediate

Q5. How do you check Always On latency without using the GUI?

A: Query `sys.dm_hadr_database_replica_states` joined to `sys.availability_replicas/sys.availability_groups`, watching `log_send_queue_size` and `redo_queue_size`; both should trend toward zero on a healthy, caught-up secondary.

Q6. How do you add database files to a database that is part of an AG?

A: Run `ALTER DATABASE ... ADD FILE` on the primary; the engine creates the matching file automatically on every secondary as long as the directory path exists and matches across all replicas — mismatched paths are the most common reason a secondary goes unhealthy right after a file-add maintenance window.

Q7. What DMVs do you rely on to troubleshoot AG health?

A: `sys.dm_hadr_availability_replica_states` and `sys.dm_hadr_availability_group_states` for replica/AG-level health, `sys.dm_hadr_database_replica_states` for per-database sync queues, and `sys.dm_hadr_cluster_members` / `sys.dm_hadr_cluster` for the underlying WSFC view from inside SQL Server.

Q8. How do you set backup priority in an Availability Group?

A: Configure each replica's Backup Priority (0-100) in the AG's Backup Preferences; SQL Server's backup job logic checks `sys.fn_hadr_backup_is_preferred_replica` to decide whether the current node should run the scheduled backup.

Q9. How do you restore a database that is currently part of an AG?

A: Remove the database from the AG first, restore it normally on the primary, then re-add it to the AG (automatic seeding, or a manual `WITH NORECOVERY` restore chain on each secondary) so every replica's LSN chain matches again.

Q10. How do you apply TDE to databases already in (or being added to) an AG?

A: Create the TDE certificate on the primary, back up the certificate and private key, then `CREATE CERTIFICATE ... FROM FILE` using that backup on every secondary's master database before the encrypted database can seed or join successfully.

Q11. What is a Distributed Network Name (DNN) and why doesn't it show a Listener IP?

A: A DNN (2019+) resolves clients to whichever replica is currently primary purely through DNS, without allocating a dedicated Virtual IP/Listener object — each replica's own IP doubles as the connection endpoint, which is why `sys.availability_group_listener_ip_addresses` shows nothing for a DNN-based AG.

4.13 Interview Questions — Advanced & Scenario-Based

Q12. Walk through patching a 3-node AG that spans two datacenters (2 sync nodes + 1 async DR node).

A: Set the secondary you are about to patch to Manual failover, suspend data movement from that secondary, patch and restart it, resume data movement, wait for SYNCHRONIZED, then run a planned manual failover so it becomes primary.

Patch the former primary using the identical suspend → patch → resume sequence, then fail back if your change window requires the original topology.

For the async DR replica, pause data movement, patch, validate, then resume — async replicas can absorb a longer pause since they are off the RPO-critical synchronous path.

Q13. A user reports they can reach the primary but not a read-only secondary. What do you check, in order?

A: ApplicationIntent=ReadOnly present in the connection string (without it, routing always goes to primary).

READ_ONLY_ROUTING_LIST configured on the current primary, and each replica's own READ_ONLY_ROUTING_URL.

The secondary database's ALLOW_CONNECTIONS setting and whether the database is actually online/synchronized, not still recovering.

Q14. How would you resolve recurring blocking on a readable secondary caused by a reporting application?

A: Diagnose whether the redo thread or the reporting sessions are the blocker (sys.dm_exec_requests on the secondary, watching for the redo thread's session and its wait type); shorten reporting transactions, ensure the secondary has adequate I/O/CPU headroom for redo to keep pace, and consider isolating heavy ad-hoc reporting to its own lower-priority secondary in the routing list rather than the closest one to the primary.

Q15. Both replicas of an async-commit, manual-failover AG were restarted at the same time. Will the AG come back healthy automatically?

A: If the restart was graceful, both primary and secondary databases go through normal crash recovery and the AG should come back online by itself; if a secondary database does not auto-resume afterward, it needs an explicit RESUME. An ungraceful/simultaneous loss that also breaks cluster quorum is a different (and more serious) scenario — see Chapter 6's force-quorum runbook.

Q16. What permissions does the account creating the AG Listener actually need in Active Directory, and why does this trip people up in interviews?

A: It's not sysadmin/Administrator rights on the SQL/Windows side that matter here — it's AD prestaging: Create Computer Objects on the OU for the Cluster Name Object (CNO), and List Contents / Read All Properties / Read Permissions / Create Computer Objects for the Virtual Computer Object (VCO) backing the Listener. Candidates who answer only with sysadmin/Administrator usually get redirected toward this exact CNO/VCO terminology.

Q17. A database with column-level encrypted data needs to be restored on a different server. Do you need to export private keys first?

A: No — that requirement is specific to TDE's certificate/DEK chain. Column-level encryption has no such dependency for backup/restore; the database restores normally, and the only failure mode is an application failing to decrypt the column if it lacks the right symmetric key or Key Vault/Always Encrypted access on the new server.

4.14 Quick Revision Notes

- AG = database-level HA/DR, no shared storage, up to 8 secondaries (5 sync); needs one WSFC + one AD domain underneath (traditional AG).
- Listener = VNN + VIP; DNN = no VIP, pure DNS-based redirect to whichever node is primary.
- Secondary backups: COPY_ONLY full or log only — never differential, never a normal full.
- Patch order: secondary → manual failover mode → suspend → patch → resume → failover → patch old primary → failback.
- TDE needs the certificate exported to every secondary before the encrypted DB can seed; column-level encryption has no such dependency.
- TRUSTWORTHY never survives a RESTORE — re-enable manually if required.
- Health DMVs: dm_hadr_availability_replica_states, dm_hadr_availability_group_states, dm_hadr_database_replica_states (watch log_send_queue_size / redo_queue_size).

Chapter 5 · Distributed Availability Groups (DAG) & Modern HA Patterns

5.1 Concept Overview

A Distributed Availability Group (DAG, SQL Server 2016+) is an AG of AGs: it links two independent Availability Groups — each with its own, separate WSFC — into a single replication topology. The AG-1 primary streams its single log feed only to the AG-2 primary, called the Forwarder, which then redistributes it to its own secondaries. This removes the single-cluster, single-AD-domain constraint that traditional AG carries, and is the technology underneath several Azure migration features (Managed Instance Link, Azure Managed Instance Link for SQL Server 2019/2022).

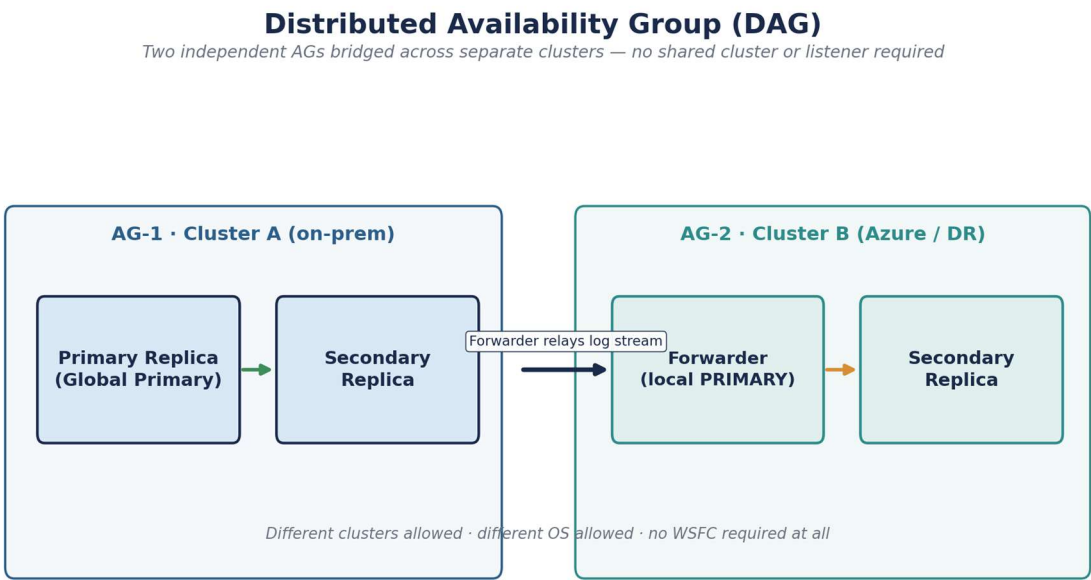


Figure 5.1 — Distributed AG: two independent WSFCs, connected only through the AG-2 Forwarder.

5.2 DAG vs. Traditional AG

Aspect	Traditional AG	Distributed AG
Underlying cluster	Single WSFC shared by all replicas	Two independent WSFCs (one per AG)
Operating system	Should match across replicas	Can differ between the two clusters
Listener required	Yes (or DNN)	No listener/WSFC needed at all on a cluster-less DAG endpoint
Log distribution	Primary sends directly to every secondary	Primary sends once to the Forwarder, which redistributes locally

Typical use case	Local/regional HA + one DR replica	Cross-cluster DR, cross-version migration, on-prem ↔ Azure MI
-------------------------	------------------------------------	---

5.3 Forwarder, Seeding & Failback

- The Forwarder is AG-2's primary replica; it is read-only from the DAG's perspective even though it is the primary of its own local AG — applications can read from it, but writes must go to AG-1's primary.
- Backups on the Global Forwarder are limited to COPY_ONLY full and log backups — the same restriction as any other secondary-style role, even though the Forwarder reports itself as a primary within its own cluster.
- Automatic seeding is the default and recommended initialization method for DAG — it streams the data directly over the connection rather than requiring a manual backup/restore, provided the target SQL Server build matches the source. If you are intentionally upgrading versions during the move, automatic seeding cannot be used and you must seed manually via backup/restore.
- DAG cannot currently be layered on top of an AG that itself uses a Distributed Network Name (DNN) listener — this is a known platform limitation to flag if asked.

5.4 Primary Use Cases

Near-zero-downtime migration (e.g. SQL Server 2017 → 2019)

From SQL Server 2017 onward, DAG can be configured even between two standalone instances — no WSFC and no AG Listener required on either side. Because DAG streams log records continuously rather than relying on a one-time backup/restore, the cutover is an instantaneous flip: build the 2019 target, let DAG catch it up to near-zero lag, then fail over — turning what used to be hours of backup/restore/log-apply on a multi-terabyte database into under a minute of actual downtime.

Azure SQL Managed Instance Link

The Managed Instance Link feature establishes a DAG behind the scenes between an on-premises SQL Server (2016+ with the latest CUs, or 2019/2022) and an Azure SQL Managed Instance. Critically, the on-premises side does not need Always On configured at all — even a standalone instance qualifies. This lets you stand up MI as a live, readable replica well before cutover (a reporting/DR target during migration) and then perform the actual cutover as a DAG failover rather than a backup/restore migration.

5.5 Interview Questions

Q18. What are the advantages of DAG over a traditional AG?

A: No shared WSFC and no requirement for matching operating systems across the two clusters; DAG even works on cluster-less standalone instances on SQL Server 2017+; and the Forwarder model means the primary only ever sends its log stream once across the DAG boundary, rather than to every remote secondary individually.

Q19. Can backups be taken on the Global Forwarder?

A: Yes, but only COPY_ONLY full backups and log backups — never a normal full or a differential — exactly the same restriction that applies to any other non-primary-in-its-own-AG role.

Q20. Does Managed Instance Link require Always On to already be configured on-premises?

A: No. It builds a Distributed Availability Group on the fly between the on-premises instance (even a standalone one) and the target Managed Instance, which is also how you can stage MI as a readable reporting target ahead of the actual cutover.

Q21. Can you configure automatic seeding for a DAG?

A: Yes, and it is the default/preferred method, but the target SQL Server build should match the source for automatic seeding to work; if you deliberately upgrade the engine version as part of the move, you must seed manually via backup/restore instead.

Q22. How would you migrate a 4 TB standalone database from SQL Server 2017 to 2019 with under a minute of downtime?

A: Configure a Distributed Availability Group directly between the two standalone instances (no WSFC or Listener required from 2017 onward), let automatic seeding/log streaming catch the target up to near-zero lag, validate, then perform the DAG failover — the actual downtime is just the instant of the cutover flip, not a backup/restore cycle against a 4 TB database.

Q23. Can a DAG be layered on top of an Always On AG that uses a Distributed Network Name (DNN) listener?

A: No — this combination is not currently supported.

Q24. Is DAG itself a single point of failure if the Forwarder goes down?

A: The Forwarder is just AG-2's primary; if it fails, AG-2's own automatic/manual failover promotes one of its local secondaries as usual, and that new primary continues to be visible to AG-1 as the Forwarder once it resumes the link — DAG availability still rides on each side's own local AG failover, not on a single dedicated relay node.

5.6 SQL Server 2025/2026 — HA & Availability Group Enhancements

SQL Server 2025 adds a meaningful set of refinements directly relevant to AG operations interviewers increasingly probe for in 2026:

- Optimized Locking — a new database-scoped option that avoids classic row/page lock escalation paths entirely for many workloads (no application code changes required), directly reducing one of the most common blocking root causes covered in Chapter 3.
- TempDB resource governance — the ability to “sandbox” a session/user's TempDB consumption, including sort spills, to contain noisy-neighbor TempDB contention (see Chapter 10).
- Query Store support on readable secondaries — finally lets you capture and analyze the actual query plans/regressions happening against your reporting secondaries, not just the primary.
- Resiliency and diagnostic improvements specifically called out for Always On Failover Groups (the Azure SQL DB/MI geo-replication construct, distinct from on-prem AG) — faster, more observable failovers.
- Fabric mirroring (change-feed based) gives AG-protected databases a near-real-time, zero-ETL analytics offload path directly into Microsoft Fabric OneLake — see Chapter 18.

5.7 Quick Revision Notes

- DAG = AG of AGs, linked via a single Forwarder; independent WSFCs; OS can differ; cluster-less on 2017+.
- Forwarder backups: copy-only full + log only, same as any secondary.
- MI Link = DAG under the hood; works even from a standalone on-prem source.
- Automatic seeding requires matching source/target build; version upgrades during migration force manual seeding.

- DAG + DNN-based AG underneath = unsupported combination.
- 2025/2026: Optimized Locking, TempDB sandboxing, and Query Store on secondaries are the headline AG-adjacent engine upgrades to mention.

Chapter 6 · Failover Clustering, Quorum & Windows Server Failover Clustering

6.1 Concept Overview

Both classic SQL Server Failover Clustered Instances (FCI) and traditional Availability Groups run on top of a Windows Server Failover Cluster (WSFC). The cluster's job is to keep exactly one set of nodes “in charge” of a resource at any moment — quorum is the voting mechanism that prevents two halves of a partitioned cluster from both believing they own the same disk or the same AG primary role at once (split-brain).

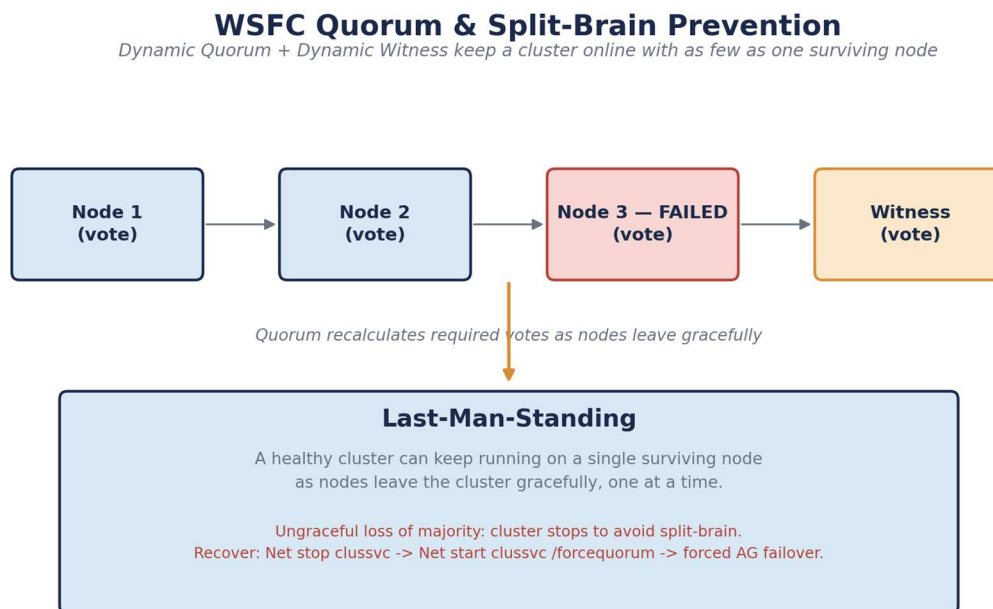


Figure 6.1 — Quorum voting: the majority side stays online; the minority side is evicted from cluster membership.

6.2 Quorum Models & Last-Man-Standing

- The quorum drive (legacy Node and Disk Majority model) was historically a small shared LUN used purely to hold the cluster's quorum vote/log — modern Windows Server clusters more commonly use Node Majority, Node and File Share Majority, or Cloud Witness instead of a dedicated quorum disk.
- Dynamic Quorum (Windows Server 2012+) recalculates the number of votes needed for quorum as nodes leave the cluster gracefully, so the cluster keeps running even after losing more than half its original membership — this is what enables Last-Man-Standing, where a cluster can ultimately keep running on a single surviving node.
- Dynamic Witness (2012 R2+) similarly lets the cluster add/remove the witness vote dynamically to avoid even-vote ties.
- Both mechanisms only work for graceful node departures. An ungraceful, simultaneous loss of a majority of votes (e.g. two of three nodes disappearing at once) makes the surviving minority side shut itself down to avoid split-brain, even if it would otherwise be the “healthy” side.

6.3 Force Quorum & Disaster Recovery Runbook

PRODUCTION SCENARIO — 3-node Node Majority cluster, 2 nodes lost simultaneously (ungraceful)

1. Log in to the one surviving node and stop the cluster service: `net stop clussvc`
2. Restart the cluster service forcing quorum on that single node: `net start clussvc /forcequorum`
3. Fail the Availability Group over to that node, accepting potential data loss since the other replicas are unreachable: `ALTER AVAILABILITY GROUP AGTest FORCE_FAILOVER_ALLOW_DATA_LOSS;`
4. Once the other nodes are back online and rejoin the cluster, resume data movement on each formerly-down replica: `ALTER DATABASE [DBName] SET HADR RESUME;`
5. Be aware that log truncation on the primary is delayed for as long as any secondary remains suspended — if the outage drags on, consider removing the failed replica from the AG entirely rather than risking a full transaction-log drive.

6.4 AD Prestaging for Clustering & AG (cross-reference)

See Section 4.3 for the full CNO/VCO breakdown — building the WSFC itself needs Create Computer Objects rights for the Cluster Name Object (CNO), and the AG Listener's Virtual Computer Object (VCO) needs List Contents / Read All Properties / Read Permissions / Create Computer Objects on the same OU.

6.5 Active-Active vs. Active-Passive & Client Connectivity

- Active-passive cluster — one node actively serves the workload while the other sits idle, ready to take over on failover; this is how a single-instance SQL Server FCI normally runs.
- Active-active cluster — each node actively hosts its own separate SQL Server instance simultaneously, and each instance is the other's failover target; common on 2-node clusters hosting two differently-named instances to make full use of both nodes' hardware day-to-day.
- Clients always connect to the FCI's Virtual Network Name / IP (or the AG Listener), never to a physical node name directly — that virtual identity is what migrates silently between nodes on failover, with no connection-string change required on the application side.

6.6 Interview Questions

Q25. What is the quorum drive and what is it used for?

A: Historically, a small shared disk resource dedicated to holding the cluster's quorum log/vote under the Node and Disk Majority model; modern Windows Server clustering more commonly uses Node Majority, Node and File Share Majority, or a Cloud Witness instead of a dedicated physical quorum disk.

Q26. What is the concept of 'last man standing' in a SQL Server cluster (FCI or AG)?

A: Dynamic Quorum recalculates how many votes are needed for quorum as nodes leave the cluster gracefully, allowing the cluster to keep running even after it has lost a majority of its original nodes — down to, and including, a single surviving node.

Q27. What is the difference between an active-active and an active-passive cluster?

A: Active-passive has one node carrying the workload while its partner sits idle as failover target; active-active has each node running its own separate instance simultaneously, each acting as the other's failover target.

Q28. How do clients connect to SQL Server running in a cluster?

A: Always through the cluster's virtual name/IP (the FCI network name, or the AG Listener) — never a physical node name — so failover is transparent to the application's connection string.

Q29. Three nodes are running Node Majority quorum and two nodes go down at the same instant. What happens to your Always On databases, and how do you recover?

A: An ungraceful simultaneous loss of a majority of votes makes the surviving single node shut its own cluster service down to prevent split-brain, even though it is the only node left.

Recovery: stop and restart the cluster service with /forcequorum on the surviving node, run ALTER AVAILABILITY GROUP ... FORCE_FAILOVER_ALLOW_DATA_LOSS to bring the AG online accepting possible data loss, then RESUME data movement on each replica once the other nodes rejoin.

Watch transaction log growth on the primary throughout — log truncation is held up while any replica remains suspended.

Q30. Why does dynamic quorum/witness not save a cluster from an ungraceful loss of majority?

A: Both mechanisms only recompute vote requirements in response to graceful, sequential node departures recognized by the cluster service; a sudden, simultaneous loss of multiple nodes looks identical to a network partition from the surviving side's perspective, so the cluster defensively shuts down rather than risk two sides independently believing they hold quorum.

6.7 Quick Revision Notes

- Quorum prevents split-brain; majority side stays up, minority side is evicted from membership.
- Dynamic Quorum + Dynamic Witness = Last-Man-Standing, but only across graceful node losses.
- Ungraceful majority loss → manual /forcequorum + FORCE_FAILOVER_ALLOW_DATA_LOSS + per-database RESUME once nodes return.
- CNO = cluster's own AD computer object; VCO = Listener's AD computer object — both need explicit OU permissions, not just sysadmin rights.
- Clients always target the virtual name/IP, never a physical node.

Chapter 7 · Log Shipping, Database Mirroring & Replication

7.1 Why Always On Displaced the Older HA/DR Technologies

Technology	Strength	Weakness vs. Always On AG for reporting
Log Shipping	Simple, cheap, works across editions, configurable delay as a safety buffer against corruption	Restores apply on a schedule (e.g. every 15 min) and kick out connected readers each time a log backup is restored — unsuitable for long-running reporting queries
Database Mirroring (deprecated)	Near-real-time, simple two-server topology, witness-based automatic failover in High Safety mode	Read-only access requires manually scheduling database snapshots against the mirror — operationally clunky, and mirroring itself has been deprecated since SQL Server 2012
Transactional Replication	Granular (table/row/column-level filtering), can fan out to many subscribers, cross-platform publishers possible	Only replicates objects explicitly published — and not every table is a good replication candidate (tables without a primary key cannot be replicated by standard transactional replication)
Always On AG	Whole-database failover unit, multiple readable secondaries, native backup offload, no read-only snapshot scheduling needed	Needs a WSFC (traditional AG) and Enterprise edition for full capability

Microsoft's own positioning is that Always On AG was the first dedicated, purpose-built reporting-offload HA solution it shipped — the comparison table above is exactly the argument an interviewer is listening for when they ask “why is AG preferred for reporting?”

7.2 Log Shipping

- Log shipping can be configured on any database in the FULL or BULK_LOGGED recovery model — bulk-logged is supported (unlike mirroring), although minimally-logged bulk operations under bulk-logged recovery do reduce your point-in-time recoverability within that log backup.
- Recovering the secondary after a primary failure: stop the copy/restore jobs, restore the final, already-shipped log backups (and the tail-log backup from the primary if it is reachable) WITH RECOVERY, then repoint applications at the now-standalone secondary.
- Log shipping can absolutely serve as a reporting solution if the business can tolerate the restore interval's inherent delay — it is a valid, low-cost answer when asked, with the caveat clearly stated.

7.3 Database Mirroring (Legacy)

- High Safety (synchronous) — principal waits for the mirror to harden the log before committing; supports a witness for automatic failover.
- High Performance (asynchronous) — principal commits without waiting for the mirror; no automatic failover, manual/forced failover only, and some potential data loss on failover.
- Mirroring requires the FULL recovery model only — it cannot be configured against a database in bulk-logged or simple recovery.
- Mirroring is deprecated (since SQL Server 2012) and still present mainly for backward compatibility through SQL Server 2022/2025 — Microsoft's stated guidance is to migrate any remaining mirroring topology to an Availability Group.
- If transactional replication is configured on a mirrored database and you fail over to the mirror, replication does not follow automatically — the distributor/publisher metadata must be manually repointed at the new principal (typically via `sp_changedistributor` or a redirect/reconfigure of the publication) before replication resumes.

7.4 Transactional Replication — Architecture & Agents

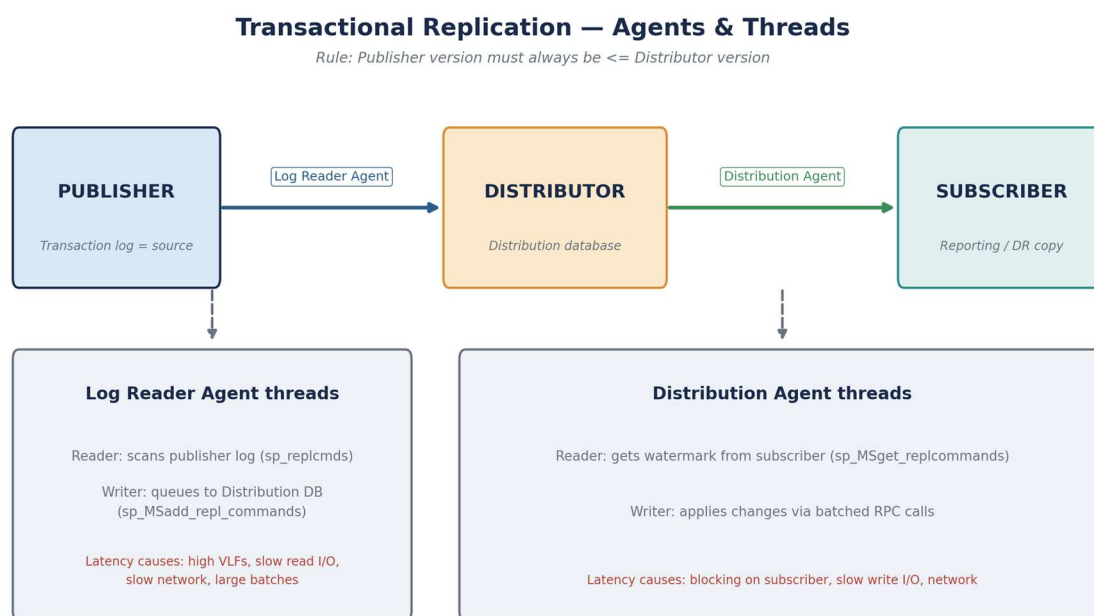


Figure 7.1 — Transactional replication: four distinct threads across two agents, each a separate latency-troubleshooting point.

Internally, replication's continuous data movement runs through two agents and four threads, and knowing exactly which thread is slow is the difference between a generic and an expert answer in an interview:

Thread	Mechanism	Typical root causes when slow
Log Reader Agent — Reader	Scans the publisher's transaction log via <code>sp_replcmds</code>	High VLF count, slow read I/O, slow network, very large batches of replicated transactions

Log Reader Agent — Writer	Queues transactions into the distribution database via <code>sp_MSadd_repl_commands</code>	Blocking on the distribution database, high I/O, slow write I/O, slow network
Distribution Agent — Reader	Reads the subscriber's watermark from <code>MSreplication_subscriptions</code> , then pulls pending commands via <code>sp_MSget_replcommands</code>	Distribution database contention, network latency to the subscriber
Distribution Agent — Writer	Applies changes to the subscriber using batched RPC calls	Subscriber-side blocking/locking, slow subscriber I/O, slow network

To pinpoint whether latency sits between Publisher→Distributor or Distributor→Subscriber without guessing, insert tracker tokens (`sp_posttracertoken`) and read back their timestamps from the Replication Monitor or `MStracer_tokens/MStracer_history` — this gives a precise timestamp at each hop rather than an estimate.

7.5 Replication Topology Rules & Operations

- Publisher version must be less than or equal to Distributor version — a 2019 publisher with a 2017 distributor is unsupported; the distributor must be at the same or a newer build than every publisher it serves.
- Oracle as a transactional replication publisher into SQL Server is technically possible but the feature has been deprecated by Microsoft — the recommended modern path is Change Data Capture (CDC) plus SSIS instead.
- Non-clustered indexes are not replicated to subscribers by default; if a reporting subscriber needs the same indexing as the publisher for query performance, create those non-clustered indexes manually on the subscriber.
- Azure SQL Managed Instance supports only snapshot replication and uni/bi-directional transactional replication — merge replication is not supported, and MI cannot blend roles (e.g. publisher on-prem with distributor on MI and subscriber on Azure SQL DB is not a supported topology; MI's publisher and distributor must both be local to MI or both be remote).
- Changing the distributor admin password: use `sp_changedistributor_password`, then restart the affected distribution agents and monitor the Replication Monitor to confirm subscribers reconnect cleanly.
- Setting up replication inside an Always On environment: configure the publication against whichever replica is currently primary (replication is publisher-role-bound, not AG-aware) — after any AG failover, replication agents must be reconfigured/redirected to the new primary, and this redirection step should be built into your patching and failover runbooks rather than discovered after the fact.

7.6 Interview Questions

Q31. What are the differences between mirroring and log shipping?

A: Mirroring is near-real-time and transaction-by-transaction (optionally synchronous with automatic failover via a witness); log shipping moves whole log backups on a schedule and necessarily lags by at least one backup/restore interval, but is simpler, cheaper, and works fine even on editions/scenarios where mirroring/AG aren't available.

Q32. Can mirroring be configured with the bulk-logged recovery model?

A: No — mirroring requires the FULL recovery model only.

Q33. Can log shipping be configured with the bulk-logged recovery model?

A: Yes, log shipping supports both FULL and BULK_LOGGED recovery models, though bulk-logged reduces the precision of point-in-time recovery within any log backup that contains a minimally-logged bulk operation.

Q34. Explain the modes available in database mirroring.

A: High Safety (synchronous, with optional witness for automatic failover) and High Performance (asynchronous, manual/forced failover only, with possible data loss).

Q35. How will you recover the secondary database in log shipping after the primary fails?

A: Stop the copy/restore jobs, apply the tail-log backup from the primary if reachable plus any already-shipped, un-restored log backups WITH RECOVERY, then repoint application connections at the now-standalone former secondary.

Q36. How does data move from publisher to subscriber in transactional replication?

A: The Log Reader Agent's reader thread scans the publisher's transaction log (sp_replcmds) and its writer thread queues commands into the distribution database (sp_MSadd_repl_commands); the Distribution Agent's reader thread then pulls pending commands for that subscriber (sp_MSget_replcommands) and its writer thread applies them via batched RPC calls.

Q37. How do you determine whether replication latency sits between publisher and distributor, or between distributor and subscriber?

A: Insert tracker tokens (sp_posttracertoken) and read the per-hop timestamps back from the Replication Monitor/MStracer tables — this isolates exactly which leg of the pipeline is slow instead of guessing from aggregate latency.

Q38. Is it possible to configure replication where the publisher is SQL Server 2019, the distributor is SQL Server 2017, and the subscriber is SQL Server 2022?

A: No — the publisher's version cannot exceed the distributor's version, so a 2019 publisher requires a distributor at 2019 or later; the subscriber's higher version (2022) is not the problem here.

Q39. Can you configure transactional replication from an Oracle publisher into SQL Server?

A: Technically yes, but Microsoft has deprecated this feature; the recommended modern approach is Change Data Capture (CDC) combined with SSIS instead.

Q40. If replication is configured and you fail over a mirrored database to the mirror, does replication continue automatically?

A: No — replication metadata is bound to the principal at configuration time; after a mirroring failover you must manually repoint/reconfigure the publication (e.g. via sp_changedistributor or redirecting the publisher) to the new principal before replication resumes.

Q41. How do you manage replication during Windows patching or an AG failover, if replication is layered on top of Always On?

A: Replication is bound to whichever replica is primary, not AG-aware on its own — build an explicit step into your patching/failover runbook to reconfigure or redirect replication agents to the new primary immediately after every planned or unplanned failover, rather than discovering broken replication after the fact.

Q42. What replication types does Azure SQL Managed Instance support, and what's the one topology restriction to remember?

A: Snapshot and uni/bi-directional transactional replication only (no merge replication); and MI cannot mix roles across boundaries — both the publisher and distributor must be local to MI, or both must be remote, never blended with an on-prem/Azure SQL DB endpoint in between.

Q43. How do you change the distributor admin password safely?

A: Run `sp_changedistributor_password`, then restart the affected distribution agent(s) and watch the Replication Monitor to confirm subscribers reconnect and catch up cleanly.

7.7 Quick Revision Notes

- Mirroring needs FULL recovery only; log shipping accepts FULL or BULK_LOGGED.
- Mirroring is deprecated since 2012 — migrate remaining mirroring setups to AG.
- 4 replication threads: LogReader-Reader, LogReader-Writer, DistAgent-Reader, DistAgent-Writer — each maps to a distinct failure mode.
- Tracker tokens (`sp_posttracertoken`) pinpoint which leg (Pub→Dist or Dist→Sub) is actually slow.
- Publisher version ≤ Distributor version, always.
- Replication is not AG-aware — repoint it manually after every failover.
- Azure MI replication: snapshot + transactional only, and never a blended publisher/distributor/subscriber topology across the on-prem/cloud boundary.

Part III — Performance Engineering

Performance questions separate a DBA who can keep an instance alive from one who can make it fast under load. This part covers indexing and statistics strategy, how to read what the optimizer actually decided to do (and why), and TempDB — the one shared system database every workload on the instance fights over, and a perennial favorite for scenario-based interview questions.

Chapter 8 · Indexing, Statistics & Query Optimization

8.1 Concept Overview

Indexes and statistics are how the optimizer decides a plan's shape; get either wrong and every other tuning effort fights an uphill battle. This chapter covers the index-design vocabulary interviewers expect (covering vs. included columns, heap-specific phenomena like forwarded fetches), how SQL Server keeps statistics current, and the SARGability/data-type rules that quietly turn a seek into a scan.

8.2 Covering Indexes vs. Included Columns

- A covering index is any index — clustered or non-clustered — whose key (and, if non-clustered, INCLUDE) columns satisfy every column referenced by a query, so the engine never needs to look up the base table/clustered index at all.
- Included columns specifically: columns added via INCLUDE(...) live only at the leaf level of a non-clustered index, not in the intermediate/root levels — keeping the B-tree's non-leaf levels narrow (and therefore cheaper to traverse) while still letting the leaf satisfy the query without a lookup.
- The practical guidance: put columns used in WHERE/JOIN/ORDER BY in the key; put columns only ever SELECTed (never filtered/sorted on) in INCLUDE.

8.3 Statistics

- Index statistics and column statistics are both created and maintained automatically by the engine (AUTO_CREATE_STATISTICS / AUTO_UPDATE_STATISTICS); manual statistics on arbitrary column combinations are possible via CREATE STATISTICS but are rarely needed outside specific tuning exercises.
- Leaving auto-update-statistics enabled is the right default; for very large, business-critical tables the recompile-and-rescan cost triggered by an automatic stats update can itself cause a performance dip at the wrong moment, which is why some shops disable auto-update on specific huge tables and instead refresh statistics on a controlled schedule.
- Incremental statistics (2014+) track changes per-partition, so updating statistics on a partitioned table only rebuilds the statistics for the partitions that actually changed, instead of rescanning the entire table.
- The histogram on any single statistics object has a maximum of 201 steps, and it is built only on the leading column of a composite index/statistics object — columns 2..N have no histogram of their own, only the density vector.
- To check whether a statistics object was last updated with a full scan or sampling, run DBCC SHOW_STATISTICS and compare the header's Rows vs. Rows Sampled — equal values mean a full scan.

Estimates without a histogram — variables and table-valued functions

Construct	Where the estimate comes from	Notes
Local variables in a query (@var)	Density vector only, never the histogram	The optimizer cannot “sniff” a variable's value the way it can a parameter, so it always uses the average density-based guess

Multi-statement table-valued functions (MSTVF)	Fixed guess until 2017	1 row estimated through SQL Server 2012; 100 rows from 2014-2016; accurate cardinality from 2017 onward via interleaved execution / intelligent query processing
Table variables	Fixed 1-row guess until 2017	Still effectively 1 row through 2017; deferred compilation (2019+) improves this by compiling the statement only after the table variable is first populated, giving a real row count

Automating a legacy fix at scale

If you need LEGACY_CARDINALITY_ESTIMATION turned on across thousands of databases rather than one at a time: on SQL Server 2014 enable Trace Flag 9481 instance-wide to force the legacy CE regardless of compatibility level; from SQL Server 2016 onward, loop through sys.databases and toggle the LEGACY_CARDINALITY_ESTIMATION database-scoped configuration per database with a cursor/while-loop script.

8.4 SARGability & Implicit Conversion

- A SARGable predicate lets the engine seek an index directly (e.g. WHERE OrderDate >= '2026-01-01'); wrapping the column in a function or expression on the left side of the comparison (e.g. WHERE YEAR(OrderDate) = 2026) forces a scan, because the engine must evaluate the function for every row before it can compare — this is the single most common, fixable source of unnecessary CPU and index/table scans.
- Comparing nvarchar to varchar causes no performance penalty when SQL Server needs to widen the varchar value to nvarchar; the expensive direction is comparing varchar to nvarchar where the varchar column itself must be implicitly converted for every row — this manifests as high CPU and, on busy systems, visible SOS_SCHEDULER_YIELD waits as the CPU spends its quantum doing conversion work instead of yielding cooperatively.

8.5 Heap-Specific Behavior: Forwarded Fetches

A forwarded record occurs only on a heap (a table with no clustered index): when a row grows too large to stay on its original page, SQL Server moves it to a new page and leaves a forwarding pointer behind at the old location. Every subsequent scan that hits the old pointer has to chase it to the new page — an invisible tax on heap scan performance that clustered tables simply do not experience.

- Identify forwarded fetches across every database quickly with the community tool sp_BlitzIndex @GetAllDatabases=1 (or query sys.dm_db_index_physical_stats with mode DETAILED and inspect forwarded_record_count).
- Fix: either add a clustered index to the table (converting it out of heap form), or simply rebuild the heap — ALTER TABLE <table> REBUILD — which removes existing forwarding pointers without changing the table's heap structure.

8.6 Maintenance: Reindex vs. Reorganize, and Tracking Progress

Operation	What it does	When to prefer it
-----------	--------------	-------------------

REORGANIZE	Walks the leaf level in logical order, defragmenting in place with minimal log/resource impact; always online	Lower fragmentation (roughly 5-30%), limited maintenance window, need to stay fully online
REBUILD	Drops and recreates the index from scratch (online or offline depending on edition/options), also refreshes statistics with a full scan	Higher fragmentation (>30%), need stats refreshed, or want to change fill factor/compression

Fragmentation analysis itself: on modern versions, query `sys.dm_db_index_physical_stats`; on SQL Server 2000/2005-era systems the equivalent tool was `DBCC SHOWCONTIG`.

- `sys.dm_db_index_physical_stats` supports three scan modes: `LIMITED` (default — fastest, least accurate, safe to run anytime in production), `SAMPLED` (a statistical sample of pages — reasonably fast and reasonably accurate, generally safe in production), and `DETAILED` (scans every page — most accurate, but should not be run during business hours against very large tables due to the I/O it generates).
- To track the percentage completion of a long-running index build/rebuild, the durable, version-proof technique is querying `sys.dm_exec_requests.percent_complete` for the session running the operation (covers index rebuilds, several `DBCC` operations, and rollbacks); `SET STATISTICS PROFILE ON` is a different, older mechanism for annotating an executed query's actual row counts per operator and is not the right tool for tracking rebuild progress, despite sometimes being cited as such.

8.7 Disabling a Clustered Index

Disabling the clustered index (`ALTER INDEX ... DISABLE`) makes the entire table inaccessible for any DML, and even most `SELECT`s, until the index is rebuilt or dropped — because the clustered index is the data; disabling it doesn't just remove a lookup path, it removes the table's storage structure from active use.

8.8 Interview Questions — Beginner

Q1. What is the difference between a covering index and included columns?

A: A covering index is the overall property that an index's columns fully satisfy a given query without a base-table lookup. Included columns are a specific mechanism for achieving that: columns added with `INCLUDE()` exist only at the leaf level of a non-clustered index, keeping the index's upper levels compact while still letting the leaf satisfy the query.

Q2. What is the difference between reindex (REBUILD) and reorganize (REORGANIZE)?

A: `REORGANIZE` defragments in place online with minimal resource impact and is suited to lighter fragmentation; `REBUILD` drops and recreates the index (refreshing statistics with a full scan in the process) and suits heavier fragmentation or when you also need to change fill factor/compression.

Q3. What is fragmentation, and how do you find it?

A: Fragmentation is the degree to which an index's logical page order no longer matches its physical order on disk, increasing the I/O needed to scan it. On modern SQL Server, query `sys.dm_db_index_physical_stats`; on SQL Server 2000/2005, the equivalent was `DBCC SHOWCONTIG`.

Q4. Is it generally a good idea to leave auto-update statistics enabled?

A: Yes, in general — but for very large, business-critical tables it can be worth disabling auto-update specifically on those tables, because an automatic stats refresh at the wrong moment consumes

significant resources and can itself cause a performance dip; refresh those tables on a controlled schedule instead.

Q5. What are statistics, and why update them regularly?

A: Statistics are histograms/density information the optimizer uses to estimate row counts and choose a plan shape; stale statistics lead to wrong cardinality estimates and therefore poor plan choices, so keeping them current (via auto-update or scheduled maintenance) is essential for stable query performance.

8.9 Interview Questions — Intermediate

Q6. Is disabling parameter sniffing a good general performance-tuning practice?

A: No — disabling it forces every execution plan to use density-vector-based average estimates instead of estimates tailored to the actual parameter values supplied, which is usually worse for typical, varied workloads. Parameter sniffing problems are better solved with targeted techniques (OPTIMIZE FOR, query hints, recompiles on specific procedures, or Query Store plan forcing) rather than a blanket disable.

Q7. What are incremental statistics and why do they matter on partitioned tables?

A: Incremental statistics (2014+) track changes at the partition level, so updating statistics only rescans the partitions that actually changed instead of the entire table — a major cost saving on large partitioned tables where only the newest partition typically receives new data.

Q8. What are the three modes inside sys.dm_db_index_physical_stats, and which is safe to run during business hours on very large tables?

A: LIMITED (default, fastest, least accurate), SAMPLED (statistical sample, reasonably accurate), and DETAILED (full scan, most accurate). LIMITED and SAMPLED are generally safe in production; DETAILED should be avoided during business hours on very large tables because of the I/O it drives.

Q9. How do you track the percentage completion of an index creation/rebuild?

A: Query sys.dm_exec_requests.percent_complete for the session running the operation — it reports live progress for index rebuilds and several other long-running operations.

Q10. What is a forwarded fetch, and which tables are affected?

A: A forwarded record happens only on heap tables, when a row that has grown too large for its page is relocated and a forwarding pointer is left behind; every scan that touches the old pointer must chase it to the new location. Clustered tables are not affected because there is no heap structure for a row to be “forwarded” within.

Q11. How do you find and fix forwarded fetches across every database on an instance?

A: Identify them with sp_BlitzIndex @GetAllDatabases=1 or by inspecting forwarded_record_count from sys.dm_db_index_physical_stats in DETAILED mode; fix by adding a clustered index to the table, or by rebuilding the heap with ALTER TABLE <table> REBUILD.

Q12. Will comparing an nvarchar column to a varchar value cause a performance problem?

A: Comparing varchar to nvarchar is fine — SQL Server widens the varchar value with no real cost. The expensive direction is the reverse: comparing nvarchar to varchar forces an implicit conversion of the varchar column for every row, driving high CPU and potentially visible SOS_SCHEDULER_YIELD waits.

8.10 Interview Questions — Advanced & Scenario-Based

Q13. You need LEGACY_CARDINALITY_ESTIMATION enabled across thousands of databases without touching each one by hand. How?

A: On SQL Server 2014, enable Trace Flag 9481 instance-wide so the legacy CE applies regardless of compatibility level. From SQL Server 2016 onward, write a script that loops over sys.databases (excluding system databases and read-only/offline ones) and sets the LEGACY_CARDINALITY_ESTIMATION database-scoped configuration to ON for each, wrapped in TRY/CATCH to log and skip any database that errors out.

Q14. Why does a multi-statement table-valued function sometimes cause terrible plans on older SQL Server versions even when the underlying query is fine?

A: Through SQL Server 2012 the optimizer assumed exactly 1 row would come back from an MSTVF, and 100 rows from 2014-2016 — both fixed guesses, regardless of reality. If the function actually returns hundreds of thousands of rows, every downstream join/aggregate in the plan is built around a wildly wrong cardinality estimate. SQL Server 2017+ fixes this with interleaved execution as part of intelligent query processing, which actually executes the MSTVF first to get a real row count before optimizing the rest of the plan.

Q15. A vendor application uses table variables heavily in stored procedures and performance degrades as data grows. What's the underlying cause and the fix?

A: Table variables were estimated at a fixed 1 row through SQL Server 2017, regardless of how many rows they actually hold, leading the optimizer to under-allocate memory and choose nested-loop-style plans that don't scale. SQL Server 2019's deferred compilation defers compiling the statement until after the table variable is first populated, giving the optimizer a real cardinality to work from — upgrading/raising compatibility level to 150+ is the structural fix; rewriting hot paths to use temp tables (which do have real statistics) is the workaround on older versions.

Q16. A report writer insists their query 'should' use an index, but the plan shows a scan. The WHERE clause is WHERE CONVERT(varchar(10), OrderDate, 101) = '06/19/2026'. What's wrong and how do you fix it?

A: The CONVERT() wrapped around the indexed column makes the predicate non-SARGable — the optimizer must evaluate the conversion for every row before it can compare, forcing a scan regardless of any index on OrderDate. Rewrite the predicate to compare OrderDate directly against a date range (e.g. WHERE OrderDate >= '2026-06-19' AND OrderDate < '2026-06-20') so the column stays bare and the engine can seek.

8.11 Quick Revision Notes

- Covering index = property; INCLUDE columns = one mechanism, leaf-level only.
- Histogram: max 201 steps, leading column of the index/stats object only.
- Variables → density vector; MSTVF 1→00→accurate (2012→2014-16→2017+); table variables 1-row until deferred compilation (2019+).
- varchar→nvarchar: free. nvarchar column compared to varchar value: implicit conversion, high CPU, SOS_SCHEDULER_YIELD.
- Forwarded fetches = heap-only; fix with clustered index or ALTER TABLE ... REBUILD.
- Index build/rebuild progress: sys.dm_exec_requests.percent_complete, not STATISTICS PROFILE.
- DETAILED mode of dm_db_index_physical_stats = avoid during business hours on huge tables.

Chapter 9 · Execution Plans, Optimizer Internals & Wait Statistics

9.1 Concept Overview

Reading an execution plan fluently — knowing why a particular operator appears, what it costs, and what it implies about missing indexes or stale assumptions — is the most discriminating skill in a performance-tuning interview. This chapter also covers Accelerated Database Recovery, a SQL Server 2019+ engine change that quietly affects every long-running-transaction rollback scenario.

9.2 Why Specific Operators Appear

Sort

- An explicit ORDER BY with no supporting index in that order.
- GROUP BY, when no index already orders the grouped columns — the optimizer sorts first so it can group adjacent rows together.
- DISTINCT — behaves like GROUP BY internally; if the data isn't already ordered (e.g. via an ordered index seek/scan), a Sort precedes duplicate removal.
- Merge Join — requires both inputs to already be sorted on the join key; if neither side has a supporting clustered/ordered index, the optimizer injects a Sort before the join.

Hash

- JOIN — when the optimizer picks Hash Join over Nested Loop or Merge Join, usually because no useful index exists on the join column(s); Hash Join consumes Query Execution (QE) memory to build its hash table and is the most resource-intensive of the three physical join operators.
- DISTINCT and UNION — a Hash Aggregate / Hash Match operator removes duplicates without requiring sorted input first.
- SUM/AVG/MAX/MIN and other aggregates — can be implemented as a Hash Aggregate when grouping is involved and no ordered path exists.

Compute Scalar

Appears whenever the engine needs to evaluate an expression or function result that isn't a stored column — classic examples are computing a derived value (e.g. SELECT (MonthlySalary + PF) AS TotalSalary) or, less obviously, when a function/expression has been pushed to the right-hand side of an equality predicate as part of the optimizer's own rewriting (rather than appearing as a Filter), which is the more advanced detail interviewers sometimes probe for.

9.3 Subtree Cost, Parallelism & MAXDOP

- Subtree cost shown at any operator in a graphical plan is the cumulative estimated cost of that operator plus every operator beneath it in the tree.
- Cost Threshold for Parallelism (default 5) is the subtree-cost bar a query (or part of one) must cross before the optimizer will even consider a parallel plan; below the threshold, the query always runs single-threaded regardless of MAXDOP.
- Things that prevent or restrict parallelism even above the threshold: certain scalar/multi-statement UDF calls, some system-function references, explicit MAXDOP 1 hints, Resource Governor workload group caps, and (historically) certain cursor types.

9.4 Caching & Cardinality Behaviors

- Retrieving the actual execution plan straight from the cache (rather than re-executing with actual-plan capture turned on) became possible from SQL Server 2019 by enabling the `LAST_QUERY_PLAN_STATS` database-scoped configuration, letting you pull `sys.dm_exec_query_plan_stats` for a cached plan after the fact.
- Local variable estimates always come from the density vector, never the histogram — see Chapter 8.3 for the full estimate-source comparison across variables, MSTVFs, and table variables.

9.5 Scalar UDF Performance

Scalar user-defined functions historically forced row-by-row execution and blocked batch-mode/parallel processing. Mitigations, roughly in order of impact:

20. Avoid the UDF altogether by inlining its logic directly into the calling query.
21. Add `SCHEMABINDING` to the function definition — this alone unlocks several optimizer improvements (including, on 2019+, automatic scalar UDF inlining where eligible).
22. Add `RETURNS NULL ON NULL INPUT` where applicable, letting the engine skip the function call entirely for `NULL` inputs.
23. Convert to a natively compiled (memory-optimized) function for extreme-frequency call patterns.
24. Rely on automatic Scalar UDF Inlining (2019+), which rewrites many eligible scalar UDFs into the surrounding query as relational expressions, restoring batch-mode and parallel eligibility without any code change beyond meeting the inlining requirements (deterministic, no side effects, `SCHEMABINDING` in some cases).

9.6 Accelerated Database Recovery (ADR) & the Persisted Version Store

Introduced in SQL Server 2019, ADR maintains a Persistent Version Store (PVS) of row-version data so that a long-running transaction's rollback no longer has to walk and undo the transaction log sequentially. Recovery time for a crashed instance — and the time to roll back a single very long transaction — becomes essentially independent of how many or how large the active transactions were, which is precisely why ADR is the recommended mitigation for workloads that have seen the transaction log balloon due to long-running active transactions.

9.7 Wait Statistics Methodology

- `sys.dm_os_wait_stats` — cumulative, instance-wide waits since the last restart/clear; pair with `sys.dm_os_wait_stats` deltas captured over a known interval to identify what the instance is spending its time waiting on right now, not historically.
- `sys.dm_exec_requests` / `sys.dm_exec_session_wait_stats` — per-session, real-time view of what an individual active request is currently blocked on.
- Query Store — captures plan-level runtime statistics (duration, CPU, logical reads, and from recent versions wait stats per query) over time, making regression hunting (“this query got slower after Tuesday's deployment”) tractable without needing to have been watching live at the time.
- `HADR_SYNC_COMMIT` — covered in Chapter 4.10 as an AG-specific wait; mentioned here as the canonical example of a wait type whose fix is almost never query tuning, but infrastructure (I/O, network, CPU on the synchronous secondary).

9.8 Interview Questions — Beginner

Q17. Why would a query plan include a Sort operator?

A: An ORDER BY with no supporting index in that order; a GROUP BY/DISTINCT where the data isn't already ordered by an index; or as a setup step before a Merge Join, which requires both inputs pre-sorted on the join key.

Q18. Why would a query plan include a Hash operator?

A: Typically a Hash Join chosen over Nested Loop/Merge Join when no useful index exists on the join column(s); also Hash Aggregate for DISTINCT, UNION, or grouped aggregates when there's no ordered path available.

Q19. What is subtree cost in an execution plan?

A: The cumulative estimated cost of an operator plus every operator below it in the plan tree.

Q20. What is cost threshold for parallelism?

A: The subtree-cost value (default 5) a query must exceed before the optimizer will even consider a parallel plan; below it, the query always runs single-threaded no matter what MAXDOP allows.

9.9 Interview Questions — Intermediate & Advanced

Q21. When would you see a Compute Scalar operator in a plan?

A: Whenever an expression or function result that isn't a stored column needs evaluating — a derived/computed SELECT column is the obvious case; a subtler case is when the optimizer has pushed a function/expression to the right-hand side of an equality predicate during its own query rewriting.

Q22. What can prevent SQL Server from using parallelism even when subtree cost exceeds the threshold?

A: Certain scalar/multi-statement UDF calls, references to specific system functions, explicit MAXDOP hints or session-level settings, Resource Governor workload-group MAXDOP caps, and some cursor types.

Q23. How can you retrieve the actual execution plan for a query straight from the plan cache, without re-running it with actual-plan capture on?

A: From SQL Server 2019, enable the LAST_QUERY_PLAN_STATS database-scoped configuration, then query sys.dm_exec_query_plan_stats for the cached plan handle to get the most recent actual runtime statistics.

Q24. What is the Persisted Version Store and how does it change rollback/recovery time?

A: It's the row-versioning store maintained by Accelerated Database Recovery (2019+); because old row versions are kept in this store rather than requiring a sequential transaction-log walk to undo, rollback and crash-recovery time become largely independent of how long-running or numerous the active transactions were — directly mitigating log-growth problems caused by long open transactions.

Q25. What are the main techniques to fix performance problems caused by scalar UDFs?

A: Inline the logic directly, add SCHEMABINDING (which also enables automatic inlining eligibility), add RETURNS NULL ON NULL INPUT, convert to a natively compiled function for very hot call paths, or rely on SQL Server 2019+'s automatic Scalar UDF Inlining for functions that meet the inlining requirements.

Q26. A nightly batch job suddenly takes 5x longer after a large transaction was rolled back mid-batch the previous night, even though that transaction's table wasn't touched by tonight's job. What would you check?

A: Whether Accelerated Database Recovery is enabled — without it, a large rollback (or the recovery that followed an unclean shutdown during that rollback) walks the transaction log sequentially, and log

growth/secondary log file activity from that rollback can leave artifacts (VLF count growth, log fragmentation) that slow subsequent operations until a log backup/checkpoint catches up; enabling ADR on the database makes future rollbacks of this kind near-instant regardless of transaction size.

9.10 Quick Revision Notes

- Sort: ORDER BY / unordered GROUP BY-DISTINCT / pre-Merge-Join setup.
- Hash: Hash Join (no useful index) / Hash Aggregate for DISTINCT-UNION-grouped aggregates.
- Subtree cost = cumulative cost of an operator + everything beneath it; Cost Threshold for Parallelism (default 5) gates parallel consideration entirely.
- Actual plan from cache: LAST_QUERY_PLAN_STATS (2019+) + sys.dm_exec_query_plan_stats.
- ADR + Persisted Version Store (2019+): rollback/recovery time decoupled from transaction size.
- Scalar UDF fixes, in order of effort: inline > SCHEMABINDING > NULL-on-NULL > native compilation > rely on auto-inlining (2019+).

Chapter 10 • TempDB Internals & Contention

10.1 Concept Overview

TempDB is the one system database every workload on the instance shares — temp tables and table variables, sort/spool/hash work tables, and row versioning for snapshot isolation, online index rebuilds, triggers, and MARS all land here. Recreated fresh from model on every restart, it is also the most common source of allocation contention on busy, highly concurrent instances, making it one of the most reliable scenario-question topics in any DBA interview.

TempDB Allocation Contention

PAGELATCH waits on PFS / GAM / SGAM pages signal contention; fix with multiple equal-sized data files

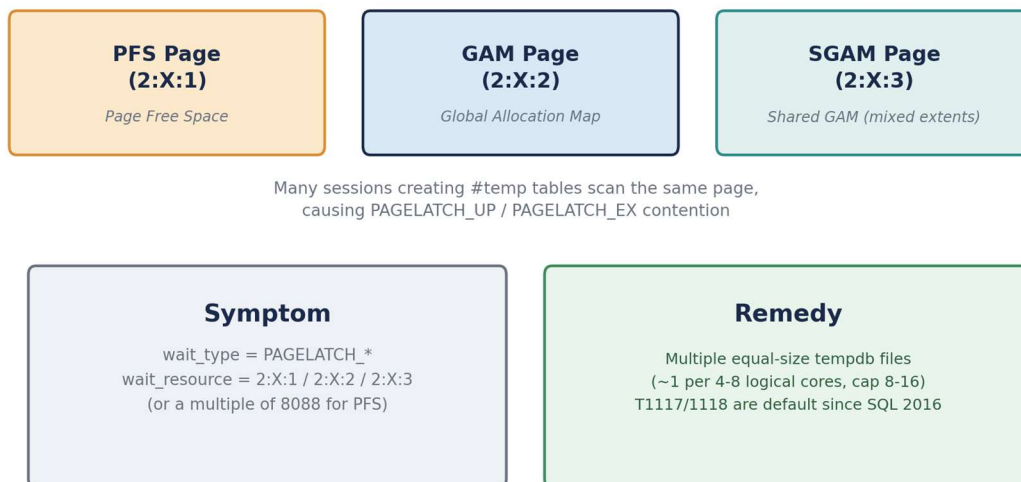


Figure 10.1 — PFS / GAM / SGAM page contention vs. metadata contention inside TempDB.

10.2 Object Allocation Contention vs. Metadata Contention

Contention type	Symptom (wait_resource pattern)	Root cause
Object allocation contention	PAGELATCH_xx on 2:X:1 (PFS), 2:X:2 (GAM), 2:X:3 (SGAM), or 2:X:<multiple of 8088> (also PFS)	Many sessions concurrently creating objects (temp tables, work tables) all need to scan the same PFS/GAM/SGAM pages to find a free page in a mixed extent to allocate — fundamentally an I/O-bound, allocation-page bottleneck
Metadata contention	PAGELATCH_xx on higher-numbered index/data pages such as 2:1:111, 2:1:118, 2:1:122	Many sessions simultaneously creating/dropping temp tables contend on TempDB's own system catalog pages (in the buffer)

	pool/memory), not on disk allocation structures
--	---

Object creation mechanics: creating a new object needs two pages allocated from a mixed extent — one for the Index Allocation Map (IAM) page, one for the object's first data page. To find a free mixed page, the engine scans a Page Free Space (PFS) page, and always starts that scan from the same file/SGAM page — which is exactly why many concurrent object-creation requests collide on the same few allocation pages under heavy concurrency.

10.3 Mitigations

- Multiple, equally-sized TempDB data files — a long-standing rule of thumb is roughly one data file per logical CPU core up to 8 files, then evaluate further increases based on observed contention rather than blindly continuing to add files. Equal sizing matters because the proportional-fill algorithm distributes new allocations round-robin across files based on free space ratio — uneven file sizes skew that distribution back toward a single “hot” file.
- Trace Flags 1117 (grow all files in a filegroup together, keeping them equally sized) and 1118 (always allocate full extents for user objects, avoiding mixed-extent contention) were essential tuning flags through SQL Server 2014, but became the default, built-in behavior for TempDB specifically from SQL Server 2016 onward — you no longer need to set them manually on 2016+.
- Memory-optimized TempDB metadata (2019+) moves the system tables backing temp table/table variable metadata into lock-free, memory-optimized structures, eliminating metadata-page contention as a class of problem entirely when enabled.
- Temporary object caching reuses the same temp table's metadata/space across repeated calls to the same stored procedure rather than fully creating/dropping it every execution, reducing allocation-page churn from object-creation-heavy procedures.

10.4 Sizing, File Placement & a Real Production Fix

- Azure SQL Managed Instance provisions 12 TempDB data files and 1 log file by default — configurable, but a sensible starting point already reflecting the “multiple files” guidance above.
- On Azure VM IaaS deployments, TempDB should sit on the local, ephemeral, low-latency disk (the D: drive on Windows Azure VMs) rather than a remote/persistent data disk — this is the single highest-leverage TempDB placement decision on IaaS.

PRODUCTION SCENARIO — Post-migration performance degradation traced to TempDB

Symptom: after migrating a critical SQL Server 2016 workload from on-premises to an Azure VM, the SQL error log showed repeated “operations exceeding 15ms” I/O warnings.

Root cause #1: TempDB had 32 data files (matched to 32 vCPUs) sitting on standard, non-premium remote disk — far more files than needed and on the wrong storage tier.

Root cause #2: TempDB files were left on a generic data drive instead of the VM's local D: drive (the fast, ephemeral disk every Azure VM provides).

Root cause #3: the VM series itself (E32asv4) was a lower-throughput-per-vCPU SKU than the workload needed.

Fix: reduced TempDB from 32 files to 12, relocated the files to the D: drive, and moved the VM to the E32bdsV5 series for better storage throughput per core — eliminating the I/O warnings.

10.5 Readable Secondaries & Other TempDB Touchpoints

- Enabling readable secondaries in an Always On AG adds roughly a 14-byte row-versioning overhead per row for DML, because the secondary's forced snapshot isolation drives row versioning into TempDB — a small but real TempDB sizing input to factor in when secondaries serve heavy write-replication volumes (the redo stream itself, not application writes).
- Default statistics created for ad hoc queries against a read-only secondary are stored in TempDB, since the secondary database itself cannot be written to.
- DBCC CHECKDB stages fact tables and aggregation results in TempDB — slow TempDB I/O directly slows down CHECKDB, which is why TempDB health belongs on the checklist whenever CHECKDB runtimes regress (see Chapter 1.5).
- SQL Server 2025's TempDB resource governance option lets you cap/“sandbox” an individual session or login's TempDB footprint, including sort spills — a direct, modern answer to the classic “one runaway query fills TempDB and blocks the whole instance” scenario.

10.6 Interview Questions — Beginner

Q27. Which filegroup is the transaction log file created in?

A: None — log files are never part of any filegroup; filegroups are a data-file-only concept.

Q28. How many TempDB files does Azure SQL Managed Instance create by default?

A: 12 data files and 1 log file, though this is configurable.

Q29. Do you still need Trace Flags 1117 and 1118 on modern SQL Server?

A: No — both behaviors (uniform filegroup growth and full-extent allocation for user objects) became the built-in default for TempDB from SQL Server 2016 onward.

10.7 Interview Questions — Intermediate & Advanced

Q30. What is the difference between object allocation contention and metadata contention in TempDB?

A: Object allocation contention is PAGELATCH waiting on PFS/GAM/SGAM allocation-map pages (2:X:1, 2:X:2, 2:X:3, or PFS-multiples of 8088) when many sessions race to allocate pages from mixed extents — an I/O-bound problem. Metadata contention is PAGELATCH waiting on TempDB's own system catalog pages (higher page numbers, e.g. 2:1:111/118/122) when many sessions simultaneously create/drop temp tables — a memory/buffer-bound problem, fixed most directly by enabling memory-optimized TempDB metadata.

Q31. How many TempDB data files should you configure, and why does equal sizing matter?

A: A common starting rule is one equally-sized data file per logical CPU up to 8 files, adjusted from there based on observed contention. Equal sizing matters because the proportional-fill algorithm allocates new extents round-robin weighted by each file's free-space ratio — unevenly sized files skew allocations back onto whichever file currently has more free space.

Q32. Will turning on readable secondaries in an AG add any overhead to TempDB?

A: Yes — roughly a 14-byte row-versioning overhead per row for DML, because the secondary's forced snapshot isolation pushes row versioning into TempDB; this scales with the volume of changes replicated to that secondary.

Q33. A heavily concurrent OLTP workload shows constant PAGELATCH_EX waits on tempdb pages numbered 2:3:1, 2:5:1, 2:7:1 (etc.) across all TempDB files. What's happening and how do you fix it?

A: Those page numbers are PFS pages — this is classic object allocation contention from many sessions simultaneously creating temp tables/work tables and all scanning the same allocation-map pages to find free space. Mitigate by ensuring enough equally-sized TempDB files exist for the core count (TF 1117/1118 behavior is already default on 2016+), enabling memory-optimized TempDB metadata if eligible, and reviewing whether application code is creating temp tables more often than necessary (e.g. inside tight loops) rather than caching/reusing them.

Q34. After migrating to an Azure VM, the error log fills with 'I/O requests taking longer than 15 seconds/operations >15ms' warnings specifically tied to tempdb. What do you check first?

A: TempDB file placement and count, in that order: confirm TempDB data/log files sit on the VM's local ephemeral D: drive rather than a remote persistent disk, confirm the file count is sized to actual core count rather than blindly copied from the source server, and confirm the VM SKU itself provides adequate storage throughput per vCPU for the workload — exactly the three findings from the real-world E32asv4-to-E32bds5 migration case in Section 10.4.

10.8 Quick Revision Notes

- Allocation contention = PFS/GAM/SGAM pages, I/O-bound. Metadata contention = system catalog pages, memory-bound.
- TF 1117/1118 = default behavior since SQL Server 2016; no longer need to set manually.
- TempDB on Azure VM → local D: drive, not remote persistent disk.
- Readable secondaries add ~14 bytes/row of TempDB row-versioning overhead.
- MI default: 12 data files + 1 log file.
- 2025: TempDB resource governance can sandbox a session's TempDB/spill footprint directly.

Part IV — Security & Compliance

Security questions in a Senior/Lead DBA interview rarely stay theoretical — they arrive as compliance scenarios (HIPAA, GDPR), legacy-version constraints (“we’re on 2008 Standard, no native auditing”), or identity-model gotchas (orphaned logins, AD groups as owners). This part consolidates encryption, auditing, and identity/Active Directory topics, then layers on the SQL Server 2025 security baseline you are expected to know in a 2026 interview.

Chapter 11 • Encryption, Auditing & Compliance

11.1 Concept Overview

Database security layers stack in a predictable order: authentication (who are you), authorization (what are you allowed to do), encryption (protecting data at rest and in transit even if a layer above is bypassed), and auditing (proving what actually happened). This chapter focuses on the encryption and auditing layers; identity/authorization is covered in Chapter 12.

11.2 Transparent Data Encryption (TDE) — Deep Dive

TDE encrypts the physical data and log files on disk, transparently to applications, using a certificate-protected Database Encryption Key (DEK). The full key chain is: Service Master Key (SMK, instance-level, protected by the Windows DPAPI/machine key) → Database Master Key (DMK, in master) → Certificate → DEK → the actual data/log pages.

- Always back up the TDE certificate and its private key immediately after creation, and store that backup somewhere other than the server itself — losing the certificate means the encrypted database becomes permanently unreadable, even with valid SQL Server media backups.
- Restoring a TDE-protected database to a different instance requires restoring the certificate (CREATE CERTIFICATE ... FROM FILE ... WITH PRIVATE KEY (...)) into that instance's master database first — see Chapter 4.8 for the Always On AG-specific seeding sequence.
- Azure SQL Database and Managed Instance enable TDE automatically on every database by default — a frequent “what changes when you migrate to PaaS” interview point, since on-premises databases are not TDE-enabled unless someone explicitly turned it on.

11.3 Column-Level Encryption & Always Encrypted

- Column-level (cell-level) encryption uses a Database Master Key and certificate/symmetric key inside the user database itself, encrypted/decrypted explicitly via ENCRYPTBYKEY/DECRYPTBYKEY in T-SQL — it is independent of TDE's certificate chain, so backup/restore mechanics for the database succeed regardless of whether you have the DMK password (see Chapter 2's scenario on this exact distinction).
- Always Encrypted (2016+) moves the encryption/decryption boundary to the client driver — SQL Server never sees plaintext sensitive values at all, even sysadmin cannot view them in clear text via T-SQL — which is the strongest answer when a compliance scenario explicitly requires that DBAs themselves cannot view certain columns.
- Multiple passwords can protect a single Database Master Key (and optionally the Service Master Key can also protect it); this is unlike the Service Master Key itself, which does not support multiple passwords.

11.4 SQL Server Audit & Compliance

- SQL Server Audit (Server Audit + Server/Database Audit Specifications) is the native, supported mechanism for capturing login attempts (successful and failed), permission changes, schema changes, and data access at scale, writing to the Windows Security log, the Application log, or a binary file target.
- Auditing failed logins specifically serves two distinct purposes that interviewers want both of: regulatory compliance (HIPAA, GDPR, SOX, PCI-DSS all require login-attempt evidence), and active

security monitoring (detecting brute-force/credential-stuffing patterns, which can also be an early signal of a denial-of-service attempt against the authentication layer).

- Audit data should be shipped off-box (centralized SIEM, Log Analytics workspace, or a dedicated audit database) rather than left only on the source server — local-only audit logs are themselves a single point of failure/tampering risk that compliance auditors specifically check for.

11.5 Finding Last Login Activity Without Native Auditing

PRODUCTION SCENARIO — SQL Server 2008 Standard edition, no audit feature available, need last-login evidence

Constraint: SQL Server Audit requires Enterprise edition prior to SQL Server 2016 (it became available on Standard from 2016 onward), so a 2008 Standard instance genuinely cannot use native Audit.

Option 1 — Logon triggers: capture EVENTDATA() into a logging table on every successful logon; simple, but adds per-connection overhead and is generally not recommended as a primary mechanism on high-connection-churn systems.

Option 2 — SQL Trace / Profiler with filters: capture Audit Login events to a server-side trace file (never Profiler GUI in production); lighter than a logon trigger but still adds overhead and requires active log rotation/cleanup management.

Option 3 — Scheduled snapshotting: a SQL Agent job that runs every few minutes, capturing sys.sysprocesses (legacy) or sp_whoisactive output into a logging table, building an approximate login-activity history over time from repeated point-in-time snapshots.

All three are valid, defensible answers in an interview — the key insight to volunteer is recognizing which native feature (Audit) is unavailable on that specific edition/version before reaching for a workaround.

The identical three-option toolkit also answers “how do you find unused logins to clean up” — capture login activity over a representative window (ideally including month-end/quarter-end batch jobs), then cross-reference logins that never appear against the full login list to build a safe deprecation candidate list.

11.6 SQL Server 2025/2026 — Security Baseline Update

SQL Server 2025 ships the most significant security upgrade to the on-premises engine in several releases, closing much of the historical gap with Azure SQL's security posture:

Capability	What changed	Why it matters for a 2026 interview
Credential-free connections	SQL Server instances can authenticate outbound to Azure Storage, Key Vault, and REST endpoints using Managed Identity, with no stored secret/connection string credential	On-premises finally gets parity with what Azure SQL has offered for years — removes hard-coded secrets as an attack surface
Password hashing	PBKDF2 replaces the older SHA-based password hash scheme for SQL logins	Materially raises the cost of offline brute-force attacks against a stolen password hash, relevant to any compliance/security-hardening question
Transport security	TDS 8.0 over TLS 1.3 used by default, including for internal	“Is traffic between replicas encrypted by default” now has a

	features (replication, Always On, SQL Agent) — strict encryption enforced by default	materially different, stronger answer than on 2019/2022
Identity	Deeper Microsoft Entra ID integration, Zero Trust principles applied to every access request (authenticate, authorize, encrypt)	Bridges the on-prem vs. Azure SQL identity-model gap covered in Chapter 12

11.7 Interview Questions — Beginner

Q1. What is the purpose of auditing failed logins?

A: Two purposes: regulatory compliance (HIPAA, GDPR, and similar frameworks require evidence of login attempts) and active security monitoring — detecting unauthorized-access attempts or brute-force/credential-stuffing patterns, which can also be an early indicator of a denial-of-service attempt against the login surface.

Q2. Can a Database Master Key be protected by more than one password?

A: Yes — a DMK can have multiple passwords (and optionally the Service Master Key as an additional protector); the Service Master Key itself does not support multiple passwords.

Q3. Does TDE protect data while it is being queried by an application?

A: No — TDE protects data at rest (the physical .mdf/.ldf files and backups); data is decrypted transparently in memory for any authorized query. Protecting specific columns even from query results requires column-level encryption or Always Encrypted instead.

11.8 Interview Questions — Intermediate & Advanced

Q4. Your organization is on SQL Server 2008 Standard edition and needs evidence of who last accessed the database, but native Audit isn't available on that edition/version. What do you do?

A: Use one (or a combination) of: a logon trigger capturing EVENTDATA() into a logging table, a server-side SQL Trace filtered to Audit Login events, or a scheduled job snapshotting sys.sysprocesses/sp_whoisactive output into a table at short intervals — all three build an approximate login history without the native Audit feature, which only became available on Standard edition from SQL Server 2016 onward.

Q5. What's the practical difference between column-level encryption and Always Encrypted for a compliance requirement that says 'DBAs must not be able to view sensitive columns'?

A: Column-level encryption still decrypts inside the SQL Server engine when a query with the right key runs — a sysadmin with access to the key/certificate can still decrypt. Always Encrypted keeps the encryption/decryption boundary entirely on the client driver side, so SQL Server (and therefore any DBA querying it directly) never sees plaintext at all, which is the stronger answer for that specific compliance requirement.

Q6. How would you migrate a strict on-premises encryption posture (TDE + column-level encryption on specific tables) to Azure SQL Managed Instance, and what changes automatically?

A: TDE is enabled automatically on every database in Managed Instance regardless of source configuration, so that part requires no action; column-level encryption/Always Encrypted configuration must be migrated explicitly (keys, certificates, and — for Always Encrypted — column master key/column encryption key metadata) since it is not a default-on platform behavior the way TDE is.

Q7. What concrete security changes does SQL Server 2025 bring that you would highlight in a security-hardening conversation?

A: Credential-free outbound authentication to Azure Storage/Key Vault/REST via Managed Identity, PBKDF2 password hashing replacing the older SHA-based scheme, TDS 8.0 over TLS 1.3 enforced by default (including for internal AG/replication/Agent traffic), and deeper Microsoft Entra ID/Zero Trust integration — together closing most of the historical security gap between on-premises SQL Server and Azure SQL.

11.9 Quick Revision Notes

- TDE chain: SMK → DMK → Certificate → DEK → data/log pages. Always back up the cert + private key off-box.
- Column-level encryption survives a restore without needing keys exported; TDE absolutely does not.
- Always Encrypted = SQL Server (and DBAs) never see plaintext at all; column-level encryption still can, inside the engine.
- No native Audit on Standard edition before SQL Server 2016 → logon triggers / server-side trace / scheduled sp_whoisactive snapshots.
- Azure SQL DB/MI: TDE on by default for every database, no opt-out.
- SQL Server 2025: Managed Identity (credential-free), PBKDF2 hashing, TDS 8.0/TLS 1.3 by default, deeper Entra ID/Zero Trust.

Chapter 12 • Identity, Active Directory & Microsoft Entra ID

12.1 Active Directory Fundamentals for a DBA

- **Organizational Unit (OU)** — a container the AD team uses to apply Group Policy and delegate permissions consistently to a logical grouping of computer/user/service accounts. A typical enterprise layout keeps separate OUs per environment (e.g. Prod, Pre-Prod, UAT, Dev), letting the same delegated permission model scale down in privilege as environments move toward production — for example, granting broader rights like sysadmin more freely in a Dev OU while tightly restricting it in the Prod OU.
- **Windows Service** — a process that the Service Control Manager can start without any user being logged on, and that keeps running after a user logs off, analogous to a daemon on Linux. SQL Server itself, SQL Server Agent, and the Always On cluster components all run as Windows Services, which is why the service account's permissions (not the DBA's interactive login) govern what the engine itself can do on the OS/network.

12.2 AD Groups as Owners — Why the Answer Is Almost Always “No”

Question	Answer	Why
Can a SQL Agent job have an AD group as its owner?	No	A job owner must resolve to a single security context for execution and permission-checking purposes; a group has no single, unambiguous identity to run under
Can a database owner (dbo) be an AD group?	No	Database ownership requires a single principal whose SID drives ownership-chaining and permission checks; a group cannot serve as that single principal

Both restrictions trace back to the same underlying principle: ownership and execution context need exactly one resolvable security identity, while a group is, by definition, a many-member abstraction — useful for granting permissions to many people at once, but unusable wherever the engine needs to resolve “who, specifically, owns/executes this.”

12.3 SPNs, Kerberos & Delegation (Cross-Reference)

See Chapter 4.4 for the full Always On Listener SPN discussion. The general rule for any SQL Server endpoint (default instance, named instance, FCI virtual name, or AG Listener): if Kerberos authentication is required (e.g. for double-hop scenarios through a middle-tier application server), the matching SPN must be registered against the service account running SQL Server for that specific virtual or physical name — missing or duplicate SPNs silently fall back to NTLM, breaking delegation without an obvious error pointing at the real cause.

12.4 Authorization Model: Roles, Ownership Chaining & TRUSTWORTHY

- Fixed server roles (sysadmin, dbcreator, securityadmin, etc.) grant instance-wide capabilities; fixed database roles (db_owner, db_datareader, db_datawriter, etc.) grant database-scoped capabilities; user-defined database roles let you build least-privilege groupings tailored to an application's actual needs.

- Contained database users (2012+) authenticate directly against the database without a matching server-level login at all — useful for AG portability (no orphaned-user reconciliation needed after failover) and for Azure SQL Database, where there is no traditional server-level login concept in the same way.
- Ownership chaining lets a user execute a stored procedure/view that touches underlying objects they have no direct permission on, as long as every object in the chain is owned by the same principal — this breaks the moment ownership differs across objects, or across databases unless the source database has TRUSTWORTHY ON (which is exactly why TRUSTWORTHY being reset on every restore, as covered in Chapter 4.6, is a security feature, not a bug).

12.5 Microsoft Entra ID (Azure AD) Integration

- Azure SQL Database, Managed Instance, and SQL Server 2025 on-premises (via the credential-free/Managed Identity work in Chapter 11.6) can all authenticate principals through Microsoft Entra ID instead of (or alongside) SQL/Windows authentication.
- Entra ID brings centralized identity governance that traditional on-prem AD-bound SQL logins lack out of the box: Conditional Access policies, Multi-Factor Authentication enforcement, Privileged Identity Management (just-in-time elevation instead of standing sysadmin membership), and a single identity plane across Azure resources rather than a per-server local/AD login list.
- Migrating logins specifically: there is no supported native mechanism to transfer SQL logins (with their passwords) from on-premises directly into Azure SQL Database — the recommended approach is to recreate logins with new passwords on the Azure side, or move identity to Entra ID entirely rather than carrying forward SQL authentication. For Azure SQL Managed Instance, however, `sp_help_revlogin`-generated scripts do work to recreate logins with matching SIDs, and Azure Database Migration Service (DMS) is the recommended tool to automate that login migration at scale.

12.6 Interview Questions — Beginner

Q8. What is an Organizational Unit (OU) in Active Directory?

A: A container the AD team uses to apply Group Policy and delegate administrative permissions to a logical grouping of accounts — typically split by environment (Prod, Pre-Prod, UAT, Dev) so permission scope can tighten as you move toward production.

Q9. What is a Windows Service?

A: A process the Service Control Manager can start without an interactive user logged on, and which keeps running after any user logs off — analogous to a Linux daemon; SQL Server, SQL Server Agent, and the cluster service all run this way.

Q10. Can a SQL Agent job have an AD group as its owner?

A: No — a job owner must be a single, resolvable security principal for execution-context purposes; a group cannot serve that role.

Q11. Can a database's owner (dbo) be an AD group?

A: No, for the same reason as job ownership — ownership and the permission/ownership-chaining model require one specific resolvable principal.

12.7 Interview Questions — Intermediate & Advanced

Q12. How do you transfer SQL logins from on-premises to Azure SQL Managed Instance versus Azure SQL Database?

A: For Managed Instance, `sp_help_revlogin`-generated scripts work and preserve SIDs, and Azure Database Migration Service is the recommended way to automate this at scale. For Azure SQL Database, there is no supported native mechanism to carry over SQL logins with their existing passwords — the standard approach is to recreate logins with new passwords, or move to Entra ID-based authentication entirely.

Q13. What advantage do contained database users give you in an Always On AG context?

A: They remove the orphaned-user problem entirely — because authentication happens directly against the database rather than through a matching server-level login, a database can fail over to any replica without needing the SQL login's SID to be manually synchronized and reconciled beforehand.

Q14. Explain ownership chaining and why it depends on the TRUSTWORTHY property across databases.

A: Ownership chaining lets a stored procedure/view execute against underlying objects without the caller needing direct permission on those objects, as long as every object in the chain shares the same owner. Once the chain crosses a database boundary, the source database additionally needs `TRUSTWORTHY ON` for that cross-database chain to be honored — which is precisely why `TRUSTWORTHY` is deliberately not preserved across a `RESTORE` (Chapter 4.6): it forces a conscious decision to re-enable a capability with real security implications, rather than silently carrying it forward.

Q15. What does Microsoft Entra ID integration give you that traditional AD-bound SQL/Windows logins don't?

A: Centralized, policy-driven identity governance across Azure resources: Conditional Access, enforced Multi-Factor Authentication, Privileged Identity Management for just-in-time elevation instead of standing sysadmin membership, and a single identity plane that spans Azure SQL, Managed Instance, and (from SQL Server 2025) on-premises instances as well — rather than per-server local logins or AD-domain-bound logins with no centralized conditional policy layer.

Q16. A security review finds a critical database has TRUSTWORTHY ON and is owned by sa, and several cross-database stored procedures rely on it. What's the risk, and how do you remediate without breaking those procedures?

A: `TRUSTWORTHY ON` combined with `sa` (or any highly-privileged) ownership lets code inside that database execute with elevated, ownership-chained privilege in other databases — a classic privilege-escalation vector if the database is ever compromised. Remediate by changing ownership to a dedicated, least-privilege principal, replacing the cross-database ownership-chaining dependency with explicit, scoped permissions (or signed modules using a certificate-mapped login granted only the specific cross-database rights needed) rather than leaving `TRUSTWORTHY ON` as a blanket workaround, then disabling `TRUSTWORTHY` once the explicit grants are validated in a lower environment first.

12.8 Quick Revision Notes

- OU = AD delegation/policy container, usually split per environment.
- AD groups cannot own a job or a database — ownership needs one resolvable principal.
- SPNs must match the exact virtual/physical name being connected to, or Kerberos silently falls back to NTLM.
- Contained users remove the AG orphaned-login problem entirely.
- `TRUSTWORTHY` gates cross-database ownership chaining — never preserved across `RESTORE`, by design.

- Login migration: MI → sp_help_revlogin + Azure DMS works; Azure SQL DB → recreate logins or move to Entra ID.

Part V — Azure & Cloud Architecture

Every modern SQL Server DBA interview now spends real time on Azure — not as a separate specialism, but as the default deployment target. This part covers the three Azure SQL deployment models end-to-end (Database, Managed Instance, and SQL Server on Azure VMs/IaaS), and the networking constructs that tie them together in an enterprise landscape.

Chapter 13 · Azure SQL Database (PaaS — Database as a Service)

13.1 Concept Overview & Cloud Service Models

Model	SQL Server mapping	Who manages what
IaaS	SQL Server on an Azure Virtual Machine	You manage the OS, SQL Server instance, patching, backups, HA
PaaS	Azure SQL Managed Instance (instance-as-a-service) and Azure SQL Database (database-as-a-service)	Microsoft manages the OS and most infrastructure; you manage databases, security, and performance tuning
SaaS	Not directly applicable to relational databases	Examples: Outlook, Microsoft Teams — fully managed end-user applications, not a database deployment target

Azure SQL Database is the most PaaS of the three SQL options: no instance-level surface at all (no SQL Agent in the traditional sense, no cross-database queries, no native BACKUP/RESTORE statements), in exchange for the least operational overhead and the tightest integration with Azure-native HA/DR and elastic scaling.

13.2 Backup, Restore & Recovery Model Constraints

Question	Answer
Who manages backups?	Fully automated and managed by Microsoft — there is no native BACKUP DATABASE statement available against Azure SQL Database at all
What CAN you export?	Schema via DACPAC, schema+data via BACPAC, or table-level export/import through the wizard if firewall ports are open
Backup retention	Basic tier: 7 days of point-in-time restore; higher tiers (including most vCore tiers): up to 35 days, configurable via long-term retention (LTR) policies for far longer archival
How do you restore to a point in time?	Azure Portal, PowerShell, or Azure CLI, restoring to a new database name within the retention window; Azure Data Factory copy activities or Azure Functions can also drive this but are not the simplest path
Can the pricing tier change during restore?	Yes — restore speed depends on the target service tier, so you can restore to a higher tier for faster restore and downsize afterward; CREATE DATABASE db_copy AS COPY OF server.db_original (SERVICE_OBJECTIVE = 'P2') is the T-SQL equivalent for an online database copy
Can you restore to a server in another region/country?	Yes, within the same region directly; cross-region restore uses geo-restore from the geo-redundant

	backup copy, subject to that backup's own replication lag to the target region
Recovery model	Always FULL — cannot be changed, because Microsoft must guarantee point-in-time restore and the platform's SLA across every database
Default isolation level	READ COMMITTED SNAPSHOT (RCSI) by default
Dropped database recovery	A dropped database remains restorable from its automated backups for the remainder of its retention window (minimum 7 days) — this also answers the 'can we drop databases on weekends to save cost and restore on weekdays' scenario: yes, within the retention window, but the restore itself takes real time and a new logical database name, so this is rarely a true cost-saving pattern in practice once restore time and risk are factored in

Size & tier limits

DTU model: Basic tier caps at roughly 2 GB; Standard tier at roughly 1 TB. vCore model (and DTU Premium): up to roughly 4 TB for General Purpose/Business Critical at most service objectives — so a 5 TB database cannot be restored directly into a Basic-tier target regardless of purchasing model; it must land on a tier whose maximum size accommodates it, then can be downsized only if the data itself is reduced.

BACPAC vs. DACPAC: a BACPAC contains schema and data (used for full database export/import or migration); a DACPAC contains only the schema (used for application/schema deployment and drift comparison without touching data).

13.3 HA/DR, Networking & Cross-Database Access

- Always On AG in the traditional sense does not apply to Azure SQL Database — built-in HA is automatic and zone-redundant depending on tier (Business Critical/Premium maintain multiple synchronously-replicated copies under the hood). Cross-region DR uses Active Geo-Replication (up to 4 readable secondaries) or Auto-Failover Groups (a managed group of databases that fails over together, with an application-facing listener endpoint).
- There is no dedicated VNet for Azure SQL Database the way there is for a VM or Managed Instance — connectivity is secured via Private Endpoint (recommended, no public exposure) or Virtual Network Service Endpoints (restricts the public endpoint to specific VNets/subnets without giving the database its own private IP).
- You cannot directly query across two Azure SQL Databases the way you can across databases on the same on-premises/VM/MI instance — there is no USE statement crossing database boundaries. Use Elastic Query / External Tables to query another Azure SQL Database, or move to Managed Instance/IaaS if true cross-database T-SQL is a hard requirement.
- SQL Agent does not exist on Azure SQL Database; scheduled/automated workloads use Elastic Database Jobs, Azure Automation runbooks, or Azure Functions on a timer trigger instead.

13.4 Interview Questions

Q1. What is the maximum point-in-time restore window for Azure SQL Database?

A: Basic tier: 7 days. Premium and most vCore tiers: up to 35 days, with optional Long-Term Retention policies extending archival far beyond that for compliance needs.

Q2. How are backups performed on Azure SQL Database, and who is responsible?

A: Fully automated and managed by Microsoft — there is no native BACKUP DATABASE statement; the only manual export options are schema (DACPAC) or schema+data (BACPAC).

Q3. Can the recovery model of an Azure SQL Database be changed?

A: No — it is always FULL, since the platform must guarantee point-in-time restore for every database regardless of customer configuration.

Q4. Is it feasible to restore a 5 TB database from a Standard-tier (DTU) backup directly to a Basic-tier test server?

A: No — Basic tier tops out at roughly 2 GB, far below 5 TB; the restore target must be a tier whose maximum database size accommodates the data, then it can be downsized afterward only by actually reducing the data volume.

Q5. Can you access data in one Azure SQL Database from a query running against another Azure SQL Database?

A: Not via a direct cross-database query the way you can on-prem/laaS/MI — use Elastic Query with External Tables instead, or reconsider Managed Instance if true cross-database T-SQL is required.

Q6. How do you restore yesterday's backup of an Azure SQL Database to a different server, possibly in another country?

A: Use the Azure Portal, PowerShell, or Azure CLI to restore to a specific point in time within the retention window; for a different region, this becomes a geo-restore from the geo-redundant backup copy. Azure Data Factory copy activities and Azure Functions are alternative but less direct paths for the same outcome.

Q7. A team wants to drop their Azure SQL Database every weekend and restore it on Monday to save cost. Will this work, and is it actually a good idea?

A: It will technically work, because point-in-time recovery (minimum 7 days) covers dropped databases within the retention window — but in practice the restore itself takes real time and creates a new logical database object, so the apparent cost saving needs to be weighed against restore latency, automation complexity, and the risk of restore failure before relying on this as a standing cost-optimization pattern.

Q8. How do you run scheduled jobs (the equivalent of SQL Agent) against Azure SQL Database?

A: Elastic Database Jobs (purpose-built for fan-out across many Azure SQL Databases), Azure Automation runbooks, or Azure Functions on a timer trigger — SQL Server Agent itself is not available on Azure SQL Database.

13.5 Quick Revision Notes

- No native backups, no SQL Agent, no cross-database queries on Azure SQL Database — the three classic PaaS-DB gaps.
- Recovery model always FULL; default isolation RCSI.
- Retention: Basic 7 days, Premium/most vCore up to 35 days + optional LTR.
- HA built-in by tier; cross-region DR = Active Geo-Replication or Auto-Failover Groups.
- No dedicated VNet — Private Endpoint or VNet Service Endpoints instead.

Chapter 14 · Azure SQL Managed Instance (MI)

14.1 Concept Overview & Tier Architecture

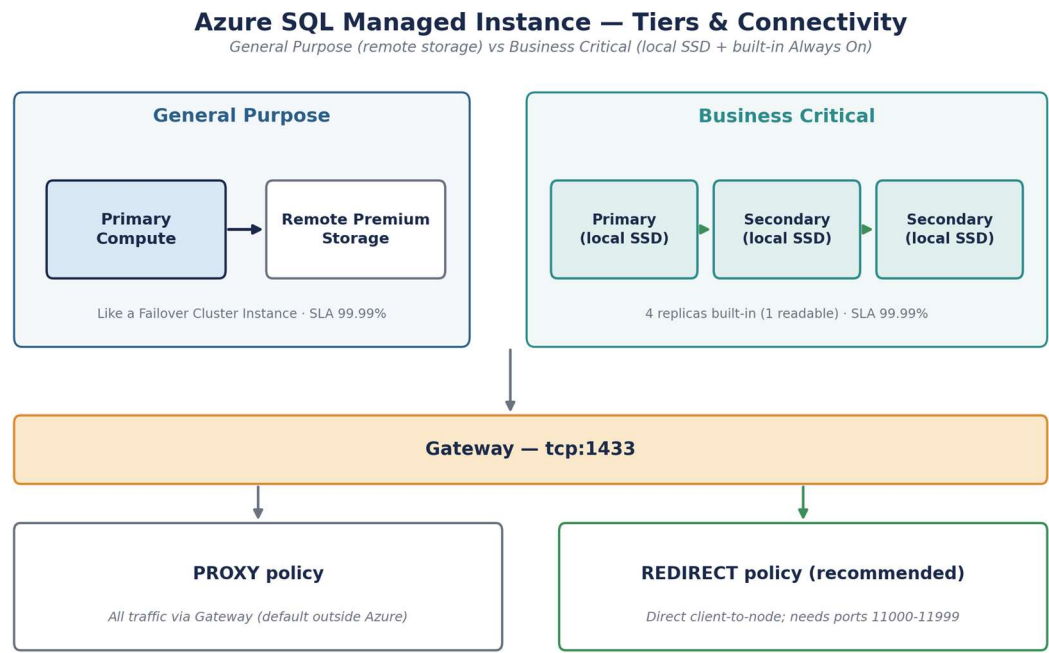


Figure 14.1 — Managed Instance network model: GP vs. BC service tiers, and the Proxy/Redirect/Public connectivity paths.

Aspect	General Purpose (GP)	Business Critical (BC)
Storage	Remote (Azure Premium Storage)	Local SSD attached to the compute node
Architecture	Similar to a failover cluster instance over remote storage	4 replicas built in (1 primary + 3 secondary), an Always On AG under the hood
Built-in HA	Yes (platform-managed)	Yes, plus a readable secondary available via ApplicationIntent=ReadOnly
Typical fit	Most general workloads, cost-sensitive	I/O-intensive, low-latency workloads needing local storage performance

14.2 Connectivity: Proxy vs. Redirect, Endpoints & Ports

Connectivity path	Port(s)	Behavior
Proxy (default for connections originating outside Azure)	1433 only	All traffic flows through the Azure Gateway for the life of the connection

Redirect (Microsoft's recommendation; default from inside Azure)	11000-11999 in addition to 1433	Initial handshake hits the Gateway, then the client connects directly to the node — lower latency, higher throughput, especially valuable for chatty applications
Public Endpoint	3342 (management), 3343 (data)	Internet-reachable, secured via firewall rules; needed for tools/agents that can't reach the private VNet

PRODUCTION SCENARIO — “Unable to connect to SQL Server tcp:localhost,11000” on a SQL Agent job step

This is a connection-policy mismatch: the job step (or its driver) is trying to use Redirect (ports 11000-11999) but those ports are not open on the path it's actually taking. Check the Managed Instance connection policy setting, confirm the NSG/firewall path actually allows 11000-11999 end to end for connections originating outside Azure, and confirm the client/driver is configured consistently with whichever policy (Proxy or Redirect) is actually enforceable on that network path.

- Other ways to connect to MI: Public Endpoint, VNet Endpoint (private, from within the same/peered VNet), and Private Endpoint.
- MI supports a custom alias in the form <dnsaliasname>.<domainname.com>, instead of always using the default <MIName>.<region>.database.windows.net.

14.3 HA/DR for Managed Instance

- High availability is built in by default — no separate configuration required. On Business Critical, the 3 secondary replicas (beyond primary) include one usable for read-only reporting via ApplicationIntent=ReadOnly.
- Disaster recovery across regions uses Auto-Failover Groups, not a traditional Always On AG you configure yourself.
- A Business Critical MI's secondary replica deploys within the same region as the primary by default (not automatically cross-region) — cross-region protection requires explicitly configuring an Auto-Failover Group to a paired/target region. Zone redundancy protects against a datacenter-level failure inside the region, since each Azure region with zone support offers 3 Availability Zones.
- Traditional Always On AG (the kind you build and manage yourself) is not something you configure manually on MI — the platform's built-in HA and Auto-Failover Groups are the supported mechanisms instead.

14.4 Backup & Restore Specifics on MI

Capability	MI behavior
Native backup statements	Supported, but restricted: copy-only full backups and log backups only — no normal full or differential backups can be taken manually
Automated backups	Microsoft manages the platform's own automated backup chain for point-in-time restore, independent of any manual copy-only backups you take

Restore history tracking	Not recorded/exposed the way on-prem restore history is; track an in-progress restore's status via <code>sys.dm_operation_status</code> instead
Cross-version restore	Restoring a native MI backup down to a lower-version standalone SQL Server is not supported (e.g. you cannot restore an MI backup onto SQL Server 2019 on a VM) — the supported path back out of MI to an older or different target is BACPAC/DACPAC export, import/export wizard, or replication; restoring an MI backup onto SQL Server 2022 is the one same/forward-version exception
Daily Dev refresh from Prod (MI → MI)	Options are limited compared to on-prem → MI: take a copy-only backup to Blob Storage from Prod MI, then drop the Dev database and restore from that Blob backup — MI does not allow overwriting an existing database in place via restore

14.5 TempDB, Memory & Replication on MI

- Default TempDB layout: 12 data files + 1 log file (configurable) — see Chapter 10.4.
- Memory per vCore (approximate, varies by exact SKU generation): Standard series ~5.1 GB/vCore; Premium series ~7 GB/vCore; Premium series, memory-optimized ~13.6 GB/vCore.
- Replication support is limited to Snapshot and uni/bi-directional Transactional replication — no merge replication — and MI cannot blend roles across the on-prem/cloud boundary (publisher and distributor must both be local to MI, or both remote; see Chapter 7.5).

14.6 Migration-Adjacent MI Features

Managed Instance Link (Chapter 5.4) and Log Replay Service (Chapter 17) are the two flagship MI migration technologies; both are covered in depth in their dedicated chapters and are summarized again in the migration decision tree there.

14.7 Interview Questions

Q9. Can you restore a Managed Instance native backup onto a standalone SQL Server 2019 VM?

A: No — restoring an MI backup down to an older/lower target version is unsupported; SQL Server 2022 is the one supported forward-compatible exception. For 2019 or other targets, use BACPAC/DACPAC export, the import/export wizard, or replication instead.

Q10. Can rollback transaction details be tracked through Query Store on Managed Instance?

A: No — Query Store does not expose rollback-transaction-level detail; for that you'd look at transaction log analysis tooling or, going forward, the kind of versioned recovery insight that Accelerated Database Recovery's persisted version store provides (Chapter 9.6), not Query Store itself.

Q11. Where does the secondary replica of a Business Critical MI deploy by default?

A: In the same region as the primary, across Availability Zones if zone redundancy is enabled — not automatically in another region. Cross-region protection requires explicitly configuring an Auto-Failover Group.

Q12. What ports does the Redirect connection policy require, and why does Microsoft recommend it?

A: Ports 11000-11999 in addition to 1433; Redirect lets the client connect directly to the hosting node after the initial Gateway handshake, reducing latency and improving throughput compared to Proxy, which keeps every packet flowing through the Gateway for the life of the connection.

Q13. How would you achieve a daily refresh of a Dev database in MI from a Prod database also in MI?

A: Take a copy-only full backup of the Prod database to Blob Storage, then drop the existing Dev database and restore it from that Blob backup — MI does not support overwriting a database in place via restore, so the drop-then-restore pattern is required for MI-to-MI refreshes (unlike on-prem-to-MI, which has more flexible options).

Q14. Does Managed Instance track restore history the way an on-premises instance does?

A: No — there is no persisted restore-history view; monitor an in-progress restore's status via `sys.dm_operation_status` instead.

Q15. What is the approximate memory you get per vCore on each Managed Instance series?

A: Roughly 5.1 GB/vCore on Standard series, 7 GB/vCore on Premium series, and 13.6 GB/vCore on Premium series memory-optimized — since MI has no OS-level memory configuration knob, the vCore tier/series you choose is what determines available memory.

Q16. A SQL Agent job step intermittently fails with 'Unable to connect to SQL Server tcp:localhost,11000'. What's your troubleshooting path?

A: This points to the connection policy: confirm whether the instance is using Proxy or Redirect, confirm that ports 11000-11999 are genuinely reachable end-to-end if Redirect is in effect (the source of the connection matters — anything originating outside Azure defaults to Proxy unless explicitly configured otherwise), and align the client/driver configuration with whichever policy the network path actually supports.

Q17. Can you create a custom DNS alias for a Managed Instance instead of using its default name?

A: Yes — an alias in the form `<dnsaliasname>.<domainname.com>` can be configured instead of always using `<MIName>.<region>.database.windows.net`.

14.8 Quick Revision Notes

- GP = remote storage, FCI-like; BC = local SSD, 4 built-in replicas, 1 usable read-only.
- Proxy = 1433 only, via Gateway always. Redirect (recommended) = 11000-11999, direct after handshake.
- MI native backups: copy-only full + log only — never differential, never normal full.
- MI → lower version restore unsupported except MI → SQL Server 2022.
- No restore history view — use `sys.dm_operation_status`.
- Default TempDB: 12 data files + 1 log file. RAM/vCore: ~5.1 / 7 / 13.6 GB across Standard/Premium/Premium-memory-optimized.
- BC secondary stays same-region by default; cross-region DR needs an explicit Auto-Failover Group.

Chapter 15 • SQL Server on Azure Virtual Machines (IaaS)

15.1 Concept Overview

SQL Server on an Azure VM is full IaaS — you own the OS, patching, and every classic on-premises HA/DR/backup decision, but on Azure infrastructure. The trade-off versus PaaS is more control and full feature parity with on-premises SQL Server, at the cost of carrying the full operational burden the PaaS tiers absorb for you.

15.2 Disk Host Caching — the One Setting That Silently Hurts Performance

Disk	Correct host caching setting	Why
Data file disk(s)	Read-Only	Caches reads (which benefit from caching) without risking cached writes being lost on a host-level event
Log file disk	None	Log writes must be durably committed for transactional correctness — any caching (Read-Only or Read/Write) risks acknowledging a write that hasn't actually been hardened, threatening data integrity on a host failure

Read/Write host caching should never be used for either data or log disks on a production SQL Server VM — it is the one combination that risks silent data loss if the host experiences an unexpected failure.

15.3 Backups on IaaS

Because IaaS gives you a real OS, full VM-level backups are possible (and commonly used) here in a way that simply doesn't exist for PaaS, which has no OS layer to snapshot. Options span native SQL Server backups (to local disk, Azure Blob via BACKUP ... TO URL, or a backup tool), Azure Backup for SQL Server on Azure VM (an Azure-native, SQL-aware backup service integrated with the SQL Server IaaS Agent extension), and storage-level VM snapshots for full-VM disaster recovery.

15.4 Sizing & Real-World Lessons

See Chapter 10.4 for the detailed TempDB placement case study (E32asv4 → E32bds5, T: drive → D: drive, 32 → 12 files) — the same logic applies to general VM sizing decisions for SQL Server IaaS:

- Pick a VM series with adequate storage throughput per vCPU for the actual workload, not just enough vCPU/RAM on paper — storage-optimized series (e.g. the Esv5/Easv5/Edsv5/Ebds5 families) consistently outperform general-purpose series for I/O-heavy SQL Server workloads at the same core count.
- Use Premium SSD or Ultra Disk for data/log volumes in production; pin VMs to Availability Zones (or at minimum an Availability Set) for resilience against datacenter-level failure; enable Accelerated Networking for lower-latency, higher-throughput networking, particularly relevant for Always On AG replica traffic.

- Licensing: Pay-As-You-Go images bundle SQL Server licensing into the VM's hourly cost; Bring-Your-Own-License (BYOL) images let you apply existing Software Assurance-covered licenses, and Azure Hybrid Benefit extends that further by letting on-prem licenses with active SA reduce the Azure VM's effective cost.

15.5 Interview Questions

Q18. What host caching setting should data file and log file disks use on an Azure VM running SQL Server?

A: Data file disks: Read-Only. Log file disks: None. Read/Write caching should never be used on either, since it risks an acknowledged write that hasn't actually been durably hardened if the host fails.

Q19. Can you take full VM-level backups of a SQL Server Azure VM?

A: Yes — this is one of the clear advantages of IaaS over PaaS, since IaaS has a real OS/VM to snapshot; Azure Backup for SQL Server on Azure VM and storage-level VM snapshots are both common approaches, alongside native SQL Server backups.

Q20. A workload migrated from on-premises to an Azure VM is showing repeated 'I/O taking longer than 15ms' warnings. What general categories would you investigate?

A: VM series/SKU storage throughput per vCPU, disk tier (Standard vs. Premium SSD vs. Ultra Disk), TempDB file placement and count, and host caching configuration on each disk — see the detailed production case study in Chapter 10.4 for the exact root causes found in one such migration.

Q21. What's the difference between Pay-As-You-Go, BYOL, and Azure Hybrid Benefit licensing for SQL Server on an Azure VM?

A: PAYG bundles SQL Server licensing into the VM's per-hour cost; BYOL applies your own Software Assurance-covered licenses to an unlicensed image; Azure Hybrid Benefit specifically lets existing on-premises SQL Server licenses with active Software Assurance reduce the effective cost of running SQL Server on the Azure VM.

15.6 Quick Revision Notes

- Data disk host caching = Read-Only. Log disk host caching = None. Never Read/Write on either.
- IaaS = full OS → full VM backup options unavailable on PaaS.
- Storage-optimized VM series almost always beats general-purpose series for SQL Server I/O at the same core count.
- TempDB → local ephemeral disk (commonly D:), not remote persistent storage.

Chapter 16 · Azure Networking for Data Platforms

16.1 Concept Overview

Connectivity design is where Azure SQL Database, Managed Instance, and VM-based SQL Server genuinely diverge, and where enterprise architecture interviews probe hardest — because the “right” answer depends on which of the three deployment models, and which compliance/latency requirements, are in play.

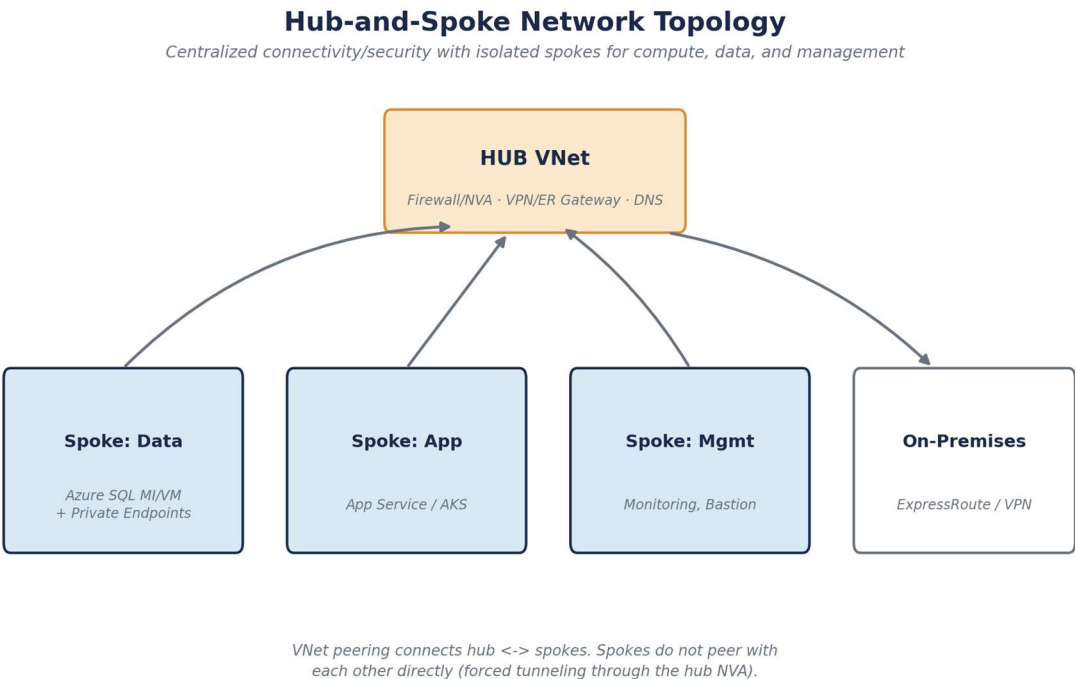


Figure 16.1 — Hub-and-spoke landing zone: MI, IaaS SQL, and app-tier spokes connected through a shared hub, with ExpressRoute/VPN back to on-premises.

16.2 Connectivity Options Compared

Mechanism	Scope	Typical use
Public Endpoint	Internet-reachable, secured by firewall rules/NSGs	Quick access, third-party tooling that can't reach a private network — generally discouraged for production data-plane traffic
Private Endpoint	A private IP inside your VNet, backed by Azure Private Link	The recommended default for Azure SQL Database/MI from app tiers inside Azure or connected via ExpressRoute/VPN
VNet Service Endpoint	Restricts a resource's public endpoint to specific VNets/subnets without giving it a private IP	A lighter-weight alternative to Private Endpoint when a private IP isn't required

VNet Peering	Direct, low-latency connection between two VNets in the same or different regions	Connecting spokes to a hub, or two regional landing zones to each other
VNet Gateway (VPN/ExpressRoute)	Connects a VNet to on-premises or to another cloud	Hybrid connectivity back to a corporate datacenter that still hosts Always On DR replicas, etc.

Two different networks within Azure are connected via VNet Peering (regional/low-latency, same or cross-region) or via VNet Gateways (typically for hybrid on-prem connectivity, though gateways can also connect VNets); peering is generally preferred within Azure for its lower latency and lack of a gateway bandwidth bottleneck.

16.3 Hub-and-Spoke Landing Zone Pattern

- Hub VNet centralizes shared services: Azure Firewall or a network virtual appliance, the VPN/ExpressRoute gateway, and Private DNS zones used by every spoke.
- Spokes host workload-specific resources — a Managed Instance subnet, an IaaS SQL Server VM subnet (often pinned to specific Availability Zones), and an application/AKS tier connecting to Azure SQL Database via Private Endpoint — each spoke peered back to the hub rather than directly to each other.
- On-premises connectivity (e.g. an Always On secondary or DR replica still living on-prem during a phased migration) comes into the hub via ExpressRoute (dedicated, private, higher-bandwidth, SLA-backed) or Site-to-Site VPN (cheaper, internet-routed, lower guaranteed bandwidth) — ExpressRoute is the enterprise-grade default once volume/latency requirements justify the cost.

16.4 Port & Endpoint Cheat-Sheet

Service	Port(s)	Notes
SQL Server default instance	1433/TCP	Named instances negotiate dynamically via the SQL Server Browser service (UDP 1434) unless a static port is configured
MI Redirect connection policy	11000-11999/TCP	In addition to 1433; required end-to-end for the recommended Redirect policy to function
MI Public Endpoint	3342 (management) / 3343 (data)	Internet-reachable management/data ports specifically for the MI public endpoint
Always On / WSFC cluster communication	Varies (often 3343 internally + dynamic RPC ranges)	Confirm with current cluster validation report; do not assume a single fixed port across all cluster traffic

16.5 Interview Questions

Q22. Is there a dedicated VNet for Azure SQL Database the way there is for a VM or Managed Instance?

A: No — Azure SQL Database has no dedicated VNet of its own; secure connectivity uses Private Endpoint (a private IP inside your VNet via Private Link) or VNet Service Endpoints instead.

Q23. What are the two main ways to connect two different networks within Azure?

A: VNet Peering (direct, low-latency, same or cross-region) and VNet Gateways (commonly used for hybrid on-premises/cross-cloud connectivity, though gateways can also link VNets).

Q24. What port does Managed Instance's Public Endpoint use for data traffic versus management traffic?

A: 3343 for data, 3342 for management.

Q25. Design the network connectivity for a new Managed Instance, an IaaS SQL Server VM, and an AKS application tier that all need to reach SQL data, while a phased migration still keeps an Always On DR replica on-premises for six months.

A: Adopt a hub-and-spoke landing zone: a hub VNet with the firewall/NVA, ExpressRoute (or VPN) gateway, and Private DNS zones for *.database.windows.net and the MI private DNS zone.

Spoke 1: Managed Instance in its own delegated subnet, reached privately from other spokes via VNet peering back through the hub.

Spoke 2: the IaaS SQL Server VM, pinned to Availability Zones, with Accelerated Networking enabled and NSG rules scoped tightly to the app tier's subnet.

Spoke 3: the AKS application tier, reaching Azure SQL Database (if also in scope) via Private Endpoint rather than the public endpoint.

On-premises DR replica: ExpressRoute from the hub back to the datacenter for the duration of the phased migration, decommissioned once the on-prem replica is retired post-cutover.

16.6 Quick Revision Notes

- Azure SQL DB: no dedicated VNet — Private Endpoint or Service Endpoints only.
- MI: Public Endpoint (3342/3343), VNet Endpoint, Private Endpoint, plus Proxy(1433)/Redirect(11000-11999) connection policy.
- VNet Peering = intra-Azure; VNet Gateway = hybrid/on-prem (or cross-VNet too).
- Hub-and-spoke: shared firewall/gateway/DNS in the hub; workload-specific subnets in spokes.
- ExpressRoute = dedicated/SLA-backed; Site-to-Site VPN = cheaper/internet-routed.

Part VI — Migration & Modernization

Migration questions test whether you can match the right technology to a real constraint set — source version, downtime budget, database count, and target platform — rather than reciting a single favorite tool. This part builds that decision framework, then brings the handbook current with SQL Server 2025/2026 and Microsoft Fabric, the two developments most likely to appear in a 2026 interview that an older study guide would miss entirely.

Chapter 17 · On-Premises ↔ Azure Migration Strategies

17.1 Concept Overview

There is no single “best” migration tool — the right choice is a function of source version, target platform, database count, and the downtime budget the business will actually accept. Interviewers increasingly present this as a scenario (“given X source version, Y database count, and Z downtime tolerance, what do you use?”) rather than asking you to define each tool in isolation.

On-Premises -> Azure Migration: Decision Path

Choosing a migration technology by source version, target platform, and downtime tolerance

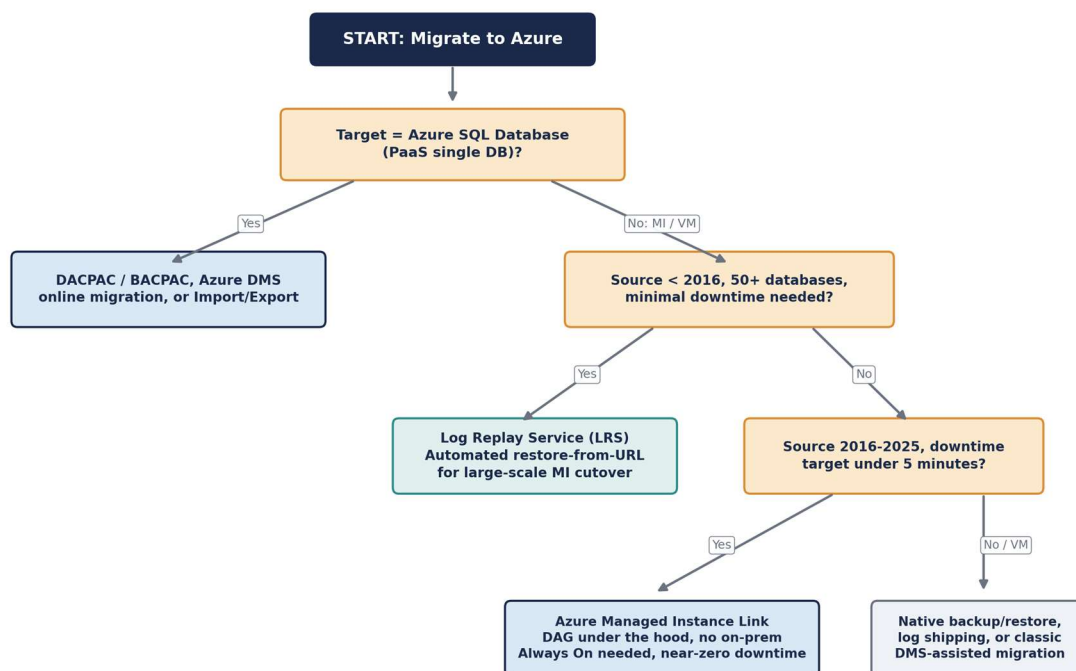


Figure 17.1 — On-premises to Azure migration decision tree, branching on source version, database count, and downtime budget.

17.2 Migration Toolkit at a Glance

Tool / Technique	Best for	Key constraint
Log Replay Service (LRS)	Source < SQL Server 2016, large database counts (50+), automated/repeatable migration to MI with minimal downtime	Built on backups to URL; restores apply WITH NORECOVERY in sequence, then a final cutover restore WITH RECOVERY
Azure Managed Instance Link	Source SQL Server 2016 (latest CU)/2019/2022, downtime budget under ~5 minutes, want a live readable target ahead of cutover	Builds a Distributed AG behind the scenes; on-prem side needs no Always On configured at all, even standalone qualifies

Distributed Availability Group (manual)	Standalone-to-standalone or cluster-to-cluster moves, cross-version upgrades, sub-minute cutover	Available cluster-less from SQL Server 2017+; automatic seeding requires matching source/target build
BACPAC / DACPAC	Schema-only or schema+data moves, especially toward Azure SQL Database, smaller databases	BACPAC carries data and is slower/larger; DACPAC is schema-only; not ideal for very large databases due to export/import time
Native backup/restore (incl. BACKUP/RESTORE ... TO/FROM URL)	MI (with the copy-only/log restrictions in Chapter 14.4), VM/laaS targets, any same-or-newer-version on-prem move	RESTORE ... FROM URL implicitly adds WITH RECOVERY — no further log apply afterward, which is exactly why traditional log shipping cannot be configured to/from MI
Azure Database Migration Service (DMS)	Assessment plus migration at scale, especially for login migration to MI and heterogeneous source migrations	Recommended explicitly for transferring SQL logins to Managed Instance alongside sp_help_revlogin-generated scripts
Transactional Replication / Import-Export wizard	Selective table-level moves, or as a fallback when a direct restore path isn't supported (e.g. MI backup down to an unsupported lower target version)	Replication has its own topology rules (Chapter 7.5); the wizard works table-by-table provided firewall ports are open

17.3 Why Traditional Log Shipping Doesn't Work To/From Managed Instance

MI requires backups to be staged via Backup to URL (Azure Blob Storage). RESTORE ... FROM URL implicitly applies WITH RECOVERY, which closes the database to any further log restores — exactly the mechanism traditional log shipping depends on (repeated WITH NORECOVERY/STANDBY restores). This is precisely why Log Replay Service exists: it is purpose-built automation that works around this restriction by managing the restore sequence and the final cutover restore for you, rather than relying on the manual log-shipping restore chain.

17.4 Post-Migration Performance Checklist (On-Prem → Managed Instance)

This is one of the most common scenario questions in the chapter — “we migrated and performance got worse, what do you check?” — and the checklist below is the complete expected answer (see also Chapter 14):

25. Recovery model — MI supports FULL recovery only; if the source relied on SIMPLE or BULK_LOGGED for minimally-logged bulk operations, those operations now generate full logging, increasing log volume and backup time.
26. TDE — enabled automatically on every MI database regardless of source configuration, adding CPU overhead the source may not have had.
27. Storage tier mismatch — General Purpose uses remote storage; if the source ran on local SSD or a high-performance SAN, GP may simply not match that I/O profile, and Business Critical (local SSD) may be the correct target tier instead.

28. Engine/database settings drift — compatibility level, MAXDOP, cost threshold for parallelism, and database-scoped configurations (LEGACY_CARDINALITY_ESTIMATION, PARAMETER_SNIFFING, QUERY_OPTIMIZER_HOTFIXES) and database options (AUTO_UPDATE_STATISTICS, DELAYED_DURABILITY) should all be compared explicitly between source and target rather than assumed to carry over.
29. Backup cadence — MI automatically runs log backups roughly every 5-10 minutes by default, a cadence the on-premises environment may not have matched; factor this into log-volume and I/O expectations.
30. Transport encryption overhead — MI enforces SSL/TLS transport encryption always; if the on-premises environment didn't enforce encryption, the additional CPU/network overhead on MI is a real, measurable difference worth validating under load.

17.5 Interview Questions — Beginner

Q1. What technology would you use to migrate 50+ databases from a pre-2016 SQL Server source to Managed Instance with minimal downtime?

A: Log Replay Service (LRS) — purpose-built automation for exactly this combination of older source version, large database count, and a minimal-downtime requirement.

Q2. What technology would you use to migrate a SQL Server 2019 source to Managed Instance with under 5 minutes of downtime?

A: Azure Managed Instance Link — it builds a Distributed Availability Group behind the scenes and requires no Always On configuration on the on-premises side at all.

Q3. Why can't you configure traditional log shipping to or from Managed Instance?

A: Because MI restores must go through Backup to URL, and RESTORE ... FROM URL implicitly applies WITH RECOVERY, leaving no opportunity for further log restores afterward — exactly the repeated-restore mechanism log shipping depends on.

Q4. What's the difference between a BACPAC and a DACPAC?

A: A BACPAC contains schema and data; a DACPAC contains schema only.

17.6 Interview Questions — Intermediate & Advanced

Q5. Can you restore a backup taken from Managed Instance onto a standalone SQL Server 2019 VM?

A: No — restoring an MI backup to a lower-version target is unsupported (SQL Server 2022 is the one forward-compatible exception); for 2019 or other targets, fall back to BACPAC/DACPAC export, the import/export wizard, or replication.

Q6. A team needs to migrate 80 databases from SQL Server 2014 to Managed Instance with minimal downtime, and another team needs to migrate a single 2019 database with under 5 minutes downtime. Are these the same tool?

A: No — the 80-database, pre-2016 scenario is the textbook Log Replay Service case (automated, repeatable, scales across many databases); the single 2019 database with a tight downtime window is the textbook Managed Instance Link case (Distributed AG-based, near-instant cutover, no Always On prerequisite on-prem). Matching the tool to source version + database count + downtime budget, rather than defaulting to one favorite tool, is exactly what this question is testing.

Q7. After migrating from on-premises to Managed Instance, query performance has degraded. Walk through your full diagnostic checklist.

A: Compare recovery models — MI is FULL-only; minimally-logged bulk operations under the old SIMPLE/BULK_LOGGED model now generate full logging.

Check whether TDE's automatic enablement on MI is adding measurable CPU overhead the source never had.

Verify the storage tier (GP remote vs. BC local SSD) actually matches the source's I/O profile.

Diff compatibility level, MAXDOP, cost threshold for parallelism, and database-scoped configurations between source and target.

Account for MI's automatic ~5-10 minute log backup cadence and its always-on SSL/TLS transport encryption overhead, both of which may be new relative to the source environment.

Q8. What is the recommended way to migrate SQL logins to Managed Instance at scale, and why doesn't it work the same way for Azure SQL Database?

A: For Managed Instance, sp_help_revlogin-generated scripts (which preserve SIDs) work, and Azure Database Migration Service is the recommended way to automate this across many logins. Azure SQL Database has no equivalent supported mechanism for carrying over SQL logins with existing passwords — recreate logins with new passwords there, or move to Entra ID-based authentication instead (Chapter 12.5).

17.7 Quick Revision Notes

- Source < 2016 + many DBs + minimal downtime → Log Replay Service.
- Source 2016+/2019/2022 + downtime budget under ~5 minutes → Managed Instance Link (DAG-based).
- Standalone-to-standalone or cross-version, sub-minute cutover → manual Distributed AG (2017+).
- MI restore FROM URL = implicit WITH RECOVERY → no traditional log shipping to/from MI; LRS exists specifically to work around this.
- Post-migration perf regression checklist: recovery model, TDE overhead, storage tier mismatch, config drift, log-backup cadence, TLS overhead.
- Login migration: MI → sp_help_revlogin + Azure DMS; Azure SQL DB → recreate or move to Entra ID.

Chapter 18 • SQL Server 2025/2026 & Microsoft Fabric for DBAs

18.1 SQL Server 2025 — Release Overview

SQL Server 2025 reached general availability on November 18-19, 2025, and is positioned by Microsoft as its most significant release for developers in a decade, alongside the most substantial security upgrade to the on-premises engine in several releases (Chapter 11.6). The headline themes a 2026 interview expects you to know:

Area	What's new in SQL Server 2025
AI-ready engine	Native vector data type and vector indexes for similarity search directly inside the relational engine; built-in REST API support via a system stored procedure, enabling AI-agent-style integration without an external API layer
Developer productivity	Native JSON data type (up to 2 GB per row) with a dedicated JSON index; built-in Regular Expression (RegEx) support in T-SQL; Change Event Streaming directly to Azure Event Hubs (a lighter-weight alternative to CDC for event-driven architectures); SSMS 22 (ARM64 support, dark mode, GitHub Copilot integration); the new mssql-python driver with Entra ID auth support
Performance & availability	Optimized Locking (avoids classic lock-escalation paths, no code changes required); TempDB resource governance/sandboxing; Query Store support on readable secondaries; columnstore index maintenance improvements; resiliency and diagnostic improvements for Always On Failover Groups
Security	Credential-free Managed Identity authentication outbound to Azure services; PBKDF2 password hashing; TDS 8.0 over TLS 1.3 enforced by default, including internal AG/replication/Agent traffic; deeper Microsoft Entra ID/Zero Trust integration (full detail in Chapter 11.6)
Editions & licensing	Standard edition resource limits raised to 32 cores / 256 GB memory; Express Advanced features now included by default; Web edition discontinued starting with 2025 (SQL Server 2022 remains the last version to include it, supported through January 2033)
Fabric integration	Native mirroring (change-feed based) from SQL Server 2025 directly into Microsoft Fabric OneLake — see Section 18.2

A practical interview note: many SQL Server 2025 features ship as opt-in via database-scoped configuration first (vector indexes being a notable example), with Microsoft's stated goal of moving preview features to fully general availability within roughly 12 months of the initial release — worth mentioning if asked whether a specific 2025 feature is “ready for production” on day one.

18.2 Microsoft Fabric Mirroring — What a DBA Actually Needs to Know

SQL Server 2025 Mirroring to Microsoft Fabric

Zero-ETL, near real-time change feed into OneLake Delta Parquet

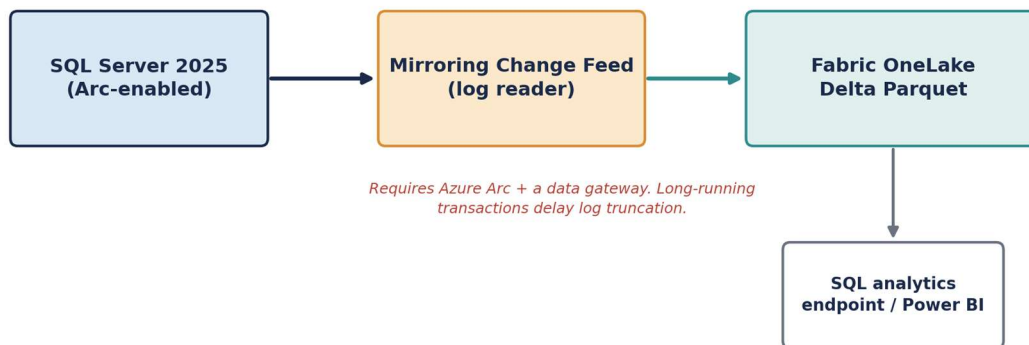


Figure 18.1 — SQL Server 2025 change-feed mirroring into Microsoft Fabric OneLake, surfaced through a read-only SQL analytics endpoint.

Fabric Mirroring is a zero-ETL, near-real-time replication mechanism into Microsoft Fabric's OneLake, storing data as open Delta Parquet tables and exposing them through an auto-generated, read-only SQL analytics endpoint — eliminating the custom pipelines/connectors a data engineering team would otherwise build to get operational data into a lakehouse.

- SQL Server 2025 mirroring is change-feed based: it reads committed transaction log changes and streams them to OneLake. It requires the source instance to be Azure Arc-enabled (the Azure Extension for SQL Server) and an on-premises (or VNet) data gateway to broker the secure connection; it is not currently supported for SQL Server on Linux.
- Azure SQL Database and Fabric SQL Database mirroring use a different underlying mechanism — Change Data Capture (CDC) on the source — rather than SQL Server 2025's change-feed approach; the end result (a live Delta Parquet mirror in OneLake) looks the same to a consumer, but the source-side mechanics differ, which matters when troubleshooting replication lag.
- Operationally, the one DBA-relevant caution: active, long-running transactions on the mirrored source hold up transaction log truncation until they commit (or the mirror catches up) — a long-running transaction can therefore cause unusually large log growth on a mirrored database, the same general pattern as a slow Always On secondary, just with a different downstream consumer.
- Mirrored tables are strictly read-only in Fabric — no INSERT/UPDATE/DELETE is possible against the mirror, which removes any risk of analytics workloads accidentally writing back into what is conceptually still “your” operational data.
- For a DBA, the practical payoff is an analytics-offload option that doesn't require standing up and maintaining a dedicated Always On reporting secondary purely for BI/analytics workloads — though an AG secondary remains the right tool when the consumer needs row-level, transactional, low-latency reads rather than analytical/BI-style querying.

18.3 Where Fabric Fits Next to Always On (Not Instead Of)

Need	Better fit
Sub-second, transactional, row-level reads for an application (e.g. a read replica for an OLTP app)	Always On AG readable secondary
Near-real-time analytics, BI dashboards, cross-source joins with other Fabric data, data science notebooks	Fabric Mirroring into OneLake
Both, simultaneously	Run them together — an AG secondary for application reporting, with the primary (or the AG itself) also mirrored into Fabric for analytics — they are complementary, not competing, technologies

18.4 Interview Questions

Q9. When did SQL Server 2025 become generally available, and what's the single most quoted developer-facing headline feature?

A: General availability was announced in mid-November 2025. The most commonly cited headline is native JSON support (up to 2 GB per row) with a dedicated JSON index, alongside built-in RegEx support in T-SQL — together described by Microsoft as the most significant release for developers in a decade.

Q10. What is Optimized Locking in SQL Server 2025, and why does it matter for the blocking/lock-escalation topics covered earlier in this handbook?

A: It's a database-scoped option that avoids many classic row/page lock-escalation paths without requiring any application code changes, directly addressing one of the most common production blocking root causes covered in Chapter 3.3.

Q11. What does Fabric Mirroring require on the source SQL Server 2025 instance, and what's the one operational risk to monitor?

A: The source instance must be Azure Arc-enabled (via the Azure Extension for SQL Server) and reachable through an on-premises or VNet data gateway. The operational risk to monitor is transaction log growth: active, long-running transactions on the mirrored database hold up log truncation until they commit or the mirror catches up, the same pattern as a lagging Always On secondary.

Q12. Is Fabric Mirroring a replacement for an Always On readable secondary used for reporting?

A: Not a direct replacement — Fabric Mirroring is built for near-real-time analytics/BI consumption against a read-only Delta Parquet copy in OneLake, while an AG secondary is built for transactional, row-level, low-latency application reads. Many architectures run both: an AG secondary for application-style reporting, and Fabric Mirroring (from the primary or an AG member) for BI/data-science consumption.

Q13. A client asks whether they can use SQL Server 2025's vector search feature in production today. How do you answer?

A: Several SQL Server 2025 features, vector indexes included, ship initially as opt-in via a database-scoped configuration rather than being on by default — Microsoft's stated intent is to graduate these to full general availability within roughly 12 months of the initial release based on customer feedback. The honest answer is: it's usable and supported in preview/opt-in form now, but validate current documentation for that specific feature's GA status before committing a production timeline to it.

Q14. How does Azure SQL Database mirroring into Fabric differ mechanically from SQL Server 2025's mirroring?

A: Azure SQL Database (and Fabric SQL Database) mirroring is built on Change Data Capture (CDC) against the source; SQL Server 2025's on-premises/laaS mirroring instead reads the transaction log directly via a change-feed mechanism. The visible result — a live Delta Parquet mirror in OneLake — is the same, but understanding which mechanism is in play matters when you're troubleshooting replication lag on one platform versus the other.

18.5 Quick Revision Notes

- SQL Server 2025 GA: mid-November 2025. Headline: native JSON (2 GB/row + JSON index), RegEx in T-SQL, vector search, REST APIs, Change Event Streaming, SSMS 22 + Copilot.
- Optimized Locking, TempDB sandboxing, Query Store on secondaries = the AG/performance-adjacent 2025 upgrades to name.
- Security: Managed Identity (credential-free), PBKDF2, TDS 8.0/TLS 1.3 default, deeper Entra ID/Zero Trust.
- Fabric Mirroring (SQL Server 2025) = change-feed based, needs Azure Arc + data gateway, read-only Delta Parquet in OneLake, watch log growth on long transactions.
- Azure SQL DB/Fabric SQL DB mirroring = CDC-based (different mechanism, same visible outcome).
- Fabric complements, rather than replaces, an Always On reporting secondary.

Part VII — The Modern DBA Operating Model

A 2026 Senior/Lead DBA interview increasingly asks not just “can you fix this database” but “can you operate a fleet of them like an engineering discipline” — version-controlled, automated, observable, and increasingly AI-assisted. This part covers DevOps and Infrastructure as Code practices for databases, where AI genuinely helps (and where it doesn't), and the monitoring/observability stack that ties everything else in this handbook together.

Chapter 19 • DevOps, Automation & Infrastructure as Code for DBAs

19.1 Concept Overview

Database DevOps treats schema and configuration the same way application DevOps treats code: version-controlled, peer-reviewed, automatically tested, and deployed through a pipeline rather than by a DBA running an ad hoc script against production by hand. The shift doesn't remove the DBA from the process — it moves the DBA's judgment earlier (reviewing the migration script in a pull request) and later (owning rollback/runbook design), instead of at the keyboard during the deployment window.

19.2 Schema Change Management

- State-based deployment (SSDT/DACPAC) — you maintain the desired end-state schema in source control; a deployment tool (sqlpackage, Azure Data Studio, or an Azure DevOps/GitHub Actions task) computes and applies the diff against the target automatically. Strong for catching unintended drift; weaker for complex data-migrating changes that need explicit, ordered steps.
- Migration-based deployment (Flyway, Liquibase, DbUp, or hand-rolled numbered scripts) — every change is an explicit, versioned, ordered script, applied in sequence and tracked in a metadata table on the target. Strong for complex, data-aware changes (backfills, multi-step column splits); requires more discipline to avoid migration-script drift between environments.
- Most mature shops use both: state-based for routine schema-only changes, migration-based for anything that also moves or transforms data.

19.3 CI/CD Pipelines for Databases

31. Build — validate the DACPAC/migration scripts compile/parse cleanly and run static analysis (naming conventions, missing indexes on FKs, banned patterns like `SELECT *`) as an automated gate.
32. Test — deploy to an ephemeral or shared test database and run automated tests (tSQLt for T-SQL unit tests, or integration tests from the application layer) plus a generated deployment script for human review.
33. Approve — a human reviews the generated diff/migration script in the pull request, the same as a code review, paying particular attention to anything that rewrites a large table, drops a column, or changes a data type (the classic `int`→`bigint` scenario from Chapter 1 is exactly the kind of change a review gate should catch and route to the compression-based zero-downtime technique rather than a blind `ALTER`).
34. Deploy — the pipeline applies the change automatically to each environment in sequence (Dev → Test → Staging → Production), ideally during a defined change window for production, with an automatic rollback script generated alongside the forward migration wherever feasible.
35. Verify — post-deployment smoke tests and monitoring (Chapter 21) confirm the change behaved as expected before the pipeline is considered complete.

19.4 Infrastructure as Code for the Data Platform

- ARM templates / Bicep / Terraform provision Azure SQL Database, Managed Instance, Azure VMs, networking (VNets, subnets, Private Endpoints), and Always On/Auto-Failover Group configuration declaratively — the same landing zone described in Chapter 16 should exist as code, not as a one-time portal click-through.

- Idempotency matters specifically for database IaC: re-running a provisioning template should never silently recreate or drop a database that already holds production data — guard destructive resources behind explicit lifecycle policies/locks distinct from the rest of the infrastructure.
- Configuration drift detection — scheduled jobs (or pipeline runs) comparing live instance/database configuration (MAXDOP, cost threshold, trace flags, database-scoped configurations) against the IaC-declared baseline, flagging manual out-of-band changes before they become a silent source of environment-to-environment inconsistency (exactly the kind of drift flagged in the Chapter 17.4 migration checklist).

19.5 dbatools & PowerShell Automation

dbatools is the de facto open-source PowerShell module for SQL Server administration at scale, and it directly answers several “how do you sync X across replicas” questions raised earlier in this handbook without resorting to manual scripting:

dbatools command (representative)	Replaces this manual task
Copy-DbaLogin	Manually scripting and recreating SQL/Windows logins across AG replicas with matching SIDs (Chapter 4.5)
Sync-DbaAvailabilityGroup	Manually reconciling logins, jobs, linked servers, and other instance-level objects that sit outside the AG database's own replication scope (the “synchronize objects outside the database scope” question from Chapter 4 — Contained AG, from 2022+, narrows this further but pre-2022 and cross-cutting objects still need this)
Test-DbaLastBackup	Manually restoring a backup to a scratch instance to validate it is actually restorable, on a schedule, across an entire backup fleet
Invoke-DbaQuery / Install-DbaFirstResponderKit	Distributing and running Brent Ozar's First Responder Kit (sp_Blitz, sp_BlitzIndex, sp_BlitzCache, sp_WhoIsActive) across many instances consistently

19.6 Interview Questions

Q1. What's the difference between state-based and migration-based database deployment?

A: State-based (SSDT/DACPAC) declares the desired end-state schema and lets a tool compute/apply the diff automatically; migration-based (Flyway/Liquibase/DbUp) requires every change as an explicit, ordered, versioned script. State-based suits routine schema-only changes; migration-based suits complex, data-aware changes that need explicit control over sequencing.

Q2. A pull request proposes **ALTER TABLE Orders ALTER COLUMN OrderID BIGINT** against a 1 TB table with no maintenance window. What should the review gate catch, and what should it recommend instead?

A: It should catch that a naive ALTER COLUMN on a huge table will generate enormous transaction log volume and likely an unacceptable lock duration; the recommended alternative is the compression-based technique from Chapter 1 (enable page/row compression on the table and every non-clustered index, then run the ALTER as a near-instant metadata-only operation) — exactly the kind of change a database-aware CI/CD review gate exists to intercept before it reaches production.

Q3. How does dbatools' Sync-DbaAvailabilityGroup help with a classic Always On operational gap?

A: It automates reconciling instance-level objects (logins, jobs, linked servers, and similar) that sit outside a database's own AG replication scope across every replica — the exact manual gap described in Chapter 4.5, and one that Contained Availability Groups (2022+) only partially close for objects that remain outside the database boundary.

Q4. Why is idempotency a sharper concern for database IaC than for stateless infrastructure like a VM or a VNet?

A: Re-running a VM or networking template safely converges to the same desired state with no data-loss risk; re-running a careless database provisioning template can recreate or drop a database that holds irreplaceable production data. Database IaC needs explicit lifecycle guards (resource locks, conditional creation logic, or splitting the database resource itself out of the routinely-reapplied template) that purely stateless infrastructure doesn't require.

19.7 Quick Revision Notes

- State-based = diff-and-apply (DACPAC); migration-based = explicit ordered scripts (Flyway/Liquibase). Most shops use both.
- CI/CD gate order: build → test → human review of the generated diff → deploy → verify.
- IaC for the data platform = the Chapter 16 landing zone as code, with destructive-resource guards.
- dbatools closes the manual-sync gaps (logins, AG cross-cutting objects, backup-restore testing) raised throughout Part II.

Chapter 20 • AI for Database Administration

20.1 Concept Overview

AI tooling has moved from a novelty to a genuine part of the DBA toolchain in the SSMS 22 / SQL Server 2025 era — but the discriminating interview answer is knowing precisely where it adds leverage (drafting, summarizing, suggesting) versus where it must never be the final decision-maker (production change approval, security posture, data-loss-risk operations).

20.2 Where AI Genuinely Helps a DBA Today

Tool	What it does for a DBA
GitHub Copilot in SSMS 22	Inline T-SQL drafting/explanation, suggesting fixes for error messages, and accelerating routine script-writing inside the same SSMS session the DBA already lives in
Copilot in Azure Data Studio / VS Code mssql extension	Similar inline assistance for DBAs and developers working against SQL Server/Azure SQL from a code-editor-first workflow
Azure SQL Database Automatic Tuning	Automatically detects and (optionally) applies plan-regression fixes (force last good plan) and index recommendations, with the platform monitoring the actual impact and rolling back automatically if a change doesn't help
Query Store + AI-assisted regression detection	Surfaces and explains plan regressions over time (“this query got 5x slower after Tuesday’s deployment”) far faster than manually diffing plan XML
Natural-language-to-SQL (Copilot in Fabric, Copilot in Microsoft 365 over Power BI/Fabric semantic models)	Lets business users self-serve simple analytical questions without writing T-SQL directly, reducing ad hoc query load against operational systems when properly routed to a reporting/Fabric layer instead
AI-assisted anomaly detection (Azure Monitor dynamic thresholds, Log Analytics)	Flags abnormal CPU/IO/wait-stat patterns against a learned baseline rather than a single static threshold, reducing both false positives and missed slow-building incidents

20.3 Vector Search & Retrieval-Augmented Generation (RAG) Inside SQL Server

SQL Server 2025's native vector data type and vector indexes (Chapter 18.1) let a DBA support a RAG application's retrieval step directly inside the relational engine — storing embeddings alongside the operational data they describe, rather than standing up a separate, dedicated vector database purely for similarity search. For a DBA, this is best understood as a new workload pattern to plan capacity and indexing strategy for, not a replacement for any existing HA/DR/backup discipline covered elsewhere in this handbook — a vector-enabled database still needs the same AG, backup, and patching rigor as any other production database.

20.4 Where AI Should Not Be the Final Decision-Maker

- Production change approval — a generated migration script, however clean it looks, still needs the human review gate from Chapter 19.3; AI-assisted drafting speeds up writing the script, not the judgment call on whether/when to run it.
- Security and access decisions — granting permissions, approving Entra ID Conditional Access policy changes, or deciding TRUSTWORTHY/ownership-chaining exceptions (Chapter 12.4) remain explicit human security decisions, not something to delegate to a suggestion engine.
- Data exposure to external AI services — sensitive data (PHI, PII, regulated financial data) should never be pasted into a general-purpose AI chat tool outside your organization's governed AI tenancy/Conditional Access boundary; Copilot integrations built into Microsoft's own data tooling are designed with this boundary in mind, but ad hoc use of an ungoverned external AI tool with production data is a compliance risk in its own right.
- Root cause attribution — an AI summary of “what probably happened” during an incident is a starting hypothesis to investigate with the DMVs/Extended Events/Query Store evidence covered in Chapter 21, not a substitute for confirming root cause against that evidence before writing the post-incident report.

20.5 Interview Questions

Q5. How does Azure SQL Database's Automatic Tuning decide whether to keep an AI/automatically-applied change?

A: It monitors the actual performance impact of an automatically applied fix (e.g. forcing the last known good plan, or adding a recommended index) after the change, and automatically reverts it if the change doesn't measurably improve — or actively harms — performance, rather than applying changes blindly and leaving them in place regardless of outcome.

Q6. A junior DBA wants to paste a slow query plan, including real customer data in the sample rows, into a general-purpose public AI chatbot to get tuning advice. What's the concern, and what should they do instead?

A: Pasting real customer/PHI/PII data into an ungoverned external AI tool is a data-exposure and likely compliance risk (HIPAA/GDPR), independent of how helpful the resulting advice might be. The safer path is to either strip/mask the actual data values from the plan before sharing (keeping only structure, row counts, and operator costs), or use the organization's own governed AI tooling (e.g. Copilot integrated into SSMS/Fabric under the company's Entra ID tenancy) where data handling is covered by the organization's own agreements and Conditional Access boundary.

Q7. What's the right way to think about SQL Server 2025's vector search feature from a DBA capacity-planning perspective?

A: Treat it as a new workload pattern, not a separate system to ignore — vector indexes consume memory/storage and have their own maintenance characteristics, so capacity planning, backup strategy, and AG/DR coverage need to account for vector-enabled tables exactly as they would for any other production workload, rather than assuming an AI-flavored feature is somehow exempt from standard DBA discipline.

Q8. Where would you trust an AI-generated explanation of an incident, and where would you insist on independent verification?

A: Trust it as a fast first hypothesis pointing you toward which DMVs/Extended Events/Query Store evidence to pull first — it can meaningfully shorten the triage phase. Insist on independent verification against that actual evidence before it goes into a post-incident report or drives a permanent configuration

change, since a plausible-sounding but unverified explanation is exactly the kind of error that undermines trust in a root-cause-analysis process.

20.6 Quick Revision Notes

- Copilot in SSMS 22/VS Code/Fabric = drafting and explanation accelerator, not a decision-maker.
- Azure SQL Automatic Tuning self-validates and auto-reverts changes that don't help.
- Vector search (SQL Server 2025) = a new workload pattern requiring the same capacity/HA/DR discipline as any other table.
- Never paste regulated/sensitive data into ungoverned external AI tools — use the organization's governed tooling instead.
- AI-generated root cause = a hypothesis to verify against DMV/XEvents/Query Store evidence, never the final word.

Chapter 21 • Monitoring, Observability & Alerting

21.1 Concept Overview

Monitoring tells you something is wrong; observability lets you figure out why without having been watching live when it happened. This chapter consolidates the DMV/Extended Events foundation referenced throughout the handbook into a single monitoring strategy, and extends it to the Azure-native tooling needed for PaaS targets that don't expose a Windows Perfmon counter at all.

21.2 The On-Premises/IaaS Monitoring Stack

Layer	Tooling	What it answers
OS/hardware	Windows Perfmon counters (Disk Queue Length, Avg Disk sec/Read-Write, % Processor Time, Available MBytes)	Is the underlying hardware the bottleneck, independent of SQL Server's own view of itself?
Instance-wide, point-in-time	sys.dm_os_wait_stats, sys.dm_os_performance_counters, sys.dm_exec_requests/sessions, sp_WhoIsActive	What is the instance collectively waiting on right now, and who/what is the current top consumer?
Historical, built-in, no setup required	sys.dm_os_ring_buffers (RING_BUFFER_SCHEDULER_MONITOR), Default Trace, system_health Extended Events session	What happened in roughly the last few hours, even if nobody was watching live — the ring buffer specifically retains about 4 hours of 1-minute CPU utilization snapshots, split between SQL Server and other processes, and is the answer to 'was sqlservr.exe really the cause of that CPU spike an hour ago' when Perfmon wasn't running
Query-level, over time	Query Store (per-database, retains plan/runtime/wait stats history with built-in regression-hunting reports)	Which specific query regressed, when, and which plan changed?
Event-driven, low overhead	Extended Events (the modern, supported replacement for SQL Trace/Profiler, which are deprecated)	Capturing specific, targeted events (deadlocks, blocked-process reports, long-running queries) continuously with minimal overhead
Community toolkit	Brent Ozar's First Responder Kit (sp_Blitz, sp_BlitzCache, sp_BlitzIndex, sp_BlitzFirst) and sp_WhoIsActive	A fast, opinionated health check across configuration, plan cache, indexing, and current activity in one pass

Whenever asked 'how would you find out if sqlservr.exe was really causing a CPU spike that happened in the past,' the precise, version-proof answer is the ring buffer (sys.dm_os_ring_buffers, filtered to RING_BUFFER_SCHEDULER_MONITOR), which retains roughly the last 4 hours of 1-minute CPU samples even with no monitoring tool actively running — while the best-practice recommendation for ongoing,

longer-than-4-hour visibility is still to run dedicated Perfmon counter logging or an APM tool continuously rather than relying on the ring buffer as a permanent solution.

21.3 The Azure-Native Monitoring Stack (PaaS)

- Azure SQL Database / Managed Instance — Query Performance Insight (a simplified, portal-native view of top resource-consuming queries), Azure SQL Analytics (a Log Analytics-based solution for fleet-wide monitoring across many databases/instances), and the same DMVs covered above remain queryable directly (`sys.dm_exec_requests`, `sys.dm_os_wait_stats`, etc. — they exist on PaaS too, just scoped to what the platform exposes).
- Azure Monitor metrics and alert rules (CPU percentage, DTU/vCore percentage, storage percentage, deadlocks, connection failures) with dynamic threshold support, so an alert can trigger off a deviation from the resource's own learned baseline rather than a single static number that doesn't fit every workload's normal daily/weekly pattern.
- Diagnostic settings route Azure SQL/MI/VM telemetry to a Log Analytics workspace, Storage Account (for long-term retention/compliance), or Event Hub (for streaming to a SIEM or custom processing pipeline) — this is also the standard destination for SQL Server Audit data (Chapter 11.4) on Azure-hosted SQL.

21.4 Alerting Strategy

36. Define what actually warrants paging a human at 2 a.m. versus what belongs in a daily digest — severity 19-25 SQL Server errors (fatal, often corruption-adjacent), AG health going `NOT_HEALTHY`, and a primary failover are page-worthy; a single transient deadlock or one slow query usually is not.
37. Configure SQL Server Agent alerts for specific error severities/numbers (sysadmin-defined, instance-level) as the first layer on IaaS/on-prem.
38. Layer Azure Monitor alert rules (with action groups routing to your on-call/incident-management tool) as the equivalent first layer on PaaS, since SQL Agent itself doesn't exist on Azure SQL Database.
39. Tie every page-worthy alert to a runbook (even a short one) so the first responder has a starting checklist rather than starting cold — the production scenario callouts throughout this handbook (Chapters 4, 6, 10) are written in exactly that runbook style deliberately.
40. Review and tune alert thresholds on a regular cadence — both alert fatigue (too many low-value pages) and missed incidents (thresholds too loose) erode trust in the monitoring system over time.

21.5 Interview Questions

Q9. How do you find out if `sqlservr.exe` was the real cause of a CPU spike that happened an hour ago, if no monitoring tool was actively running?

A: Query `sys.dm_os_ring_buffers` filtered to `RING_BUFFER_SCHEDULER_MONITOR` — it retains roughly the last 4 hours of 1-minute CPU utilization samples, split between SQL Server and all other processes, with zero setup required in advance. For ongoing visibility beyond that 4-hour window, the best practice is still dedicated Perfmon counter logging or an APM tool running continuously.

Q10. What replaced SQL Trace/Profiler as the supported way to capture targeted events like deadlocks or long-running queries?

A: Extended Events — lower overhead, more flexible targeting, and the actively maintained mechanism going forward; SQL Trace and Profiler are both deprecated.

Q11. How does dynamic-threshold alerting in Azure Monitor improve on a single static CPU% alert?

A: It learns each resource's own normal baseline pattern (which can vary meaningfully across a daily/weekly cycle) and alerts on meaningful deviation from that learned baseline, rather than firing on every brief, expected peak that crosses one fixed number — reducing both false positives during legitimate peak periods and missed slow-building anomalies that never cross a static threshold at all.

Q12. Design a complete alerting strategy spanning an on-prem Always On AG, a Managed Instance, and an Azure SQL Database, given a 24/7 on-call rotation.

A: On-prem AG: SQL Agent alerts for severity 19-25 and specific AG-health error numbers, plus an Extended Events session watching AlwaysOn_health for failover/lease-timeout events, routed to the on-call tool.

Managed Instance: Azure Monitor alert rules on CPU/IO/storage percentage and connection failures (dynamic thresholds where workload variability justifies it), plus diagnostic settings streaming to Log Analytics for post-incident query.

Azure SQL Database: the same Azure Monitor alerting layer, since there is no SQL Agent equivalent here at all — DTU/vCore percentage, storage percentage, and deadlock-count metrics are the primary signals, supplemented by Query Performance Insight for triage once an alert fires.

Across all three: every page-worthy alert links to a short runbook, and alert thresholds are reviewed on a recurring cadence to manage alert fatigue.

21.6 Quick Revision Notes

- Ring buffer (RING_BUFFER_SCHEDULER_MONITOR) = ~4 hours of free, no-setup-required CPU history.
- Extended Events = the supported replacement for deprecated SQL Trace/Profiler.
- Query Store = query-level regression hunting over time; ring buffer/Perfmon = instance/OS-level point-in-time and short-history.
- PaaS monitoring = Query Performance Insight + Azure SQL Analytics + Azure Monitor metrics/alerts (dynamic thresholds) + diagnostic settings, since there's no Perfmon/Agent layer underneath.
- Every page-worthy alert needs a runbook; review thresholds regularly to manage alert fatigue.

Part VIII — Real-World Practice

Theory and DMV syntax only carry an interview so far — the strongest Senior/Lead DBA candidates can narrate a real incident end to end, defend an architecture under follow-up questions, and describe how change actually gets governed in a regulated enterprise. This closing part is built entirely around that kind of narrative: structured incident case studies, a full healthcare reference architecture, and the patching/change-management discipline that wraps every technical runbook earlier in this handbook.

Chapter 22 • Production Incident Case Studies & Root Cause Analysis

22.1 How to Structure an Incident Answer in an Interview

Every case study below follows the same five-part structure — Symptom, Investigation, Root Cause, Fix, Prevention — because that is exactly the structure a strong interview answer should follow when you're asked to walk through a past incident. Several of these case studies reference the detailed technical mechanics already covered earlier in the handbook rather than repeating them, so each entry stays focused on the narrative judgment calls an interviewer is actually listening for.

22.2 Case Study 1 — Always On Secondary Stuck “Not Synchronizing” After a Routine Patch

Incident Summary

Symptom: Following a routine CU installation on a secondary replica (per the Chapter 4.7 runbook), the secondary came back online but stayed in a NOT SYNCHRONIZING state for over an hour instead of catching up within minutes as usual.

- **Investigation:** `sys.dm_hadr_database_replica_states` showed `redo_queue_size` climbing rather than draining; `sys.dm_os_wait_stats` on the secondary showed dominant `PAGEIOLATCH` waits rather than network-related waits, pointing at the redo thread itself rather than the log-transport path.
- **Root cause:** The patch's restart had dropped the secondary's buffer pool, and a backlog of changes accumulated on the primary during the patch window now had to be applied to a cold buffer pool with no cached pages — every redo operation was paying a physical read penalty it normally wouldn't.
- **Fix:** No corrective action was actually required beyond waiting and confirming the redo queue was trending down rather than up; the team additionally pre-warmed the buffer pool on the next patch cycle by running representative read queries against the secondary immediately after restart, before resuming data movement.
- **Prevention / action items:** Add 'confirm `redo_queue_size` is trending downward, not just non-zero' as an explicit checklist item in the patching runbook (Chapter 4.7), and consider scheduling patches for lower-traffic windows specifically to reduce the size of the post-restart redo backlog.

22.3 Case Study 2 — msdb Corruption With No Recent Backup

Incident Summary

Symptom: SQL Server Agent jobs stopped running instance-wide; the error log showed msdb-related consistency errors, and msdb itself would not come fully online. The most recent msdb backup was several months old due to a misconfigured maintenance plan.

- **Investigation:** `DBCC CHECKDB` against msdb (once it could be brought online enough to run it) confirmed corruption in system tables backing the job subsystem; restoring the stale backup would have lost months of job history/schedules.
- **Root cause:** A maintenance plan change months earlier had silently excluded msdb from the nightly system-database backup job, and nobody had validated that backup chain since.
- **Fix:** Started the instance in minimal configuration mode with the global trace flag enabling msdb detach (`net start MSSQLSERVER /T3608 /f /m"SQLCMD"`), detached the damaged msdb (`EXEC sp_detach_db msdb`), then attached a fresh, clean msdb copy (or rebuilt msdb via the SQL Server setup

REBUILDDATABASE option) and manually recreated the job/schedule definitions from a combination of the stale backup's metadata and the application teams' own documentation of their job schedules.

- Prevention / action items: Made system database backups (master, model, msdb) an explicitly monitored, alerted job separate from user-database backup monitoring, with a scheduled restore-test (Test-DbaLastBackup or equivalent) covering system databases specifically, not just user databases — see Chapter 19.5.

22.4 Case Study 3 — SUSPECT Database After an Unplanned Power Event

Incident Summary

Symptom: An unplanned datacenter power event left one user database in SUSPECT state on restart, while every other database on the instance recovered normally.

- Investigation: The error log showed the database's recovery process failed partway through redo, consistent with a damaged log file rather than a damaged data file; DBCC CHECKDB could not run against a SUSPECT database until it was moved into EMERGENCY mode (read-only, bypassing normal recovery).
- Root cause: The log file lived on a different, less-resilient storage volume than the data files, and that volume had taken a write-cache-related hit during the abrupt power loss that the data-file volume (on a different, battery-backed array) did not experience.
- Fix: Set the database to EMERGENCY mode, ran DBCC CHECKDB to scope the damage, attempted the least destructive repair option first, and only used REPAIR_ALLOW_DATA_LOSS as a last resort — immediately followed by a full row-count reconciliation against a known-good baseline (the prior night's backup, restored to a scratch instance) to quantify and document exactly what data, if any, had actually been lost.
- Prevention / action items: Moved the log file onto the same resilience tier of storage as the data files (or better, since the log is the more recovery-critical artifact), and added a tail-log-backup-capable monitoring check so any future SUSPECT event captures the last log records immediately rather than relying on whatever happened to already be backed up.

22.5 Case Study 4 — TempDB I/O Storm During a Marketing Campaign Launch

Incident Summary

Symptom: A flash-sale marketing campaign drove a 10x spike in concurrent sessions; within minutes, application timeouts spiked and the instance-wide wait profile was dominated by PAGELATCH waits.

- Investigation: The waits resolved to TempDB's PFS/GAM/SGAM pages (Chapter 10.2) rather than user-database pages — the campaign's checkout flow created several temp tables per transaction, and concurrency had outgrown the existing TempDB file configuration.
- Root cause: TempDB had been sized for the steady-state workload (4 files) years earlier and never revisited as transaction volume grew; it had not been resized to match either current core count or current peak concurrency.
- Fix: Immediately increased TempDB file count (matching logical core count, equally sized) and confirmed it took effect without a restart for additional files; for the remainder of the campaign window, also asked the application team to enable a feature flag that batched several checkout steps into fewer temp-table-creating round trips.
- Prevention / action items: Added peak-concurrency load testing (specifically targeting TempDB object-creation rates) to the pre-campaign readiness checklist, and set a recurring quarterly review of

TempDB file count against current core count and observed contention, rather than treating the original sizing as permanent.

22.6 Case Study 5 — Near Split-Brain During a Datacenter Network Blip

Incident Summary

Symptom: A brief network partition between two datacenters hosting a 3-node Node Majority cluster caused the AG to fail over unexpectedly, and one node briefly showed as isolated/down in cluster diagnostics even though the underlying server was healthy.

- Investigation: Cluster logs showed the node lost contact with the other two nodes for roughly 20 seconds — long enough to cross the cluster's heartbeat threshold — before connectivity restored; this was a transient network issue, not a node failure.
- Root cause: A change to the network team's QoS policy the previous week had deprioritized cluster heartbeat traffic relative to a new bulk-transfer workload sharing the same link, occasionally starving heartbeat packets during bursts.
- Fix: Worked with the network team to re-prioritize cluster heartbeat/cluster communication traffic above bulk transfer traffic on the shared link, and reviewed (without needing to invoke, since quorum was never actually lost — only one node was briefly isolated) the force-quorum runbook from Chapter 6.3 to confirm the team could execute it confidently if a future event did cross into actual majority loss.
- Prevention / action items: Added a recurring change-management checkpoint requiring any network QoS/routing change affecting datacenter interconnects to be reviewed against cluster heartbeat requirements before deployment, closing the gap that let this slip through unreviewed the first time.

22.7 Case Study 6 — Performance Cliff Immediately After a Managed Instance Cutover

Incident Summary

Symptom: A batch ETL job that ran in 40 minutes on-premises took over 3 hours on Managed Instance immediately after cutover, with no application code changes.

- Investigation: Ran the full post-migration performance checklist from Chapter 17.4 systematically rather than guessing at a single cause.
- Root cause: Two compounding root causes: the job relied heavily on minimally-logged bulk-insert operations under the source's BULK_LOGGED recovery model (now fully logged on MI's FULL-only recovery model, multiplying log volume), and the target had been provisioned on General Purpose tier when the source's bulk-load pattern needed Business Critical's local SSD I/O profile.
- Fix: Re-architected the bulk-load step to use smaller, more frequent batches with explicit transactions to control log growth per batch, and moved the workload to Business Critical tier for the I/O profile the bulk-load pattern actually needed.
- Prevention / action items: Added 'does this workload rely on minimally-logged bulk operations' as an explicit pre-migration assessment question for every future on-prem-to-MI migration, since FULL-only recovery on the target is easy to overlook until a bulk-heavy batch job exposes it in production.

22.8 Case Study 7 — Replication Subscriber Falling Behind During Month-End Batch

Incident Summary

Symptom: A reporting subscriber's data started lagging visibly behind the publisher every month-end, recovering on its own a day or two into the new month.

- Investigation: Inserted tracker tokens (Chapter 7.4) during the next month-end window and confirmed the delay sat specifically between the Distribution Agent's reader and writer threads, not between publisher and distributor.
- Root cause: Month-end batch jobs ran heavy reporting queries directly against the subscriber at the same time replication needed to apply its own heaviest monthly change volume, and those reporting queries were blocking the Distribution Agent's writer thread.
- Fix: Rescheduled the heaviest subscriber-side reporting queries to a window an hour after the publisher's month-end batch typically completed, giving the Distribution Agent a clear runway to catch up before reporting load resumed.
- Prevention / action items: Documented the month-end batch's typical replication volume profile and added a Replication Monitor latency alert specifically tuned to the month-end period, rather than relying on a single static latency threshold that didn't account for known cyclical volume.

22.9 Case Study 8 — Runaway Transaction Log Growth From an Abandoned Open Transaction

Incident Summary

Symptom: A database's log file grew from its normal ~20 GB steady state to over 400 GB overnight, eventually triggering a disk-space alert, with no obviously unusual batch job running.

- Investigation: `sys.dm_tran_database_transactions` and `sys.dm_exec_sessions` identified a single session with an open transaction over 14 hours old, originating from an application connection pool; the application code path had a bug that occasionally failed to commit or roll back a transaction after an unhandled exception, leaving the connection (and its transaction) idle-but-open in the pool.
- Root cause: Application-side defect, not a database-side misconfiguration — but the database-side impact (uncontrolled log growth, since the log cannot truncate past the oldest open transaction) is exactly what brought the incident to the DBA team's attention first.
- Fix: Killed the offending session (after coordinating with the application team to confirm it was safe to do so) to let the transaction roll back and log truncation resume, then ran a log backup immediately afterward to reclaim space without needing to shrink the file destructively.
- Prevention / action items: Worked with the application team to add a connection-pool-level idle-transaction timeout and exception-handling fix, and added a proactive Agent alert on “transaction open longer than N minutes” (querying `sys.dm_tran_active_transactions`) so the next occurrence pages the on-call team within minutes rather than being discovered only once the log filled the disk — and flagged Accelerated Database Recovery (Chapter 9.6) as worth enabling on this database specifically because of its history of long-running-transaction incidents.

22.10 Cross-Reference Index — Incidents Embedded Earlier in This Handbook

These production scenarios were developed in full where the surrounding technical chapter needed them as worked examples, rather than repeated here:

Incident	Location
Patching a multi-DC AG (suspend/patch/resume/failover runbook)	Chapter 4.7
Reporting queries blocking on a readable secondary (redo-thread contention)	Chapters 3.4 and 4.10

Force quorum after an ungraceful 2-of-3-node loss	Chapter 6.3
TempDB placement/sizing fix after an Azure VM migration (E32asv4 → E32bdsv5)	Chapter 10.4
MI Agent job failing on tcp:localhost,11000 (connection policy mismatch)	Chapter 14.2
1 TB int→bigint column widening via compression, in seconds instead of hours	Chapter 1.5 / Chapter 19.6

22.11 Quick Revision Notes

- Structure every incident answer as: Symptom → Investigation → Root Cause → Fix → Prevention.
- Investigation should cite the actual DMV/tool used, not just the conclusion — interviewers are listening for the evidence trail.
- Prevention items should be concrete and specific (a new alert, a new checklist line, a new test) — not just “be more careful.”
- Several of the strongest incident stories combine two compounding root causes (Case Studies 6 and 5 above) — be ready to describe an incident that wasn't a single simple cause.

Chapter 23 • Patching, Upgrades & Change Management

23.1 Concept Overview

This chapter is deliberately the governance layer wrapped around the detailed technical runbooks already covered — the AG patching sequence (Chapter 4.7), CU/SP/Hotfix terminology (Chapter 1.2), and the CI/CD review gate (Chapter 19.3). What's covered here is the policy and process discipline that decides when those technical steps get triggered, who approves them, and how the organization proves afterward that the change was controlled.

23.2 Patch Cadence Policy

- N-2 patch level (staying roughly two Cumulative Updates behind the very latest) is a common, defensible default — it gives the broader SQL Server community time to surface regressions in the newest CUs before they reach your production environment, at the cost of a short delay in receiving the newest fixes.
- Security-classified fixes are the explicit exception to N-2: a CU containing a fix for an actively-exploited or high-severity vulnerability should be evaluated and fast-tracked outside the normal cadence, with its own accelerated (but still tested) approval path.
- Every CU should be piloted in a lower environment for a defined minimum soak period (a week is a common baseline) with real application validation, not just “it installed successfully” — the SSISDB-inside-an-AG patching regression referenced in Chapter 4.7 is exactly the class of defect that soak testing exists to catch before production.

23.3 Pre-Patch Validation Checklist

41. Confirm a recent, verified-restorable backup exists for every database on the target instance (Test-DbaLastBackup or an equivalent restore-test, not just a backup-completed-successfully log entry).
42. Confirm AG/cluster health is fully green (sys.dm_hadr_availability_group_states, cluster validation) before starting — never begin a patch cycle against a topology that is already degraded.
43. Confirm sufficient free disk space for the patch installer's temporary files and for the redo/log-growth pattern noted in Case Study 1 (Chapter 22.2).
44. Confirm the rollback plan is concrete: for an in-place CU, this is typically “uninstall the CU” as a documented, tested fallback; for a VM-hosted instance, a pre-patch disk snapshot is an additional safety net most cloud platforms make cheap to take.
45. Confirm the change has gone through whatever approval process (Section 24.4) the organization requires, with a defined rollback decision point and owner.

23.4 Change Approval & Audit Trail

- A lightweight Change Advisory Board (CAB) process — even an asynchronous, ticket-based one rather than a standing meeting — gives a second set of eyes on blast radius and timing (e.g. confirming a patch window doesn't collide with month-end batch processing, directly relevant to Case Study 8's lesson about cyclical load) before the change executes.
- Every production change should be traceable end-to-end: a change ticket number linked to the CI/CD pipeline run (Chapter 19.3) that performed it, linked to the specific backup taken immediately beforehand, linked to the post-change validation evidence — this traceability is exactly what most compliance frameworks (SOC 2, HIPAA, PCI-DSS) expect an auditor to be able to follow.

- Emergency changes (an active security vulnerability, a production-down incident requiring an immediate fix) need a defined fast-path approval process that still captures the same audit trail after the fact, rather than no process at all under time pressure.

23.5 In-Place Upgrade vs. Side-by-Side Migration

Approach	Pros	Cons
In-place upgrade (run setup.exe to upgrade the existing instance)	Simpler, no new hardware/connection-string changes, preserves instance-level configuration automatically	No easy rollback once the upgrade completes; both old and new engine can't run side-by-side during validation
Side-by-side migration (new instance/server at the target version, migrate databases over)	Old environment remains untouched as a fallback during validation; lets you also change hardware/topology/edition at the same time	More complex cutover (connection-string/DNS changes), and is effectively a migration project in its own right — apply the Chapter 17 framework to choose the actual data-movement technique

Always On AG itself enables a third pattern that gets the best of both — the rolling AG upgrade described in Chapter 4.7: upgrade secondaries first (the AG explicitly supports mixed versions/editions across replicas to make this possible), fail over, then upgrade the former primary, giving you an in-place-equivalent result with a side-by-side-equivalent safety margin at each step.

23.6 Post-Patch Validation

- Application smoke tests covering the critical user journeys, not just “SQL Server service is running.”
- A performance baseline comparison (Query Store, or a saved Perfmon/DMV baseline from before the patch) to catch a regression that doesn't throw an error but quietly degrades a key workload.
- A full error-log review for the patch window itself, since some issues (like the redo-backlog pattern in Case Study 1) only show up as warnings rather than hard failures.
- Sign-off from the change ticket owner, formally closing the change record with the post-validation evidence attached.

23.7 Interview Questions

Q1. Why would an organization deliberately stay on N-2 CUs instead of always installing the latest?

A: It gives the broader SQL Server user base time to surface regressions in the newest CUs before they reach production, trading a short delay in receiving the newest fixes for materially lower regression risk — with an explicit fast-track exception for high-severity security fixes.

Q2. What's the difference between an in-place upgrade, a side-by-side migration, and a rolling AG upgrade, and when would you choose each?

A: In-place upgrades the existing instance directly (simple, but no easy rollback); side-by-side stands up a new instance/server at the target version and migrates databases over (safer fallback, but a full migration project); a rolling AG upgrade patches/upgrades secondaries first, fails over, then upgrades the former primary, getting an in-place-equivalent end state with a much safer, staged path — the preferred approach whenever the databases in question are already in an AG.

Q3. What belongs in a complete pre-patch validation checklist, beyond just 'check the AG is healthy'?

A: A verified-restorable (not just verified-completed) recent backup of every database on the target, confirmed free disk space for both the installer and the expected post-patch redo/log-growth pattern, a concrete and tested rollback plan, and formal change approval with a defined rollback decision owner — skipping any one of these is how a routine patch becomes an incident.

Q4. A regulator asks you to prove that a specific production schema change last quarter went through proper approval. What evidence chain do you produce?

A: The change ticket number, linked to the CI/CD pipeline run that executed the migration script (including the human-reviewed diff from the pull request, Chapter 19.3), linked to the backup taken immediately beforehand, and linked to the post-change validation evidence and ticket sign-off — full end-to-end traceability is exactly what this kind of audit request expects, and is the reason every change should be required to produce this chain at the time it happens rather than reconstructed after the fact.

23.8 Quick Revision Notes

- N-2 patch cadence as the default; security fixes are the explicit fast-track exception.
- Pre-patch checklist: verified-restorable backups, green AG/cluster health, disk space for redo growth, a tested rollback plan, formal approval.
- Rolling AG upgrade = secondaries first → failover → former primary — the safest pattern when AG is already in play.
- Every production change needs an end-to-end audit trail: ticket → pipeline run → backup → validation evidence → sign-off.
- Post-patch validation = application smoke tests + performance baseline comparison + full error-log review, not just “service is up.”

Appendices — Cheat Sheets & Rapid-Fire Review

Use this final section in the 24-48 hours before an interview: Appendix A is a consolidated, theme-based cheat sheet; Appendix B is 100 rapid-fire one-line questions and answers spanning every chapter; Appendix C is 50 real-time production scenarios with the immediate action and a cross-reference to the full treatment earlier in the handbook.

Chapter A · Quick Revision Cheat Sheets

A.1 Backup & Recovery Cheat Sheet

Topic	Key fact
Backup chain	Full → Differential(s) → Log(s); a COPY_ONLY full never breaks the chain, and nothing can restore on top of one except more logs
AG secondary backups	COPY_ONLY full + log only — never differential, never normal full
Tail-log backup	BACKUP LOG ... WITH NO_TRUNCATE (or WITH CONTINUE_AFTER_ERROR if damaged) before restoring a damaged/offline database
TempDB backup	Not possible — TempDB cannot be backed up or restored; it is recreated fresh from model on every restart
TRUSTWORTHY	Never survives a RESTORE — must be re-enabled explicitly
Azure SQL DB retention	Basic: 7 days. Premium/most vCore: up to 35 days + optional LTR
MI native backups	Copy-only full + log only; automated platform backups run independently

A.2 Always On / HA / DR Cheat Sheet

Topic	Key fact
AG limits	Up to 8 secondaries, max 5 synchronous
Default connection_timeout	10 seconds
Default secondary isolation	SNAPSHOT, forced
DAG	AG of AGs via a Forwarder; independent WSFCs; cluster-less from 2017+
Quorum	Majority side survives; Dynamic Quorum/Witness = last-man-standing, graceful losses only
Ungraceful majority loss	/forcequorum + FORCE_FAILOVER_ALLOW_DATA_LOSS + per-DB RESUME
CNO / VCO	Cluster's AD object / Listener's AD object — both need explicit OU rights, not just sysadmin
Patching order	Secondary → manual failover mode → suspend → patch → resume → failover → patch old primary

A.3 Performance Cheat Sheet

Topic	Key fact
Histogram	Max 201 steps, leading column only
Variable estimates	Density vector, never the histogram
nvarchar vs varchar	varchar→nvarchar free; nvarchar column compared to varchar value = implicit conversion + CPU + SOS_SCHEDULER_YIELD
TempDB contention	PFS/GAM/SGAM pages = allocation (I/O-bound); higher-numbered catalog pages = metadata (memory-bound)
Lock escalation	Jumps row/page → table directly, ~5,000-lock threshold per statement
Forwarded fetches	Heap-only; fix with clustered index or ALTER TABLE ... REBUILD
ADR (2019+)	Persisted Version Store decouples rollback/recovery time from transaction size

A.4 Security Cheat Sheet

Topic	Key fact
TDE chain	SMK → DMK → Certificate → DEK → data/log pages
Column encryption vs TDE	Column-level survives restore without exporting keys; TDE certificate must travel with the database
Always Encrypted	SQL Server (and DBAs) never see plaintext at all
AD group ownership	Never allowed for a job owner or a database owner — needs one resolvable principal
SQL Server 2025 security	Managed Identity (credential-free), PBKDF2 hashing, TDS 8.0/TLS 1.3 by default, deeper Entra ID/Zero Trust

A.5 Azure Cheat Sheet

Topic	Key fact
Azure SQL DB gaps	No native backups, no SQL Agent, no cross-database queries, recovery model always FULL
MI tiers	GP = remote storage; BC = local SSD + 4 built-in replicas
MI connectivity	Proxy (1433 only) vs Redirect (11000-11999, recommended) vs Public Endpoint (3342/3343)
MI defaults	12 TempDB data files + 1 log file; TDE auto-on; FULL recovery only

Migration tool selection	<2016 + many DBs → LRS; 2016+/5-min downtime → MI Link (DAG); standalone/cross-version → manual DAG
Login migration	MI → sp_help_revlogin + Azure DMS; Azure SQL DB → recreate or move to Entra ID

Chapter B · Top 100 Interview Questions — Rapid Fire

One-line questions, one-line answers — for cramming, not for learning the concept the first time. Full explanations for every item below live in the cross-referenced chapter.

Fundamentals & Backup/Recovery (1-15)

#	Question	Answer
1	What is a SQL Server build?	The specific internal version/patch number of the engine binaries
2	Difference between hotfix, CU, and Service Pack?	Hotfix = narrow fix; CU = tested cumulative rollup (the modern model); SP = fully tested cumulative bundle (discontinued after 2017)
3	What is the Resource database?	Hidden, read-only DB holding every system object shipped with the engine
4	What is the -f startup parameter for?	Minimal configuration/single-user mode, used for disaster recovery (e.g. detaching a corrupt msdb)
5	Can you detach a SUSPECT database?	Not since SQL Server 2008 (it was allowed through 2005)
6	Can you detach msdb?	Not by default; only after starting in minimal config mode with TF3608
7	Can you back up or restore TempDB?	No — it is recreated fresh from model every restart
8	What is a tail-log backup?	The final log backup of a damaged/offline database, taken with NO_TRUNCATE or CONTINUE_AFTER_ERROR
9	What is the difference between Primary Key and Unique Key?	Both enforce uniqueness; a table can have only one Primary Key (which also disallows NULLs) but multiple Unique Keys (which allow one NULL each)
10	What is a heap table?	A table with no clustered index — rows have no enforced physical order
11	Can a table be created on a single filegroup?	Yes — specify ON [FilegroupName] in the CREATE TABLE statement

12	What should you consider when adding a new column with a default?	Adding a nullable column, or a non-null column with a constant default, is a fast metadata-only operation on 2012+; a non-null column with a non-constant/computed default forces a full table rewrite
13	Name three SQL Server 2019 highlights.	Accelerated Database Recovery, Always Encrypted with secure enclaves, memory-optimized TempDB metadata (plus intelligent query processing additions like table variable deferred compilation)
14	What does DBCC CHECKDB WITH ESTIMATEONLY do?	Reports the TempDB space CHECKDB would need, without running the actual checks
15	What's logged in the error log about CHECKDB on every restart?	Only the date CHECKDB last completed successfully — the message does not mean CHECKDB just ran

Locking, Blocking & Concurrency (16-22)

#	Question	Answer
16	Difference between locking, blocking, and deadlock?	Locking = mechanism; blocking = normal wait; deadlock = unresolvable cycle, auto-killed
17	What trace flag captures deadlocks in the error log?	1222 (detailed); legacy 1204; system_health XEvents captures them with no flag needed
18	Lock escalation path?	Row/page → table directly, never row → page
19	How do you find the head blocker?	sys.dm_exec_requests.blocking_session_id or sp_WhoIsActive
20	What does HADR_SYNC_COMMIT high waits indicate?	Slow I/O, CPU, or network on the synchronous secondary — not the primary
21	Narrow vs wide plan for an UPDATE?	Narrow = per-row update of all indexes; wide = sorts NCI changes in memory first, per-index
22	Does MSDTC need its own storage on an FCI?	Only if configured as its own clustered resource; SQL can also use local MSDTC

Always On AG (23-38)

#	Question	Answer
23	Max secondaries / max synchronous?	8 total, up to 5 synchronous
24	Default AG connection_timeout?	10 seconds
25	Default isolation on secondary?	SNAPSHOT, forced
26	What is CNO / VCO?	Cluster Name Object / Virtual Computer Object — AD prestaging for the cluster and the Listener
27	Backups allowed on a secondary?	COPY_ONLY full + log only
28	Function to check if replica is primary?	sys.fn_hadr_is_primary_replica('DBName')
29	What does TRUSTWORTHY do on RESTORE?	Always resets to OFF, must re-enable manually
30	Patching order for an AG?	Secondaries first (manual failover, suspend, patch, resume), then failover, then old primary
31	Steps to add a database file in an AG?	ALTER DATABASE ADD FILE on primary; auto-creates on secondaries if paths match
32	Key DMVs for AG health?	dm_hadr_availability_group_states, dm_hadr_availability_replica_states, dm_hadr_database_replica_states
33	What is a DNN?	Distributed Network Name — DNS-based redirect with no dedicated Listener VIP
34	Default backup priority tie-break?	Alphabetical among equal-priority eligible replicas
35	Where are default stats for secondary ad hoc queries stored?	TempDB (the secondary DB itself is read-only)
36	TempDB row-versioning overhead from readable secondaries?	~14 bytes per row for DML
37	Event ID for AG failover history?	1480, inside AlwaysOn_health Extended Events
38	Contained AG (2022+) solves what?	Automatic sync of logins/permissions/jobs across replicas

Distributed AG, Clustering & Quorum (39-48)

#	Question	Answer
---	----------	--------

39	What is the Forwarder in a DAG?	AG-2's primary, which redistributes the single log stream from AG-1's primary
40	Can DAG run cluster-less?	Yes, from SQL Server 2017+, even between standalone instances
41	DAG + DNN underneath — supported?	No
42	What is last-man-standing?	Dynamic Quorum allowing a cluster to run on a single surviving node after graceful node losses
43	Command to force quorum?	net start clussvc /forcequorum
44	Command to force AG failover with data loss?	ALTER AVAILABILITY GROUP ... FORCE_FAILOVER_ALLOW_DATA_LOSS
45	Active-active vs active-passive cluster?	Active-active = both nodes run their own instance simultaneously; active-passive = one idle standby
46	How do clients connect to a clustered SQL Server?	Always via the virtual network name/IP, never a physical node name
47	What is a File Share Witness?	An SMB-share quorum vote, often superseded by Cloud Witness for geo-dispersed clusters
48	Does MI support traditional Always On AG you configure yourself?	No — built-in HA + Auto-Failover Groups instead

Replication, Log Shipping & Mirroring (49-56)

#	Question	Answer
49	Can mirroring use bulk-logged recovery?	No — FULL recovery only
50	Can log shipping use bulk-logged recovery?	Yes
51	Publisher version vs Distributor version rule?	Publisher ≤ Distributor, always
52	Oracle as a replication publisher?	Technically possible but deprecated — use CDC + SSIS instead
53	Four replication threads?	LogReader-Reader, LogReader-Writer, DistAgent-Reader, DistAgent-Writer
54	How to pinpoint replication latency location?	Tracker tokens via sp_posttracertoken

55	Are NCIs replicated to subscribers by default?	No — create them manually on the subscriber if needed
56	Replication types supported on MI?	Snapshot and uni/bi-directional transactional only — no merge

Performance, Indexing & Query Optimization (57-70)

#	Question	Answer
57	Covering index vs included column?	Covering = the overall property; INCLUDE = leaf-level-only mechanism to achieve it
58	Histogram step limit?	201, built only on the leading column
59	MSTVF row estimate evolution?	1 row (≤ 2012) $\rightarrow$ 100 rows (2014-16) $\rightarrow$ accurate (2017+ via interleaved execution)
60	Table variable row estimate evolution?	1 row until 2017; deferred compilation (2019+) gives a real count
61	Track index rebuild progress?	sys.dm_exec_requests.percent_complete
62	What causes a forwarded fetch?	A heap row outgrowing its page and being relocated, leaving a forwarding pointer
63	REBUILD vs REORGANIZE?	REBUILD = drop/recreate + refresh stats; REORGANIZE = in-place, lighter, always online
64	3 modes of dm_db_index_physical_stats?	LIMITED (default), SAMPLED, DETAILED (avoid in business hours on huge tables)
65	Why does Hash Join appear?	Usually no useful index on the join column(s); needs QE memory for the hash table
66	Cost Threshold for Parallelism default?	5
67	Get the actual plan straight from cache?	Enable LAST_QUERY_PLAN_STATS (2019+), query sys.dm_exec_query_plan_stats
68	Best scalar UDF fix?	SCHEMABINDING (often enabling auto-inlining), or rely on automatic Scalar UDF Inlining (2019+)
69	What is Accelerated Database Recovery?	2019+ feature decoupling rollback/recovery time from transaction size via the Persisted Version Store
70	Is disabling parameter sniffing good practice?	No — it forces density-vector-based average estimates for every execution

TempDB (71-76)

#	Question	Answer
71	Allocation vs metadata contention?	PFS/GAM/SGAM pages (I/O-bound) vs system catalog pages (memory-bound)
72	TF 1117/1118 needed on 2016+?	No — built-in default behavior since SQL Server 2016
73	TempDB file rule of thumb?	~1 equally-sized file per logical CPU, up to 8
74	MI default TempDB file count?	12 data files + 1 log file
75	Best TempDB placement on an Azure VM?	Local ephemeral disk (commonly D:), not remote persistent storage
76	2025 TempDB feature?	Resource governance/sandboxing of a session's TempDB and sort-spill footprint

Security & Identity (77-84)

#	Question	Answer
77	TDE key chain?	SMK → DMK → Certificate → DEK → data/log pages
78	Multiple passwords on a DMK?	Yes; not possible on the Service Master Key
79	Column encryption vs Always Encrypted?	Column-level still decrypts inside the engine; Always Encrypted never exposes plaintext to SQL Server at all
80	Can a job owner be an AD group?	No
81	Can dbo be an AD group?	No
82	sp_change_users_login finds Windows orphans?	No — only SQL login orphans; use sp_validatelogins for Windows orphans
83	What does TRUSTWORTHY gate?	Cross-database ownership chaining
84	SQL Server 2025 password hashing?	PBKDF2, replacing the older SHA-based scheme

Azure SQL Database, MI & Networking (85-94)

#	Question	Answer
85	Azure SQL DB recovery model?	Always FULL — cannot be changed
86	Azure SQL DB retention?	Basic 7 days; Premium/most vCore up to 35 days + optional LTR

87	BACPAC vs DACPAC?	BACPAC = schema+data; DACPAC = schema only
88	MI default connection policy ports?	Proxy = 1433 only; Redirect = 11000-11999 (recommended)
89	MI Public Endpoint ports?	3342 management, 3343 data
90	GP vs BC storage?	GP = remote storage; BC = local SSD + 4 built-in replicas
91	Does Azure SQL DB have its own VNet?	No — use Private Endpoint or VNet Service Endpoints
92	Two ways to connect Azure VNets?	VNet Peering (intra-Azure) or VNet Gateways (hybrid/on-prem)
93	RAM per vCore on MI (approx.)?	Standard ~5.1GB, Premium ~7GB, Premium memory-optimized ~13.6GB
94	Can MI blend publisher/distributor/subscriber across on-prem and cloud?	No — both must be local to MI, or both remote

Migration, Fabric & 2025/2026 (95-100)

#	Question	Answer
95	Best tool: <2016 source, 50+ DBs, minimal downtime?	Log Replay Service
96	Best tool: 2019 source, <5 min downtime?	Azure Managed Instance Link (Distributed AG-based)
97	Why no traditional log shipping to/from MI?	RESTORE FROM URL implicitly applies WITH RECOVERY, closing further log restores
98	SQL Server 2025 GA date?	Mid-November 2025
99	What does Fabric Mirroring need on the source (SQL Server 2025)?	Azure Arc enablement + an on-prem/VNet data gateway
100	Does Fabric Mirroring replace an AG reporting secondary?	No — complementary; AG secondary for transactional reads, Fabric for analytics/BI

Chapter C · Top 50 Real-Time Production Scenarios

Scenario, immediate action, and where the full treatment lives in this handbook.

HA / DR / Clustering (1-12)

#	Scenario	Immediate action → reference
1	Secondary replica stuck NOT SYNCHRONIZING after a patch	Check redo_queue_size trend, confirm not still draining a cold-buffer-pool backlog → Ch.4.10, 22.2
2	Read-only app can reach primary but not secondary	Check ApplicationIntent=ReadOnly, routing list, ALLOW_CONNECTIONS → Ch.4.10
3	2 of 3 cluster nodes lost simultaneously	/forcequorum on survivor, FORCE_FAILOVER_ALLOW_DATA_LOSS, resume on rejoin → Ch.6.3
4	Reporting queries blocking on a readable secondary	Diagnose redo-thread vs reporting session contention; shorten transactions → Ch.3.4, 4.10
5	AG patch fails because SSISDB is in the AG	Known CU regression class; stay N-2, soak-test in lower env first → Ch.4.7, 24.2
6	Both AG replicas restart simultaneously (graceful)	Normal crash recovery; verify auto-resume, manually RESUME if needed → Ch.4.10
7	Brief network blip triggers unexpected AG failover	Check cluster heartbeat QoS prioritization on the interconnect → Ch.22.6
8	Migrate a 4TB standalone DB 2017→2019, <1 min downtime	Cluster-less Distributed AG, automatic seeding, instant failover cutover → Ch.5.4
9	AG creation fails despite sysadmin rights	Missing CNO/VCO AD prestaging permissions, not a SQL-side issue → Ch.4.3
10	DAG layered on an AG using a DNN listener	Unsupported combination — redesign without DNN underneath → Ch.5.3
11	Global Forwarder backup job fails for differentials	Forwarder only supports copy-only full + log backups → Ch.5.3
12	Log truncation delayed, disk filling during a prolonged replica outage	Resume the replica quickly or remove it from the AG to unblock truncation → Ch.6.3

Performance & TempDB (13-24)

#	Scenario	Immediate action → reference
13	PAGELATCH waits on tempdb 2:X:1 / 2:X:2 / 2:X:3	Object allocation contention — verify file count/sizing, enable memory-optimized metadata → Ch.10.2
14	Sudden 10x concurrency spike during a flash sale	Scale TempDB files immediately, batch app-side temp-table usage → Ch.22.5

15	Query plan shows a scan despite an index existing	Check for a non-SARGable predicate (function wrapping the column) → Ch.8.4
16	1TB INT column needs to become BIGINT in seconds	Enable page/row compression on table + all NCIs, then ALTER COLUMN (metadata-only) → Ch.1.5
17	High CPU + SOS_SCHEDULER_YIELD waits on a WHERE clause	Check for nvarchar column compared to a varchar literal/parameter → Ch.8.4
18	MSTVF-heavy procedure degrades as data grows	Pre-2017 fixed row-count estimate; check compatibility level for interleaved execution → Ch.8.3
19	Batch job 5x slower the night after a large rollback	Check whether Accelerated Database Recovery is enabled → Ch.9.6
20	Need to track an index rebuild's live progress	Query sys.dm_exec_requests.percent_complete → Ch.8.6
21	Heap table scans slow down over time with no fragmentation alert	Check forwarded_record_count; rebuild the heap or add a clustered index → Ch.8.5
22	Need proof sqlservr.exe caused a CPU spike an hour ago	Query sys.dm_os_ring_buffers (RING_BUFFER_SCHEDULER_MONITOR) → Ch.21.2
23	Need LEGACY_CARDINALITY_ESTIMATION across thousands of DBs	TF 9481 (2014) or a sys.databases loop toggling the DB-scoped config (2016+) → Ch.8.3
24	Post-migration to Azure VM, tempdb I/O warnings appear	Check file placement (local D: drive), file count, and VM SKU storage throughput → Ch.10.4, 15.4

Backup, Corruption & Recovery (25-32)

#	Scenario	Immediate action → reference
25	msdb corrupted, no recent backup, Agent jobs stopped	Minimal config mode (-f /T3608), detach/rebuild msdb, recreate jobs → Ch.22.3
26	A database is SUSPECT after an unplanned power event	EMERGENCY mode → CHECKDB → least-destructive repair → row-count audit if REPAIR_ALLOW_DATA_LOSS used → Ch.1.5, 22.4
27	Need to restore a database that's currently in an AG	Remove from AG → restore → re-add/re-seed → Ch.4.6
28	Differential restore fails on top of a copy-only full	Not supported — only a true (non-copy-only) full can serve as the differential base → Ch.4.6
29	TRUSTWORTHY-dependent cross-DB procs break after a restore	Expected — re-enable TRUSTWORTHY deliberately if truly required → Ch.4.6, 12.4
30	Need to run CHECKDB on a 30-40TB database with limited downtime	Bucket tables across the week; PHYSICAL_ONLY weekdays, EXTENDED_LOGICAL_CHECKS weekends; consider checks on a secondary → Ch.1.5

31	Log file ballooning overnight with no obvious batch job running	Find the open transaction via <code>sys.dm_tran_active_transactions</code> ; kill/roll back; log backup to reclaim space → Ch.22.9
32	TDE-encrypted DB fails to seed onto a new AG secondary	Certificate/private key not yet restored into that replica's master → Ch.4.8, 11.2

Security & Identity (33-38)

#	Scenario	Immediate action → reference
33	Need last-login evidence on SQL Server 2008 Standard (no Audit)	Logon trigger, server-side trace, or scheduled <code>sp_whoisactive</code> snapshot job → Ch.11.5
34	Compliance requires DBAs can't view a sensitive column at all	Always Encrypted, not column-level encryption alone → Ch.11.3
35	Audit finds a database owned by sa with TRUSTWORTHY ON	Re-own to a least-privilege principal, replace chaining with explicit grants, then disable TRUSTWORTHY → Ch.12.7
36	Restoring a DB with column-level encrypted data to a new server	Works without exporting keys; app fails only if it lacks the decryption key/Key Vault access → Ch.4.13, 11.3
37	Need to clean up unused SQL logins	Capture login activity over a representative window via triggers/XEvents/scheduled snapshots, cross-reference → Ch.11.5
38	AD team asks why the cluster/Listener creation failed	Missing Create Computer Objects / List Contents / Read All Properties / Read Permissions on the OU → Ch.4.3

Azure & Migration (39-50)

#	Scenario	Immediate action → reference
39	MI Agent job fails: 'Unable to connect to tcp:localhost,11000'	Connection policy mismatch — verify Proxy vs Redirect and port reachability → Ch.14.2
40	Need daily Dev refresh from Prod, both on MI	Copy-only backup to Blob → drop Dev → restore from Blob (no in-place overwrite on MI) → Ch.14.4
41	MI backup won't restore onto a SQL Server 2019 VM	Unsupported cross-version-down restore; use BACPAC/DACPAC, wizard, or replication instead → Ch.14.4, 17.6
42	Performance degraded after on-prem → MI migration	Run the full checklist: recovery model, TDE, storage tier, config drift, log cadence, TLS overhead → Ch.17.4
43	Need to migrate 80 databases from SQL Server 2014 with minimal downtime	Log Replay Service → Ch.5.4, 17.2
44	Need <5 minutes downtime migrating a 2019 database to MI	Azure Managed Instance Link (Distributed AG under the hood) → Ch.5.4, 17.2

45	Need to transfer SQL logins to Managed Instance at scale	sp_help_revlogin scripts + Azure Database Migration Service → Ch.12.5, 17.6
46	Need to transfer SQL logins to Azure SQL Database	No native carry-over — recreate with new passwords, or move to Entra ID → Ch.12.5
47	Team wants to drop/restore an Azure SQL DB every weekend to save cost	Technically possible within the 7+ day PITR window, but weigh restore latency/risk first → Ch.13.2
48	Need cross-database queries on Azure SQL Database	Not directly supported — use Elastic Query/External Tables, or reconsider MI → Ch.13.3
49	Need near-real-time analytics off a SQL Server 2025 AG without adding reporting load	Microsoft Fabric Mirroring into OneLake — watch log growth on long-running source transactions → Ch.18.2
50	24/7 critical workload needs near-zero-downtime cloud migration	Distributed AG / Managed Instance Link cutover, not backup-restore → Ch.5

Closing Note

For professionals with 5–15 years of experience, technical expertise is only one part of interview success. Employers look for DBAs who can confidently handle real-world challenges, make informed decisions, troubleshoot critical issues, and contribute to the overall reliability and performance of database systems.

As you prepare, focus on strengthening your understanding of core concepts, revisiting practical scenarios from your experience, and communicating your solutions clearly and confidently. Interviewers value candidates who can demonstrate both technical depth and the ability to apply their knowledge in production environments.

Keep learning, stay updated with evolving SQL Server technologies, and approach every interview as an opportunity to showcase your skills and gain new insights. Consistent preparation and a positive mindset will significantly improve your chances of success.

We hope this handbook has helped you organize your preparation and build confidence for your upcoming interviews.

Wishing you all the best in your SQL Server DBA career journey and future opportunities.